Curso: PostgreSQL Profesional

Duración sugerida: 40 horas (teoría + práctica)
Nivel: Intermedio a Avanzado
Requisitos: Conocimientos básicos de SQL y bases de datos relacionales.

Módulo 1: Arquitectura de PostgreSQL (6 horas)

1.1. Introducción a la arquitectura

- Modelo cliente/servidor
- Procesos postmaster, background writer, WAL writer, autovacuum, stats collector
- Memoria compartida: shared buffers, wal buffers, work_mem, maintenance_work_mem

1.2. Estructura de almacenamiento

- Tablespaces físicos y lógicos
- Estructura de una base de datos: OID, pg_class, pg_attribute
- Segmentos, extensiones, bloques (8 KB por defecto)
- TOAST (The Oversized-Attribute Storage Technique)

1.3. Índices internos

- B-Tree, Hash, GIN, GiST, BRIN, SP-GiST
- Cuando usar cada tipo (ejemplos prácticos)

1.4. MVCC (Multiversion Concurrency Control)

- Cómo PostgreSQL maneja concurrencia sin bloqueos de lectura
- xmin, xmax, cid, ctid
- Visibilidad y limpieza de tuplas muertas (dead tuples)

Ejercicio: Identificar procesos del sistema, ver configuración de memoria y crear un tablespace.

Módulo 2: Mantenimiento y Administración (8 horas)

2.1. Configuración del servidor (postgresql.conf, pg_hba.conf)

- Parámetros críticos: shared_buffers, effective_cache_size, max_connections
- Autovacuum: umbrales, escala, workers
- Logging: log_statement, log_min_duration_statement

2.2. Vacuum y Analyze

- VACUUM, VACUUM FULL, VACUUM FREEZE
- ANALYZE y estadísticas del planner
- Monitoreo de tablas con pg_stat_user_tables

2.3. Reindexación y bloat

- REINDEX, REINDEX CONCURRENTLY
- Detectar bloat en índices y tablas

2.4. Backup y recuperación

- pg_dump / pg_dumpall (lógicos)
- PITR (Point-In-Time Recovery) con WAL archiving
- pg_basebackup y replicación física

2.5. Monitoreo y tuning

- pg_stat_activity, pg_locks, pg_stat_bgwriter
- Extensiones: pg_stat_statements, auto_explain
- Identificar consultas lentas y cuellos de botella

Proyecto práctico: Configurar autovacuum óptimo, detectar dead tuples, hacer backup full + incremental.

Módulo 3: Consultas Básicas y Fundamentos SQL (6 horas)

3.1. Repaso de SELECT, INSERT, UPDATE, DELETE

- WHERE, ORDER BY, LIMIT, OFFSET
- JOIN: INNER, LEFT, RIGHT, FULL, CROSS, NATURAL
- Subconsultas escalares, de fila, de tabla, correlacionadas

3.2. Filtros avanzados

- ANY, ALL, EXISTS, IN, BETWEEN, LIKE, ILIKE
- Operadores de rango y JSON (->, ->>, @>)

3.3. Restricciones de integridad

- PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK, EXCLUDE
- ON DELETE CASCADE, SET NULL, etc.

Ejercicios: Crear esquema de ventas, clientes, productos. Realizar JOINs complejos y subconsultas.

Módulo 4: Funciones de Agregación (4 horas)

4.1. Agregaciones estándar

- COUNT, SUM, AVG, MIN, MAX, STRING_AGG, ARRAY_AGG
- GROUP BY con múltiples columnas y expresiones

4.2. Filtrado sobre agregaciones

- HAVING vs WHERE
- Agregaciones con condiciones (FILTER clause)

sql

```
SUM(venta) FILTER (WHERE fecha >= '2024-01-01')
```

4.3. GROUPING SETS, ROLLUP, CUBE

- Subtotales y totales múltiples
- GROUPING() para identificar filas de subtotal

Ejemplo práctico:
Calcular ventas totales, por mes, por categoría, y subtotales jerárquicos.

Módulo 5: Funciones de Ventana (Window Functions) (6 horas)

5.1. Conceptos básicos

- PARTITION BY, ORDER BY dentro de OVER()
- ROW_NUMBER(), RANK(), DENSE_RANK()
- LAG(), LEAD(), FIRST_VALUE(), LAST_VALUE()

5.2. Marcos (frames)

- ROWS vs RANGE vs GROUPS
- BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
- Ejemplos: promedio móvil, acumulados, delta entre filas

5.3. Agregaciones como ventana

- SUM(...) OVER(PARTITION BY ...)
- Comparación con GROUP BY tradicional

5.4. Casos de uso reales

- Ranking de ventas por región
- Diferencia de días entre compras de un cliente
- Porcentaje del total por categoría

- Detectar duplicados con ROW_NUMBER()

Ejercicio intensivo:
Sobre tabla de transacciones financieras, calcular saldo acumulado, ranking mensual, comparación mes anterior.

Módulo 6: Consultas Avanzadas y Optimización (6 horas)

6.1. CTE (Common Table Expressions)

- WITH ... AS (simple y múltiple)
- CTE recursivas (grafos, árboles, series numéricas)

sql

```
WITH RECURSIVE empleados_arbol AS (...)
```

6.2. Consultas sobre JSON y JSONB

- jsonb_path_query, jsonb_set, jsonb_agg
- Índices GIN sobre JSONB

6.3. Full Text Search

- to_tsvector, to_tsquery, @@
- Índices GIN para búsqueda textual

6.4. Optimización de consultas

- EXPLAIN, EXPLAIN ANALYZE, BUFFERS
- Interpretar node types: Seq Scan, Index Scan, Bitmap Heap Scan, Nested Loop, Hash Join, Merge Join
- Cómo forzar o evitar tipos de join (enable_* params)

Proyecto final integrador:
Sobre una base de datos de 1M de registros (ej. ventas, logística):

1. Diseñar arquitectura de índices
2. Escribir consultas con agregación y ventanas
3. Optimizar una consulta lenta usando EXPLAIN
4. Programar mantenimiento (vacuum, reindex)
5. Generar reportes con CTE recursiva (ej. jerarquía de productos)

Material complementario

- **Dataset de práctica:** DVD Rental (base de ejemplo oficial de PostgreSQL)

- **Herramientas:** psql, DBeaver, pgAdmin, DataGrip
- **Extensiones a explorar:** pg_stat_statements, PostGIS (geoespacial), pg_cron
- **Lectura recomendada:**
- *PostgreSQL 16 Administration Cookbook* (Giannini, Riggs)
- Documentación oficial: https://www.postgresql.org/docs/current/

¿Quieres que desarrolle alguna lección con ejercicios y soluciones específicas (por ejemplo, solo el módulo de funciones de ventana o el de arquitectura)?

desarrollame del modulo 1 el apartado 1.1 y 1.2

A continuación tienes el desarrollo completo de los apartados **1.1 Introducción a la arquitectura** y **1.2 Estructura de almacenamiento** del Módulo 1, con explicaciones detalladas, diagramas conceptuales, ejemplos prácticos y ejercicios.

Módulo 1: Arquitectura de PostgreSQL

1.1 Introducción a la arquitectura

Modelo cliente/servidor

PostgreSQL opera bajo el modelo **cliente/servidor** basado en procesos (no en hilos). Cada conexión de cliente recibe su propio proceso del sistema operativo.

text

```
[Cliente 1] → [postmaster] → [postgres (backend process 1)] → [Shared Memory]
[Cliente 2] → [postmaster] → [postgres (backend process 2)] → [Shared Memory]
[Cliente 3] → [postmaster] → [postgres (backend process 3)] → [Shared Memory]
```

Características clave:

- Aislamiento entre conexiones (fallo de un proceso no afecta a otros)
- Ventajas sobre hilos: más robusto, mejor uso de múltiples CPUs
- Desventaja: mayor consumo de recursos por conexión

Ciclo de una consulta:

1. Cliente envía consulta por red (TCP/IP o socket Unix)
2. Proceso backend recibe, analiza sintácticamente y semánticamente
3. Optimizador genera plan de ejecución
4. Ejecuta accediendo a disco o caché (shared buffers)
5. Retorna resultado al cliente

Procesos fundamentales

Cuando ejecutas `pg_ctl start` o `systemctl start postgresql`, se inicia el proceso **postmaster** (proceso padre), que luego lanza los siguientes procesos:

1. Postmaster (PID más bajo)

- Escucha conexiones entrantes (puerto 5432 por defecto)
- Autentica usuarios (pg_hba.conf)
- Crea procesos backend para cada conexión aceptada
- Supervisa y reinicia procesos hijos si mueren

2. Background Writer (bgwriter)

- Escribe páginas sucias (modificadas) de shared buffers a disco
- Objetivo: reducir la carga durante checkpoints
- Configuración: `bgwriter_delay`, `bgwriter_lru_maxpages`

3. WAL Writer

- Escribe registros WAL (Write-Ahead Log) desde el buffer WAL a disco
- Garantiza durabilidad (propiedad D de ACID)
- Frecuencia: cada `wal_writer_delay` (default 200ms)

4. Autovacuum Launcher + Workers

- Detecta tablas con muchas tuplas muertas (dead tuples)
- Lanza workers que ejecutan VACUUM y ANALYZE automáticamente
- Parámetros: `autovacuum_vacuum_threshold`, `autovacuum_scale_factor`

5. Checkpointer

- Escribe todos los buffers sucios al disco en momentos determinados
- Actualiza el archivo `pg_control` con el último checkpoint
- Frecuencia controlada por `checkpoint_timeout` (default 5 min) y `max_wal_size`

6. Stats Collector

- Recolecta estadísticas de actividad: número de escaneos de tablas/índices, tuplas insertadas/actualizadas/eliminadas, bloqueos, etc.`pg_default` → usa el directorio base (`$PGDATA/base/`)
- `pg_global` → almacena catálogo del sistema (`$PGDATA/global/`)

- Expone datos en vistas como `pg_stat_user_tables`, `pg_stat_database`, `pg_stat_bgwriter`

Ver todos los procesos en Linux:

bash

```
ps aux | grep postgres
```

Ejemplo de salida:

text

```
postgres  1234  0.0  0.5  postmaster
postgres  1235  0.0  0.1  postgres: walwriter
postgres  1236  0.0  0.1  postgres: autovacuum launcher
postgres  1237  0.0  0.1  postgres: background writer
postgres  1238  0.0  0.1  postgres: checkpointer
postgres  1250  0.0  0.8  postgres: user1 dbname 192.168.1.100(5432) idle
```

Memoria compartida (Shared Memory)

PostgreSQL usa memoria compartida (System V o POSIX) para comunicación eficiente entre procesos.

Área de memoria	Descripción	Parámetro principal
Shared Buffers	Caché de páginas de datos (tablas e índices). El más importante para rendimiento.	`shared_buffers` (25% RAM recomendado)
WAL Buffers	Buffer para registros WAL antes de escribirse a disco.	`wal_buffers` (default -1 = auto)
Work Mem	Memoria para operaciones por sesión: ordenamientos (ORDER BY), hash joins, agregaciones.	`work_mem` (por operación, no global)
Maintenance Work Mem	Memoria para operaciones de mantenimiento: VACUUM, CREATE INDEX, REINDEX, ADD FOREIGN KEY.	`maintenance_work_mem` (mayor que work_mem)
Temp Buffers	Buffer para tablas temporales por sesión.	`temp_buffers`

Ejemplo de configuración típica (postgresql.conf):

ini

```
shared_buffers = 4GB              # 25% de 16GB RAM
work_mem = 32MB                   # por operación de ordenamiento/hash
maintenance_work_mem = 1GB        # para VACUUM y CREATE INDEX
wal_buffers = 16MB
temp_buffers = 64MB
effective_cache_size = 12GB       # estimado de caché del SO (75% RAM)
```

Ver configuración actual:

sql

```
SHOW shared_buffers;
SHOW work_mem;
```

```
SELECT name, setting, unit FROM pg_settings
WHERE name IN ('shared_buffers', 'work_mem', 'maintenance_work_mem');
```

Ejercicio práctico 1.1

Objetivo: Familiarizarse con los procesos y la memoria de PostgreSQL.

1. **Listar procesos:**

   bash

   ```
   sudo systemctl start postgresql   # o pg_ctl start
   ps aux | grep postgres | grep -v grep
   ```

   Identifica: postmaster, bgwriter, walwriter, autovacuum launcher, checkpointer.

2. **Conectar y crear actividad:**

   sql

   ```
   CREATE DATABASE test_arquitectura;
   \c test_arquitectura
   CREATE TABLE ventas (id serial, monto numeric);
   INSERT INTO ventas (monto) SELECT random()*1000 FROM generate_series(1,100000);
   ```

3. **Observar nuevos procesos backend:**

   bash

   ```
   # En otra terminal, mientras la consulta se ejecuta:
   ps aux | grep "test_arquitectura"
   ```

4. **Monitorear uso de memoria compartida:**

   sql

   ```
   SELECT name, setting, unit, boot_val, reset_val
   FROM pg_settings
   WHERE category LIKE '%Memory%'
   ORDER BY name;
   ```

5. **Ver estadísticas de bgwriter:**

   sql

   ```
   SELECT * FROM pg_stat_bgwriter;
   ```

- `checkpoints_timed` vs `checkpoints_req` (solicitados vs programados)
- `buffers_checkpoint`, `buffers_clean`, `buffers_backend`

1.2 Estructura de almacenamiento

Tablespaces físicos y lógicos

Un **tablespace** es una ubicación de almacenamiento donde PostgreSQL guarda los archivos de datos (tablas, índices, TOAST).

Jerarquía:

text

```
Tablespace lógico → Directorio físico → Archivos de datos
```

Tablespaces por defecto:

- `pg_default` → usa el directorio base (`$PGDATA/base/`)
- `pg_global` → almacena catálogo del sistema (`$PGDATA/global/`)

Crear un nuevo tablespace:

sql

```sql
-- Requiere privilegios de superusuario
CREATE TABLESPACE ts_fast LOCATION '/mnt/ssd/postgres/data';
CREATE TABLESPACE ts_archive LOCATION '/mnt/hdd/postgres/archive';

-- Crear tabla en tablespace específico
CREATE TABLE facturas (id int, total numeric) TABLESPACE ts_fast;

-- Mover una tabla existente
ALTER TABLE ventas SET TABLESPACE ts_archive;
```

Ver tablespaces:

sql

```sql
SELECT oid, spcname, pg_tablespace_location(oid) FROM pg_tablespace;
```

Arquitectura de archivos:

bash

```bash
$PGDATA/
├── base/                       # pg_default
│   ├── 16384/                  # OID de la base de datos
│   │   ├── 16385               # archivo de tabla (relfilenode)
│   │   ├── 16385_fsm           # free space map
│   │   ├── 16385_vm            # visibility map
│   │   └── 16390               # índice
│   └── 16386/                  # otra base
├── global/                     # pg_global
│   ├── pg_control              # archivo de control del cluster
│   └── 1260                    # tabla pg_authid
├── pg_wal/                     # Write-Ahead Logs
├── pg_xact/                    # información de transacciones
└── pg_stat/                    # estadísticas persistentes
```

Estructura interna de una base de datos

OID (Object Identifier): Identificador único de 32 bits para cada objeto (base de datos, tabla, índice, etc.)

sql

```sql
-- Ver OID de una base de datos
SELECT oid, datname FROM pg_database WHERE datname = 'mi_db';

-- Ver OID de tablas
SELECT oid, relname, relfilenode FROM pg_class WHERE relname = 'ventas';
```

Nota: `relfilenode` es el nombre del archivo en disco. Cambia con VACUUM FULL, CLUSTER, o REINDEX.

Catálogos fundamentales:

- `pg_class` → metadatos de tablas, índices, vistas, secuencias
- `pg_attribute` → columnas de cada tabla
- `pg_index` → información de índices
- `pg_namespace` → esquemas

Ejemplo: encontrar archivo físico de una tabla

sql

```
-- Obtener relfilenode
SELECT relfilenode FROM pg_class WHERE relname = 'ventas';
-- Resultado: 16385

-- En el sistema operativo (dentro del directorio de la BD)
-- $PGDATA/base/[oid_de_la_db]/16385
```

Segmentos, extensiones y bloques

Organización física de una tabla:

Nivel	Descripción	Tamaño
Bloque (Block/Page)	Unidad mínima de I/O	8 KB (por defecto)
Extensión (Extent)	Grupo de bloques asignados juntos	8 bloques × 8KB = 64 KB
Segmento (Segment)	Conjunto de extensiones que forman un archivo	Hasta 1 GB (por defecto)

Mecanismo de crecimiento:

1. PostgreSQL asigna extensiones de 64 KB (8 bloques) cada vez que se llena un segmento
2. Cuando un segmento alcanza 1 GB (131072 bloques), se crea un nuevo archivo:

- `16385` (primer segmento)
- `16385.1` (segundo segmento)
- `16385.2` (tercer segmento, etc.)

Verificar segmentación:

bash

```
ls -lh $PGDATA/base/16384/16385*
# salida:
# -rw------- 16385  1.0G
# -rw------- 16385.1 1.0G
# -rw------- 16385.2 512M
```

Configurar tamaño máximo de segmento:

ini

```
# postgresql.conf - requiere recompilar (no recomendado cambiar)
# --with-segsize=SEGSIZE (por defecto 1 GB)
```

TOAST (The Oversized-Attribute Storage Technique)

Problema: PostgreSQL almacena tuplas completas dentro de un bloque de 8 KB. Si un campo excede ese tamaño, no cabe.

Solución TOAST: Los valores grandes se comprimen y/o almacenan en una tabla auxiliar separada.

Umbral TOAST: Cuando un campo supera **2 KB** (aprox), PostgreSQL decide aplicar TOAST.

Estrategias de almacenamiento TOAST:

- **PLAIN:** No TOAST (solo para tipos fijos como integer, date)
- **EXTENDED:** Comprimir + almacenar fuera de línea (default para texto, json, jsonb, bytea)
- **EXTERNAL:** Almacenar fuera pero sin compresión
- **MAIN:** Almacenar en línea si es posible, comprimir si no

Ver estructura TOAST de una tabla:

sql

```
-- Crear tabla con campo grande
CREATE TABLE logs (
    id serial,
    mensaje text,
    payload jsonb,
    archivo bytea
);

-- Ver tabla TOAST asociada
SELECT relname, reltoastrelid
FROM pg_class
WHERE relname = 'logs';
-- reltoastrelid es el OID de la tabla TOAST

-- Ver detalles de la tabla TOAST
SELECT relname, relpages, reltuples
FROM pg_class
WHERE oid = (SELECT reltoastrelid FROM pg_class WHERE relname = 'logs');
```

Ejemplo práctico de TOAST:

sql

```
-- Insertar datos grandes
INSERT INTO logs (mensaje, payload) VALUES (
    repeat('Texto muy largo ', 10000),
    '{"datos": "' || repeat('x', 5000) || '"}'
);

-- Verificar que se usó TOAST (desde psql con expansión)
```

```
\x
SELECT * FROM pg_stat_user_tables WHERE relname = 'logs';
-- Ver columna 'toast_blks_read' > 0
```

Monitoreo de TOAST:

sql

```
-- Identificar tablas con mucho uso de TOAST
SELECT relname, n_tup_ins, n_tup_upd, toast_blks_read, toast_blks_hit
FROM pg_stat_user_tables
WHERE toast_blks_read > 0
ORDER BY toast_blks_read DESC;
```

Ejercicio práctico 1.2

Objetivo: Explorar la estructura física de una base de datos.

1. **Crear una base de datos y tabla con datos:**

   sql

   ```
   CREATE DATABASE almacenamiento;
   \c almacenamiento

   CREATE TABLE productos (
       id serial PRIMARY KEY,
       nombre text,
       descripcion text,
       especificaciones jsonb,
       imagen bytea
   );

   INSERT INTO productos (nombre, descripcion, especificaciones)
   SELECT
       'Producto ' || i,
       repeat('Descripción larga ', 100),
       '{"peso": ' || (random()*100)::int || ', "color": "rojo"}'
   FROM generate_series(1, 10000) i;
   ```

2. **Encontrar archivos físicos:**

   sql

   ```
   -- Obtener OID de la base actual
   SELECT oid, datname FROM pg_database WHERE datname = 'almacenamiento';
   -- Resultado: oid = 12345 (ejemplo)

   -- Obtener relfilenode de la tabla
   SELECT relfilenode FROM pg_class WHERE relname = 'productos';
   -- Resultado: 16386
   ```

3. **En el sistema operativo (como usuario postgres):**

   bash

   ```
   cd $PGDATA/base/12345/
   ls -lh 16386*
   # Observar si hay segmentos (16386.1, 16386.2...)
   ```

4. **Ver estructura interna de un bloque:**

   sql

```
-- Instalar extensión pageinspect (requiere superusuario)
CREATE EXTENSION pageinspect;

-- Ver cabeceras de páginas
SELECT * FROM page_header(get_raw_page('productos', 0));
```

5. **Simular TOAST:**

   sql

```
-- Insertar registro con campo enorme
INSERT INTO productos (nombre, descripcion)
VALUES ('Producto GIGANTE', repeat('X', 100000));

-- Ver uso de TOAST
SELECT relname, n_tup_ins, toast_blks_read
FROM pg_stat_user_tables
WHERE relname = 'productos';
```

6. **Explorar catálogos:**

   sql

```
-- Ver todas las tablas del sistema en pg_class
SELECT relname, relkind, relpages, reltuples
FROM pg_class
WHERE relnamespace = 'pg_catalog'::regnamespace
LIMIT 20;
```

Resumen del Módulo 1.1 y 1.2

Concepto	Qué debes recordar
Modelo de procesos	Cada conexión = un proceso backend
Postmaster	Proceso padre que escucha y lanza hijos
Shared Buffers	La caché más importante (25% RAM)
WAL	Garantiza durabilidad (Write-Ahead Log)
Autovacuum	Limpia tuplas muertas automáticamente
Tablespaces	Permiten ubicar datos en diferentes discos
OID y relfilenode	Identificadores únicos, relfilenode = nombre de archivo
Segmentos	Archivos de máximo 1 GB
TOAST	Almacena campos grandes fuera de línea

¿Quieres que continúe con los apartados **1.3 (Índices internos)** o **1.4 (MVCC)** del Módulo 1?

que son los archivos wal

Archivos WAL (Write-Ahead Log) en PostgreSQL

Definición fundamental

Los **archivos WAL (Write-Ahead Log)** son el registro de transacciones de PostgreSQL, también conocido como **XLOG**. Es un mecanismo que garantiza la **durabilidad** (propiedad D de ACID) y permite la **recuperación ante fallos**.

Principio básico: Toda modificación a la base de datos se escribe **primero en el log WAL** y **después** en los archivos de datos reales (tablas e índices).

text

```
Orden correcto:  [Cambio] → [WAL en disco] → [Confirmación al cliente] → [Data
files actualizados después]
```

¿Por qué es necesario?

Problema sin WAL

Si PostgreSQL escribiera directamente en los data files y el sistema se cayera:

- Una escritura podría estar a medio completar
- Los índices podrían quedar inconsistentes
- No habría forma de saber qué transacciones estaban commit y cuáles no

Solución con WAL

- Al reiniciar, PostgreSQL **replayea** (reproduce) los registros WAL desde el último checkpoint
- Se rehacen (REDO) los cambios que estaban commit pero no en disco
- Se deshacen (UNDO) los cambios no commit (aunque PostgreSQL no hace UNDO tradicional, usa MVCC)

Estructura física de los archivos WAL

Ubicación

bash

```
$PGDATA/pg_wal/
```

Formato de nombre

Los archivos WAL tienen nombres hexadecimales de 24 caracteres con el formato:

text

```
000000010000000000000001
│       │└│       │└│       └─ Segmento (256 archivos por timeline)
│       │└│       └─ LogID (parte alta del LSN)
│       └─ LogID (parte baja del LSN)
└─ Timeline ID (normalmente 00000001)
```

Ejemplo real:

bash

```
ls -lh $PGDATA/pg_wal/
-rw------- 1 postgres postgres 16M 000000010000000000000001
-rw------- 1 postgres postgres 16M 000000010000000000000002
-rw------- 1 postgres postgres 16M 000000010000000000000003
```

Tamaño de cada archivo WAL

- Por defecto: **16 MB**
- Configurable al compilar: `--with-wal-segsize=SEGSIZE` (1,2,4,8,16,32,64,128,256,512,1024 MB)
- Ver tamaño actual:

  sql

  ```
  SHOW wal_segment_size;
  ```

Contenido de un registro WAL

Cada registro WAL contiene:

Componente	Descripción
LSN (Log Sequence Number)	Número único y secuencial (ej: `0/30001E8`)
Transaction ID (XID)	Identificador de la transacción
Operación	INSERT, UPDATE, DELETE, DDL, CHECKPOINT, etc.
Bloque afectado	Tablespace, base de datos, relfilenode, bloque número
Datos antes (opcional)	Para operaciones que requieren UNDO
Datos después	El cambio realizado (imagen completa o diferencial)

Ejemplo de un registro WAL (formato legible):

text

```
rmgr: Heap        len (rec/tot): 54/ 1008, tx: 12345, lsn: 0/030001E8, prev
0/030001B0
desc: INSERT off 2 flag 0x00, blkref #0: rel 1663/16384/16385 blk 5 FPW
```

Interpretación:

- `Heap` = operación en tabla heap

- `INSERT` = inserción de registro
- `tx: 12345` = transacción 12345
- `lsn: 0/030001E8` = posición en el log
- `rel 1663/16384/16385` = tablespace 1663, base 16384, tabla 16385
- `blk 5` = bloque número 5 de esa tabla
- `FPW` = Full Page Write (página completa)

Parámetros críticos de configuración WAL

Postgresql.conf (ejemplos reales)

ini

```
# ============================================
# Configuración WAL básica
# ============================================

# Tamaño del buffer WAL en memoria
wal_buffers = 16MB              # default -1 = 1/32 de shared_buffers (mín 64KB)

# Retraso máximo del WAL writer (ms)
wal_writer_delay = 200ms        # frecuencia de escritura a disco
wal_writer_flush_after = 1MB    # forzar flush después de N bytes

# ============================================
# Configuración de checkpoint
# ============================================

checkpoint_timeout = 5min       # checkpoint automático cada 5 min
checkpoint_completion_target = 0.9 # distribuir I/O en 90% del intervalo
max_wal_size = 1GB              # tamaño máximo total de WALs (default 1GB)
min_wal_size = 80MB             # tamaño mínimo a retener (default 80MB)

# ============================================
# Modos de sincronización (durabilidad vs rendimiento)
# ============================================

# Opción 1: Máxima seguridad (lento)
synchronous_commit = on         # espera confirmación de WAL en disco

# Opción 2: Mejor rendimiento (puede perder últimas transacciones)
# synchronous_commit = off      # no espera flush a disco

# Opción 3: Compromiso (default)
synchronous_commit = remote_write # para replicación síncrona

# ============================================
# Compresión WAL (ahorra espacio, consume CPU)
# ============================================

wal_compression = on            # comprime páginas completas (FPW)
```

Ciclo de vida de un cambio con WAL

Escenario: INSERT en una tabla

text

```
Paso 1: Cliente ejecuta INSERT
┌──────────────────────────────────────────────────────────┐
│ INSERT INTO clientes (nombre) VALUES ('Juan');           │
└──────────────────────────────────────────────────────────┘
                              ↓
Paso 2: Backend escribe en WAL Buffer (memoria)
┌──────────────────────────────────────────────────────────┐
│ [WAL Buffer] Registro: INSERT tx=12345, tabla=clientes...│
└──────────────────────────────────────────────────────────┘
                              ↓
Paso 3: WAL Writer (o checkpoint) escribe a disco (pg_wal/)
┌──────────────────────────────────────────────────────────┐
│ [Disco] /pg_wal/000000010000000000000001                 │
│ Registro WAL persistente                                 │
└──────────────────────────────────────────────────────────┘
                              ↓
Paso 4: PostgreSQL confirma COMMIT al cliente
┌──────────────────────────────────────────────────────────┐
│ Cliente recibe: "INSERT 0 1"                             │
└──────────────────────────────────────────────────────────┘
                              ↓
Paso 5: Background Writer (más tarde) escribe en data files
┌──────────────────────────────────────────────────────────┐
│ [Disco] /base/16384/16385 (tabla clientes)               │
│ El bloque correspondiente se actualiza                   │
└──────────────────────────────────────────────────────────┘
```

Punto clave: El cliente recibe confirmación en el **paso 4**, aunque los datos aún no estén en la tabla real (solo en WAL).

Full Page Writes (FPW)

¿Qué son?

Durante un **checkpoint**, la primera modificación a cada página (bloque) después del checkpoint escribe la **página completa** en el WAL, no solo el cambio incremental.

Ejemplo:

text

```
Checkpoint en LSN 0/1000000

Primer UPDATE después del checkpoint:
- Registro WAL normal (solo los bytes cambiados) → 100 bytes
- PERO: También escribe los 8KB completos de la página (Full Page Write)

Segundo UPDATE a la misma página:
- Solo registro normal (100 bytes) sin FPW
```

¿Por qué FPW?

Si ocurre una caída justo después del checkpoint, PostgreSQL necesita poder reconstruir páginas que estaban siendo escritas parcialmente. La imagen completa permite restaurar la página desde el WAL.

Costo de FPW

- Cada página modificada por primera vez después de checkpoint = **8KB extra en WAL**
- En tablas grandes con muchos checkpoints, FPW puede dominar el tamaño del WAL

Optimización:

ini

```
# Reduce frecuencia de checkpoints (menos FPW)
checkpoint_timeout = 15min
max_wal_size = 4GB

# Alternativa: usar checksums en páginas (Postgres 11+)
full_page_writes = on  # sigue siendo recomendado para producción
```

Monitoreo y diagnóstico de WAL

1. Ver posición actual del WAL (LSN)

sql

```
-- LSN actual del sistema
SELECT pg_current_wal_lsn();

-- Diferencia entre dos LSN (bytes)
SELECT pg_wal_lsn_diff('0/30001E8', '0/3000000');

-- Ubicación del último checkpoint
SELECT pg_current_wal_insert_lsn(), pg_current_wal_flush_lsn(),
pg_current_wal_lsn();
```

2. Ver archivos WAL generados

sql

```
-- Listar archivos WAL (desde SQL)
SELECT * FROM pg_ls_waldir() ORDER BY modification DESC LIMIT 10;

-- Cantidad total de WAL generado por sesión/consulta
SELECT pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), '0/0'));
```

3. Estadísticas de WAL

sql

```
-- Desde pg_stat_bgwriter
SELECT checkpoints_timed, checkpoints_req,
       wal_buffers_full, wal_writes
FROM pg_stat_bgwriter;
```

```
-- Ver tamaño de pg_wal desde SQL
SELECT pg_size_pretty(sum(size)) AS total_wal_size
FROM pg_ls_waldir();
```

4. Identificar qué genera mucho WAL

sql

```
-- Extensión necesaria
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;

-- Consultas que generan más WAL
SELECT query, wal_bytes, wal_records, calls
FROM pg_stat_statements
ORDER BY wal_bytes DESC
LIMIT 10;
```

5. Configuración actual

sql

```
SELECT name, setting, unit, context
FROM pg_settings
WHERE name LIKE '%wal%' OR name LIKE '%checkpoint%'
ORDER BY name;
```

Problemas comunes y soluciones

Problema 1: WAL crece descontroladamente

Síntomas:

bash

```
du -sh $PGDATA/pg_wal/
# Resultado: 50GB (cuando max_wal_size=1GB)
```

Causas posibles:

- Replicación detenida (replica atascada)
- Checkpoints muy espaciados
- Consultas masivas (grandes DELETE/UPDATE sin commit)
- `wal_keep_size` demasiado alto
- Slot de replicación consumiendo WAL

Soluciones:

sql

```
-- Ver slots de replicación
SELECT * FROM pg_replication_slots;

-- Ver si hay réplica atascada
SELECT application_name, state, write_lag, flush_lag, replay_lag
FROM pg_stat_replication;
```

```
-- Eliminar slot huérfano (PELIGROSO, detiene réplica)
SELECT pg_drop_replication_slot('nombre_slot');
```

Problema 2: WAL Writer cuello de botella

Síntoma: Esperas `WALWriteLock` en pg_stat_activity

Solución:

ini

```
# Aumentar buffers y reducir frecuencia de escritura
wal_buffers = 32MB
wal_writer_delay = 500ms

# O usar synchronous_commit = off para casos no críticos
```

Problema 3: Archivos WAL no se reciclan

Verificar:

sql

```
-- ¿Está archive_mode habilitado y fallando?
SHOW archive_mode;
SHOW archive_command;

-- Ver logs de archive
SELECT * FROM pg_stat_archiver;
```

WAL y replicación

PostgreSQL usa WAL para **Streaming Replication**:

text

```
Primario                                        Réplica
────────                                        ───────
Transacción escribe WAL
        ↓
WAL Sender envía registros → → → → → → → WAL Receiver
                                                ↓
                                        Replay WAL (aplica cambios)
```

Tipos de replicación:

- **Asíncrona:** Réplica puede atrasarse (por defecto)
- **Síncrona:** Primario espera confirmación de réplica
- **Lógica:** Basada en cambios por tabla, no por páginas

Resumen ejecutivo sobre WAL

Aspecto	Qué debes saber
Propósito	Garantizar durabilidad y permitir recuperación

Aspecto	Qué debes saber
Ubicación	`$PGDATA/pg_wal/`
Tamaño por archivo	16 MB (configurable)
Regla de oro	Escribir en WAL **antes** que en data files
Parámetro clave	`wal_buffers`, `checkpoint_timeout`, `max_wal_size`
Monitoreo esencial	`pg_current_wal_lsn()`, `pg_stat_bgwriter`, `pg_ls_waldir()`
Problema típico	Crecimiento infinito de pg_wal (replica atascada o falta de checkpoints)
Optimización crítica	Balance entre `synchronous_commit` (seguridad) y rendimiento

Ejercicio práctico: Experimentar con WAL

sql

```
-- 1. Verificar configuración actual
SHOW wal_buffers;
SHOW checkpoint_timeout;
SHOW max_wal_size;

-- 2. Crear tabla de prueba
CREATE TABLE prueba_wal (id serial, texto text, fecha timestamp);
INSERT INTO prueba_wal (texto, fecha) SELECT 'test', now() FROM
generate_series(1,1000);

-- 3. Ver LSN antes y después
SELECT pg_current_wal_lsn() AS antes;
INSERT INTO prueba_wal (texto, fecha) SELECT 'masivo', now() FROM
generate_series(1,100000);
SELECT pg_current_wal_lsn() AS despues;

-- 4. Calcular WAL generado
SELECT pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), '0/0'));

-- 5. Ver archivos WAL creados
SELECT name, size, modification FROM pg_ls_waldir() ORDER BY modification DESC
LIMIT 5;

-- 6. Forzar checkpoint y observar
CHECKPOINT;
SELECT * FROM pg_stat_bgwriter;

-- 7. Simular pérdida de datos (¡NO en producción!)
-- Apagar PostgreSQL abruptamente y observar recuperación al reiniciar
```

¿Quieres que profundice en algún aspecto específico del WAL, como la **recuperación ante fallos** (PITR), la **replicación streaming**, o la **compresión WAL**?

desarrollame los puntos 1.3 1.4 y 2.1

A continuación tienes el desarrollo completo de los apartados **1.3 Índices internos**, **1.4 MVCC** y **2.1 Configuración del servidor**.

Módulo 1: Arquitectura de PostgreSQL (continuación)

1.3 Índices internos

Conceptos fundamentales

Un **índice** es una estructura de datos auxiliar que acelera las operaciones de búsqueda (SELECT, UPDATE, DELETE) a cambio de:

- Espacio en disco adicional (5-30% del tamaño de la tabla)
- Costo en operaciones de escritura (INSERT/UPDATE/DELETE más lentas)

Estructura por defecto: B-Tree (balanceado)

Anatomía interna de un índice B-Tree

text

```
                    [Página raíz]
                   /      |      \
                  /       |       \
        [Página interna] [Página interna] [Página interna]
         /    |    \       /    |    \       /    |    \
    [Hoja] [Hoja] [Hoja] [Hoja] [Hoja] [Hoja] [Hoja] [Hoja]
    (tuplas + TID) (tuplas + TID) (tuplas + TID)
```

Componentes:

- **Páginas/metapáginas:** Bloques de 8 KB igual que las tablas
- **Tuplas de índice:** Contienen (valor de columna, TID)
- **TID (Tuple Identifier):** (número de bloque, offset dentro del bloque) → localización física de la fila

Ejemplo de búsqueda B-Tree:

sql

```
SELECT * FROM usuarios WHERE id = 42;
```

1. Recorre desde raíz hasta hoja (3-4 lecturas de bloque para tablas grandes)
2. Encuentra (42, bloque=125, offset=8)
3. Va directamente al bloque 125 de la tabla heap

Tipos de índices en PostgreSQL

1. B-Tree (predeterminado)

Operadores soportados: <, <=, =, >=, >, BETWEEN, IN, LIKE 'texto%' (prefijo)

Cuándo usar:

- La mayoría de las consultas (95% de casos)
- Columnas con alta cardinalidad (muchos valores únicos)
- Ordenamiento y búsquedas por rango

Ejemplo:

sql

```
CREATE INDEX idx_usuarios_email ON usuarios USING btree (email);
-- USING btree es opcional (es el default)
```

2. Hash

Operadores soportados: Solo =

Características:

- Búsquedas exactas muy rápidas (O(1) promedio)
- No soporta ordenamiento ni búsquedas por rango
- No es transaccionalmente seguro antes de PostgreSQL 10

Cuándo usar:

- Columnas con igualdad exacta (ej: status, tipo, ID de sesión)
- No necesitas ordenamiento ni rango

Ejemplo:

sql

```
CREATE INDEX idx_sesiones_token ON sesiones USING hash (token);
```

3. GIN (Generalized Inverted Index)

Operadores soportados: @>, ?, ?&, ?|, @@ (full-text search)

Estructura interna: Invertida (término → lista de TIDs)

Cuándo usar:

- Arrays (`text[]`, `int[]`)
- JSONB (búsquedas dentro de documentos)
- Búsqueda de texto completo (Full-Text Search)
- Datos con muchos valores por fila

Ejemplo:

sql

```
-- Para JSONB
CREATE INDEX idx_productos_especificaciones ON productos USING gin
(especificaciones);

-- Para texto completo
CREATE INDEX idx_articulos_contenido ON articulos USING gin
(to_tsvector('spanish', contenido));
```

4. GiST (Generalized Search Tree)

Operadores soportados: <<, &<, &>, >>, <<|, &<|, |&>, |>>, @>, <@, &&, ~=

Cuándo usar:

- Datos geométricos/espaciales (PostGIS)
- Rangos (daterange, int4range, tsrange)
- Búsquedas de vecinos más cercanos
- Datos jerárquicos (ltree)

Ejemplo:

sql

```
-- Rango de fechas
CREATE INDEX idx_eventos_periodo ON eventos USING gist (periodo daterange);

-- Búsqueda por proximidad geométrica (con PostGIS)
CREATE INDEX idx_ubicaciones ON lugares USING gist (geom);
```

5. BRIN (Block Range Index)

Operadores soportados: <, <=, =, >=, >

Estructura interna: Almacena resúmenes por rango de bloques (mínimo, máximo)

Características:

- Muy pequeño (1-5% del tamaño de la tabla)
- Más lento que B-Tree para búsquedas puntuales
- Excelente para tablas muy grandes (cientos de millones de filas)

Cuándo usar:

- Tablas masivas (log, eventos, series temporales)
- Datos con correlación física (ej: timestamp de inserción)
- Columnas donde no necesitas búsquedas exactas ultrafast

Ejemplo:

sql

```
CREATE INDEX idx_logs_fecha ON logs USING brin (fecha_registro);
```

6. SP-GiST (Space-Partitioned GiST)

Cuándo usar:

- Datos no balanceados naturalmente (puntos en plano, direcciones IP, strings con prefijos comunes)

Ejemplo:

sql

```
CREATE INDEX idx_rangos_ip ON redes USING spgist (rango_ip inet);
```

Comparativa de tipos de índice

Tipo	Tamaño	Velocidad lectura	Velocidad escritura	Uso típico
B-Tree	Medio	Alta	Media	Default, columnas PK/FK
Hash	Pequeño	Muy alta (igualdad)	Media	Igualdad exacta
GIN	Grande	Media (compleja)	Baja	JSONB, arrays, texto completo
GiST	Grande	Media	Media	Espacial, rangos
BRIN	Muy pequeño	Media-baja	Alta	Tablas masivas, series temporales
SP-GiST	Medio	Media	Media	Datos no balanceados

Índices multicolumna (compuestos)

Orden de columnas importa:

sql

```
-- Bueno para: WHERE ciudad = 'Madrid' AND edad > 18
CREATE INDEX idx_clientes_ciudad_edad ON clientes (ciudad, edad);

-- Malo para la misma consulta (no usa índice completo)
CREATE INDEX idx_clientes_edad_ciudad ON clientes (edad, ciudad);
```

Reglas:

1. Colocar columnas con mayor selectividad (más valores únicos) al principio
2. Considerar patrones de consulta reales
3. PostgreSQL puede usar solo el prefijo del índice (ej: solo `ciudad` en el primer ejemplo)

Ejemplo práctico:

sql

```
-- Tabla de pedidos
CREATE TABLE pedidos (
    id serial,
    cliente_id int,
    fecha date,
    estado text,
```

```
    total numeric
);

-- Índice óptimo para búsquedas comunes
CREATE INDEX idx_pedidos_cliente_fecha_estado
ON pedidos (cliente_id, fecha DESC, estado)
WHERE estado != 'cancelado';  -- Índice parcial
```

Índices parciales

Solo indexan un subconjunto de filas que cumplen una condición.

sql

```
-- Índice solo para pedidos activos
CREATE INDEX idx_pedidos_activos ON pedidos (fecha) WHERE estado = 'activo';

-- Beneficios:
-- - Más pequeño (menos mantenimiento)
-- - Más rápido para consultas que usan la misma condición
```

Caso de uso: Tabla donde la mayoría de consultas filtran por `activo = true` y solo unos pocos registros están inactivos.

Índices únicos y de expresiones

Único:

sql

```
-- Impide duplicados
CREATE UNIQUE INDEX idx_usuarios_email_unico ON usuarios (lower(email));
```

Expresiones (funcionales):

sql

```
-- Índice sobre función
CREATE INDEX idx_usuarios_email_lower ON usuarios (lower(email));

-- La consulta usará el índice
SELECT * FROM usuarios WHERE lower(email) = lower('Juan@Ejemplo.com');
```

Monitoreo de índices

sql

```
-- Ver todos los índices de una tabla
SELECT indexname, indexdef
FROM pg_indexes
WHERE tablename = 'pedidos';

-- Ver tamaño de índices
SELECT
    schemaname,
    tablename,
    indexname,
```

```
    pg_size_pretty(pg_relation_size(indexname::regclass)) AS index_size
FROM pg_indexes
ORDER BY pg_relation_size(indexname::regclass) DESC;

-- Estadísticas de uso de índices (identificar índices no usados)
SELECT
    schemaname,
    tablename,
    indexname,
    idx_scan AS numero_busquedas,
    idx_tup_read AS tuplas_leidas,
    idx_tup_fetch AS tuplas_recuperadas
FROM pg_stat_user_indexes
ORDER BY idx_scan ASC
LIMIT 10;  -- Índices candidatos a eliminación

-- Índices duplicados (misma definición)
SELECT
    indexrelid::regclass,
    indrelid::regclass,
    array_agg(indexrelid::regclass) AS duplicados
FROM pg_index
GROUP BY indrelid, indkey
HAVING count(*) > 1;
```

Ejercicio práctico 1.3

sql

```
-- 1. Crear tabla de pruebas con 1M filas
CREATE TABLE ventas_indices (
    id serial PRIMARY KEY,
    fecha date DEFAULT current_date,
    cliente_id int,
    producto_id int,
    cantidad int,
    monto numeric(10,2)
);

INSERT INTO ventas_indices (cliente_id, producto_id, cantidad, monto)
SELECT
    (random()*10000)::int,
    (random()*500)::int,
    (random()*10)::int + 1,
    (random()*1000)::numeric(10,2)
FROM generate_series(1, 1000000);

-- 2. Probar diferentes índices
-- B-Tree simple
CREATE INDEX idx_ventas_cliente ON ventas_indices (cliente_id);

-- B-Tree compuesto
CREATE INDEX idx_ventas_cliente_fecha ON ventas_indices (cliente_id, fecha);

-- Índice parcial
CREATE INDEX idx_ventas_grandes ON ventas_indices (cliente_id) WHERE monto >
500;

-- Índice de expresión
CREATE INDEX idx_ventas_monto_redondeado ON ventas_indices (round(monto, -1));
```

```
-- 3. Analizar uso
EXPLAIN ANALYZE SELECT * FROM ventas_indices WHERE cliente_id = 1234;
EXPLAIN ANALYZE SELECT * FROM ventas_indices WHERE cliente_id = 1234 AND fecha >
'2024-01-01';
EXPLAIN ANALYZE SELECT * FROM ventas_indices WHERE round(monto, -1) = 100;

-- 4. Ver estadísticas
SELECT * FROM pg_stat_user_indexes WHERE relname = 'ventas_indices';
```

1.4 MVCC (Multiversion Concurrency Control)

Concepto fundamental

MVCC es el modelo de concurrencia de PostgreSQL que permite:

- **Lectores no bloquean escritores**
- **Escritores no bloquean lectores**
- Cada transacción ve una "foto consistente" (snapshot) de los datos en el momento de iniciar

Sin MVCC (bloqueos tradicionales):

text

```
T1: LECTURA → ESPERA → Obtiene datos
T2:          ESCRITURA (bloquea)
```

Con MVCC (PostgreSQL):

text

```
T1: LECTURA → Obtiene versión antigua (no espera)
T2:          ESCRITURA → Crea nueva versión
```

Estructura de una tupla (fila) en el heap

Cada fila en disco contiene campos invisibles (system columns):

Columna	Descripción	Tamaño
`xmin`	ID de la transacción que insertó la tupla	4 bytes
`xmax`	ID de la transacción que eliminó/actualizó (0 si activa)	4 bytes
`cmin/cmax`	Comando dentro de la transacción	4 bytes
`ctid`	Ubicación física (bloque, offset) de la versión actual	6 bytes
Datos	Columnas reales de la tabla	Variable

Ejemplo visual:

text

```
Tabla "usuarios" después de varias operaciones:
```

```
Bloque 1, offset 1:
  xmin=100 (transacción que insertó)
  xmax=0    (activa)
  ctid=(1,1)
  nombre='Ana', email='ana@mail.com'

Bloque 1, offset 2:
  xmin=150 (UPDATE de Ana)
  xmax=0
  ctid=(1,2)
  nombre='Ana', email='ana_nueva@mail.com'

Bloque 2, offset 5:
  xmin=200 (DELETE de Ana)
  xmax=200 (eliminada)
  ctid=(2,5)
  nombre='Ana', email='ana_nueva@mail.com'
```

Ciclo de vida de una tupla con MVCC

1. INSERT

sql

```sql
-- Transacción 100
INSERT INTO usuarios (nombre) VALUES ('Juan');
```

Estado en disco:

text

```text
xmin = 100 (quién insertó)
xmax = 0   (no eliminada)
ctid = (1,1)
visible para T100 y futuras (si no hay conflicto)
```

2. UPDATE (crea nueva versión)

sql

```sql
-- Transacción 150
UPDATE usuarios SET nombre = 'Juan Carlos' WHERE nombre = 'Juan';
```

Pasos internos:

1. Marca la versión antigua como obsoleta: `xmax = 150`
2. Inserta nueva versión: `xmin = 150`, `xmax = 0`
3. Actualiza `ctid` de versión antigua para apuntar a nueva

text

```text
Versión antigua (1,1):  xmin=100, xmax=150, ctid=(1,2)
Versión nueva (1,2):    xmin=150, xmax=0
```

3. DELETE

sql

```sql
-- Transacción 200
DELETE FROM usuarios WHERE nombre = 'Juan Carlos';
```

Pasos:

- Solo marca versión actual: `xmax = 200`
- No elimina físicamente

text

```
Versión (1,2): xmin=150, xmax=200
```

4. COMMIT/ROLLBACK

- Si transacción 200 hace **COMMIT**: tupla queda muerta (dead tuple)
- Si hace **ROLLBACK**: restaura `xmax = 0` (tupla revive)

Reglas de visibilidad

Una transacción `T` (con XID = 500) ve una tupla si **todas** son verdaderas:

1. **`xmin` es visible:**

- `xmin < T` (inició antes)
- `xmin` no está abortada
- `xmin` no es un subcomando no visible

2. **`xmax` es invisible:**

- `xmax = 0` (tupla no eliminada), OR
- `xmax > T` (eliminación futura), OR
- `xmax` está abortada

Snapshot de transacción:

sql

```
-- Ver transacciones activas
SELECT * FROM pg_stat_activity WHERE state = 'active';

-- Ver XID actual
SELECT txid_current();
```

Dead Tuples (tuplas muertas)

Definición: Tuplas que ninguna transacción activa puede ver (obsoletas o eliminadas).

Causas:

- UPDATE (versión vieja)
- DELETE (marcada)

- Transacciones abortadas

Problema: Las tuplas muertas consumen espacio y ralentizan consultas (más bloques para escanear).

Ejemplo de acumulación:

sql

```
-- Tabla con 1M filas
UPDATE usuarios SET ultimo_acceso = now() WHERE 1=1;  -- Actualiza todas
-- Resultado: 1M tuplas vivas + 1M tuplas muertas = 2M tuplas totales
```

Monitoreo de tuplas muertas:

sql

```
SELECT
    schemaname,
    relname,
    n_live_tup AS tuplas_vivas,
    n_dead_tup AS tuplas_muertas,
    round(100 * n_dead_tup / (n_live_tup + 1)::numeric, 2) AS
porcentaje_muertas,
    last_vacuum,
    last_autovacuum
FROM pg_stat_user_tables
WHERE n_dead_tup > 1000
ORDER BY n_dead_tup DESC;
```

VACUUM: limpiando tuplas muertas

VACUUM normal:

- Marca espacio como reusable
- No libera espacio al sistema operativo
- Permite que la tabla crezca pero reutiliza espacio interno
- No bloquea lecturas/escrituras (solo exclusión breve)

sql

```
VACUUM usuarios;  -- Limpia tuplas muertas
```

VACUUM FULL:

- Reconstruye tabla compactando espacio
- Libera espacio al sistema operativo
- **BLOQUEA** la tabla (exclusiva)
- Requiere espacio adicional (copia la tabla)

sql

```
VACUUM FULL usuarios;  -- Compacta y libera espacio al SO
```

VACUUM FREEZE:

- Marca tuplas antiguas como "congeladas" (evita wraparound)
- Especialmente importante para tablas que no se actualizan

sql

```
VACUUM FREEZE usuarios;
```

Transaction ID Wraparound (el problema de los 32 bits)

El problema: XID tiene 32 bits → máximo ≈ 4 mil millones de transacciones.

Solución: PostgreSQL trata XID como un círculo (no lineal):

- XIDs activos están en la parte "nueva" del círculo
- XIDs muy antiguos se congelan (Frozen XID = 2)

Límite crítico: Cuando más de 2 mil millones de transacciones sin vacío, PostgreSQL se apaga.

Monitoreo:

sql

```
-- Ver antigüedad de transacciones por tabla
SELECT
    relname,
    age(relfrozenxid) AS antiguedad,
    pg_size_pretty(pg_relation_size(relid)) AS tamaño
FROM pg_class
WHERE relkind = 'r'
ORDER BY age(relfrozenxid) DESC
LIMIT 10;

-- Ver estado general del cluster
SELECT datname, age(datfrozenxid) FROM pg_database;
```

Prevención:

sql

```
-- Vacío agresivo para congelar tuplas antiguas
VACUUM FREEZE VERBOSE tabla_peligrosa;
```

Comparación con otros motores

Motor	MVCC	Manejo de versiones	Ventaja
PostgreSQL	Sí	En tabla heap (múltiples versiones)	Lectores nunca bloquean
MySQL (InnoDB)	Sí	En segmentos rollback	Menos espacio de almacenamiento
Oracle	Sí	En undo tablespace	Manejo eficiente de versiones
SQL Server	Sí (RCSI)	En tempdb	Configurable por base de datos

---

Ejercicio práctico 1.4

sql

```
-- 1. Configurar para ver xmin/xmax
CREATE TABLE mvcc_test (id int, nombre text);

-- Terminal 1 (Transacción A)
BEGIN;
INSERT INTO mvcc_test VALUES (1, 'Juan');
SELECT xmin, xmax, ctid, * FROM mvcc_test;

-- Terminal 2 (Transacción B)
BEGIN;
SELECT xmin, xmax, ctid, * FROM mvcc_test;  -- No ve el INSERT (aún no commit)

-- Terminal 1
COMMIT;

-- Terminal 2
SELECT xmin, xmax, ctid, * FROM mvcc_test;  -- Ahora sí ve

-- 2. Demostrar UPDATE y dead tuples
UPDATE mvcc_test SET nombre = 'Carlos' WHERE id = 1;

-- Ver tuplas muertas (puede necesitar pg_sleep)
SELECT n_dead_tup, n_live_tup FROM pg_stat_user_tables WHERE relname =
'mvcc_test';

-- 3. Ver el efecto de VACUUM
VACUUM mvcc_test;
SELECT n_dead_tup, n_live_tup FROM pg_stat_user_tables WHERE relname =
'mvcc_test';

-- 4. Simular wraparound (solo para entender, NO ejecutar en producción)
-- SELECT age(relfrozenxid) FROM pg_class WHERE relname = 'mvcc_test';
```

Módulo 2: Mantenimiento y Administración

2.1 Configuración del servidor (postgresql.conf, pg_hba.conf)

Archivos de configuración principales

Archivo	Ubicación	Propósito
`postgresql.conf`	`$PGDATA/`	Configuración del motor (memoria, logs, conexiones)
`pg_hba.conf`	`$PGDATA/`	Control de acceso (quién puede conectarse desde dónde)
`pg_ident.conf`	`$PGDATA/`	Mapeo de usuarios del SO a roles PostgreSQL
`postgresql.auto.conf`	`$PGDATA/`	Configuración generada por ALTER SYSTEM (no editar manualmente)

Ver ubicación actual:

sql

```sql
SHOW config_file;
SHOW hba_file;
SHOW ident_file;
```

---

postgresql.conf: Parámetros críticos

1. Conexiones y red

ini

```ini
# ============================================
# CONEXIONES Y AUTENTICACIÓN
# ============================================

# Puerto de escucha (default: 5432)
port = 5432

# Direcciones donde escuchar ('' = solo localhost, '*' = todas)
listen_addresses = '*'          # para producción, especificar IPs

# Número máximo de conexiones simultáneas
max_connections = 100           # ajustar según RAM (cada conexión ~2-5MB)

# Número de conexiones reservadas para superusuario
superuser_reserved_connections = 3

# Tiempo de espera para autenticación (segundos)
authentication_timeout = 60s
```

Fórmula de RAM para max_connections:

text

```text
RAM para backend = max_connections * (work_mem + temp_buffers + overhead)
Ejemplo: 100 conexiones * (32MB + 8MB + 5MB) ≈ 4.5GB adicionales
```

2. Memoria (los más importantes)

ini

```ini
# ============================================
# MEMORIA - CRÍTICOS PARA RENDIMIENTO
# ============================================

# Shared Buffers: caché de datos (20-25% de RAM total)
shared_buffers = 4GB            # 25% de 16GB RAM

# Work Mem: memoria por operación de ordenamiento/hash (por sesión)
work_mem = 32MB                 # 32MB * max_connections = memoria potencial

# Maintenance Work Mem: VACUUM, CREATE INDEX, etc.
maintenance_work_mem = 1GB      # Puede ser grande (10% RAM)

# Estimación de caché del sistema operativo
effective_cache_size = 12GB     # 75% de RAM total

# Buffer para tablas temporales por sesión
temp_buffers = 8MB
```

```
# Buffer WAL en memoria
wal_buffers = 16MB              # -1 = auto (1/32 de shared_buffers)
```

Regla de oro para shared_buffers:

- < 1GB RAM: 15-20% de RAM
- 1-4GB RAM: 20-25% de RAM
- 4-16GB RAM: 25-30% de RAM
- 16GB RAM: 8-16GB (no más de 32GB por overhead de gestión)

3. Escritura en disco (WAL y Checkpoints)

ini

```
# ============================================
# WRITE-AHEAD LOG (WAL)
# ============================================

# Tamaño máximo total de archivos WAL
max_wal_size = 4GB              # Aumentar en sistemas con mucha escritura
min_wal_size = 1GB              # Tamaño mínimo a retener

# Checkpoint cada N segundos
checkpoint_timeout = 10min      # Default 5min, aumentar para reducir I/O

# Completar checkpoint en X% del intervalo (evita picos I/O)
checkpoint_completion_target = 0.9

# Modo de commit síncrono
synchronous_commit = on         # on = seguro, off = más rápido (riesgo)

# Compresión de páginas completas en WAL
wal_compression = on            # Reduce espacio, aumenta CPU

# Archivo WAL segment size (requiere recompilación)
# wal_segment_size = 16MB
```

4. Autovacuum (esencial para evitar dead tuples)

ini

```
# ============================================
# AUTOVACUUM - OBLIGATORIO PARA PRODUCCIÓN
# ============================================

# Habilitar autovacuum (default ON)
autovacuum = on

# Umbral para activar VACUUM (tuplas muertas > umbral + factor * total)
autovacuum_vacuum_threshold = 50        # tuplas muertas mínimas
autovacuum_vacuum_scale_factor = 0.2    # 20% de la tabla

# Umbral para ANALYZE (actualizar estadísticas)
autovacuum_analyze_threshold = 50
autovacuum_analyze_scale_factor = 0.1   # 10% de la tabla

# Workers simultáneos
autovacuum_max_workers = 3              # Aumentar si muchas tablas grandes

# Retraso entre operaciones (evitar impacto I/O)
```

```
autovacuum_vacuum_cost_delay = 2ms      # Default 2ms
autovacuum_vacuum_cost_limit = 1000     # Default -1 = 200

# Tiempo de espera antes de matar worker bloqueado
autovacuum_naptime = 1min
```

Fórmula de activación VACUUM:

text

```
n_dead_tup > autovacuum_vacuum_threshold + (autovacuum_vacuum_scale_factor *
n_live_tup)
```

Ejemplo: Tabla con 1,000,000 filas vivas

text

```
Umbral = 50 + (0.2 * 1,000,000) = 50 + 200,000 = 200,050 tuplas muertas
```

5. Logging (debugging y auditoría)

ini

```
# ============================================
# LOGGING
# ============================================

# ¿Qué loguear?
log_statement = 'ddl'           # none, ddl, mod, all
log_min_duration_statement = 1000ms  # Consultas lentas (>1 segundo)

# ¿Dónde loguear?
log_destination = 'stderr'      # stderr, csvlog, syslog, eventlog
logging_collector = on          # Recopilar logs en archivos
log_directory = 'log'           # Directorio dentro de $PGDATA
log_filename = 'postgresql-%Y-%m-%d_%H%M%S.log'

# ¿Qué información incluir?
log_line_prefix = '%t [%p]: [%l-1] user=%u,db=%d,app=%a,client=%h '
# %t = timestamp, %p = PID, %u = usuario, %d = db, %h = host

log_connections = on            # Registrar conexiones nuevas
log_disconnections = on         # Registrar desconexiones
log_duration = off              # Duración de cada consulta (on = verbose)
log_lock_waits = on             # Registrar esperas de bloqueos
(>deadlock_timeout)
log_temp_files = 1MB            # Registrar archivos temporales > tamaño
```

6. Optimización de consultas (planner)

ini

```
# ============================================
# OPTIMIZADOR (PLANNER)
# ============================================

# Calidad de estadísticas (default 100, máximo 10000)
default_statistics_target = 500   # Mayor = mejor plan, más tiempo ANALYZE

# Forzar tipos de join (para debugging)
# enable_nestloop = on
# enable_hashjoin = on
# enable_mergejoin = on
# enable_seqscan = on
```

```
# Constantes para estimaciones
cpu_tuple_cost = 0.01            # Costo por fila
cpu_index_tuple_cost = 0.005     # Costo por entrada de índice
cpu_operator_cost = 0.0025       # Costo por operador
seq_page_cost = 1.0              # Costo lectura secuencial
random_page_cost = 1.1           # SSD=1.1, HDD=4.0
```

Ajuste de random_page_cost según disco:

- SSD/NVMe: 1.0 - 1.5
- HDD 7200rpm: 4.0 - 6.0
- SAN/NAS rápido: 2.0 - 3.0

Cómo modificar la configuración

Método 1: Editar archivo directamente

bash

```
# Editar postgresql.conf
sudo nano $PGDATA/postgresql.conf

# Recargar configuración (sin reiniciar)
SELECT pg_reload_conf();

# O desde terminal
pg_ctl reload -D $PGDATA
```

Método 2: ALTER SYSTEM (PostgreSQL 9.4+)

sql

```
-- Cambiar parámetro (escribe en postgresql.auto.conf)
ALTER SYSTEM SET work_mem = '64MB';
ALTER SYSTEM SET max_connections = 200;

-- Recargar configuración
SELECT pg_reload_conf();

-- Ver parámetros modificados
SELECT name, setting, source, sourcefile
FROM pg_settings
WHERE source NOT IN ('default', 'override');
```

Método 3: Por sesión (solo para pruebas)

sql

```
-- Para la sesión actual
SET work_mem = '128MB';
SET enable_seqscan = off;

-- Para toda la base de datos
ALTER DATABASE mi_db SET work_mem = '64MB';

-- Para un usuario específico
ALTER ROLE admin SET work_mem = '128MB';
```

---

pg_hba.conf: Control de acceso

Formato de cada línea:

text

```
tipo    base_de_datos    usuario    dirección    método    [opciones]
```

Ejemplo completo:

conf

```
# TYPE  DATABASE        USER            ADDRESS                 METHOD

# Local (Unix domain socket)
local   all             all                                     trust

# IPv4 local connections
host    all             all             127.0.0.1/32            trust
host    all             all             192.168.1.0/24          md5

# IPv6 local connections
host    all             all             ::1/128                 trust

# Replicación
host    replication     replica_user    192.168.1.10/32         md5

# Acceso específico a una base de datos
host    reporting       readonly_user   10.0.0.0/8              password

# Autenticación con LDAP
host    all             ldap_users      10.0.0.0/16             ldap
ldapserver=ldap.example.com ldapprefix="uid="

# Autenticación con certificado SSL
hostssl all             +cert_users     0.0.0.0/0               cert
clientcert=1
```

Métodos de autenticación comunes:

Método	Descripción	Seguridad	Uso típico
`trust`	Sin contraseña	Muy baja	Solo localhost, desarrollo
`reject`	Rechazar conexión	N/A	Bloquear acceso
`md5` / `scram-sha-256`	Contraseña hasheada	Alta	Producción (usar scram-sha-256)
`password`	Contraseña texto plano	Baja	Evitar
`peer`	Usuario del SO	Alta	Local, Unix socket
`cert`	Certificado SSL	Muy alta	Seguridad máxima
`ldap` / `radius`	Autenticación centralizada	Alta	Entornos corporativos

Orden importa: La PRIMERA línea que coincide se aplica.

Ejemplo de orden correcto:

conf

```
# Primero reglas específicas
```

```
host    app_db          app_user        192.168.1.100/32        md5
host    all             admin           192.168.1.0/24          md5

# Después reglas generales (más amplias)
host    all             all             0.0.0.0/0               reject
```

Configuración por objeto (tablas/índices)

sql

```sql
-- Configurar autovacuum específico para una tabla grande
ALTER TABLE logs SET (
    autovacuum_vacuum_scale_factor = 0.01,   -- 1% en lugar de 20%
    autovacuum_vacuum_threshold = 1000,
    autovacuum_vacuum_cost_delay = 10ms      -- Más agresivo
);

-- Configurar fillfactor para tablas con muchas actualizaciones
ALTER TABLE transacciones SET (fillfactor = 70);  -- Default 100

-- Deshabilitar toast compression para datos ya comprimidos
ALTER TABLE imagenes SET (toast_tuple_target = 8160);
```

Monitoreo de configuración

sql

```sql
-- Ver todas las configuraciones no default
SELECT name, setting, unit, source, boot_val, reset_val
FROM pg_settings
WHERE source NOT IN ('default', 'override')
ORDER BY name;

-- Ver cambios recientes
SELECT name, setting, unit, source, pending_restart
FROM pg_settings
WHERE pending_restart = true;  -- Requiere reinicio

-- Comparar con valores por defecto
SELECT
    name,
    setting AS current,
    boot_val AS default,
    unit
FROM pg_settings
WHERE setting != boot_val
ORDER BY name;
```

Ejercicio práctico 2.1

sql

```sql
-- 1. Ver configuración actual
SHOW ALL;

-- 2. Crear backup de configuraciones actuales
SELECT name, setting
```

ahora 2.2 2.3 2.4 2.5

A continuación tienes el desarrollo completo de los apartados **2.2 Vacuum y Analyze**, **2.3 Reindexación y bloat**, **2.4 Backup y recuperación**, y **2.5 Monitoreo y tuning** del Módulo 2.

Módulo 2: Mantenimiento y Administración (continuación)

2.2 Vacuum y Analyze

VACUUM: Limpieza de tuplas muertas

Como vimos en MVCC, las operaciones UPDATE y DELETE generan **dead tuples** (tuplas muertas) que consumen espacio y degradan el rendimiento. VACUUM es la herramienta que limpia estas tuplas.

Tipos de VACUUM

Comando	Bloqueo	Libera espacio al SO	Velocidad	Uso típico
`VACUUM`	No bloquea (solo breve)	No	Rápido	Mantenimiento regular
`VACUUM FULL`	Bloqueo EXCLUSIVO	Sí	Lento	Cuando hay mucha fragmentación
`VACUUM FREEZE`	No bloquea	No	Medio	Prevenir wraparound
`VACUUM VERBOSE`	No bloquea	No	Rápido	Diagnóstico

VACUUM normal (modo "no bloqueante")

sql

```
-- Vaciar una tabla específica
VACUUM productos;

-- Vaciar todas las tablas de la base actual
VACUUM;

-- Vaciar con información detallada
VACUUM VERBOSE productos;

-- Vaciar solo para prevenir wraparound (congelar tuplas)
VACUUM FREEZE productos;
```

Qué hace VACUUM normal:

1. Escanea la tabla identificando tuplas muertas (xmax < txid_snapshot)

2. Marca el espacio de esas tuplas como reusable
3. Actualiza el Visibility Map (páginas con solo tuplas visibles)
4. Actualiza estadísticas de pg_stat_user_tables
5. **NO reduce el tamaño del archivo en disco**

Ejemplo de reutilización de espacio:

sql

```
-- Estado inicial: tabla de 100 MB
INSERT INTO logs VALUES (1, 'a'), (2, 'b'), (3, 'c');  -- 100 MB
DELETE FROM logs WHERE id = 2;  -- Elimina 33% de las filas

-- Sin VACUUM: la tabla sigue ocupando 100 MB
-- Con VACUUM: espacio interno marcado como reusable
VACUUM logs;
INSERT INTO logs VALUES (4, 'd');  -- Reutiliza el espacio de la fila borrada
-- La tabla sigue ocupando 100 MB, pero con nuevas filas
```

VACUUM FULL (modo "compactación")

sql

```
-- Compactar tabla liberando espacio al sistema operativo
VACUUM FULL productos;

-- Con reindexación automática
VACUUM FULL VERBOSE productos;
```

Qué hace VACUUM FULL:

1. Crea una nueva copia de la tabla solo con tuplas vivas
2. Elimina la tabla original
3. Renombra la nueva copia
4. Reconstruye todos los índices
5. **Requiere espacio extra (tamaño de la tabla + índices)**
6. **BLOQUEA la tabla durante toda la operación**

Cuándo usar VACUUM FULL:

- Después de grandes eliminaciones (ej: `DELETE FROM logs WHERE fecha < '2023-01-01'`)
- Cuando la fragmentación supera el 30-40%
- Como parte de mantenimiento programado (ej: domingo 3 AM)

Monitorear necesidad de VACUUM FULL:

sql

```
SELECT
    schemaname,
    relname,
```

```
    n_live_tup,
    n_dead_tup,
    round(100 * n_dead_tup / (n_live_tup + n_dead_tup)::numeric, 2) AS
dead_ratio,
    pg_size_pretty(pg_relation_size(relid)) AS table_size
FROM pg_stat_user_tables
WHERE n_dead_tup > 10000
ORDER BY dead_ratio DESC;
```

ANALYZE: Estadísticas para el optimizador

ANALYZE recolecta **estadísticas** sobre la distribución de datos en tablas e índices. El planificador (planner) usa estas estadísticas para decidir cómo ejecutar consultas.

sql

```
-- Analizar una tabla específica
ANALYZE productos;

-- Analizar todas las tablas de la base actual
ANALYZE;

-- Analizar con información detallada
ANALYZE VERBOSE productos;

-- Analizar columnas específicas
ANALYZE productos (nombre, precio, categoria_id);
```

Qué estadísticas recolecta:

- Número total de filas (reltuples en pg_class)
- Número de bloques (relpages)
- Valores más comunes (most_common_vals, most_common_freqs)
- Histograma de distribución (histogram_bounds)
- Nulabilidad y correlación

Ver estadísticas de una columna:

sql

```
-- Ver estadísticas de la columna 'precio'
SELECT
    attname,
    n_distinct,
    most_common_vals,
    most_common_freqs,
    histogram_bounds
FROM pg_stats
WHERE tablename = 'productos' AND attname = 'precio';
```

Ejemplo de cómo ANALYZE mejora las consultas:

Sin ANALYZE (estadísticas desactualizadas):

sql

```
-- La tabla tiene 1,000,000 filas, pero ANALYZE dice 10,000
```

```
EXPLAIN SELECT * FROM productos WHERE precio > 1000;
-- Usa Seq Scan porque cree que solo hay 10,000 filas
-- En realidad, hay 500,000 filas con precio > 1000
```

Con ANALYZE actualizado:

sql

```
ANALYZE productos;
EXPLAIN SELECT * FROM productos WHERE precio > 1000;
-- Usa Index Scan si existe índice en precio
-- El planner sabe que hay muchas filas y el índice es más eficiente
```

Autovacuum: Automatización del mantenimiento

PostgreSQL incluye **autovacuum** (habilitado por defecto) que ejecuta VACUUM y ANALYZE automáticamente.

Configuración básica en postgresql.conf:

ini

```
autovacuum = on
autovacuum_vacuum_threshold = 50          # tuplas muertas mínimas
autovacuum_vacuum_scale_factor = 0.2      # 20% de la tabla
autovacuum_analyze_threshold = 50
autovacuum_analyze_scale_factor = 0.1     # 10% de la tabla
autovacuum_naptime = 1min                 # Frecuencia de verificación
autovacuum_max_workers = 3                # Workers simultáneos
autovacuum_vacuum_cost_delay = 2ms        # Retraso para reducir I/O
```

Cómo funciona:

1. Cada minuto (autovacuum_naptime), el launcher verifica tablas modificadas
2. Para cada tabla, calcula si necesita VACUUM:

   text

   ```
   dead_tuples > threshold + (scale_factor * total_rows)
   ```
3. Lanza workers que ejecutan VACUUM y ANALYZE
4. Los workers respetan autovacuum_vacuum_cost_delay para no saturar I/O

Monitorear autovacuum:

sql

```
-- Ver workers activos
SELECT
    pid,
    datname,
    usename,
    application_name,
    query,
    state,
    wait_event_type,
    wait_event
FROM pg_stat_activity
WHERE query LIKE 'autovacuum%';
```

```
-- Ver historial de ejecución
SELECT
    schemaname,
    relname,
    last_vacuum,
    last_autovacuum,
    last_analyze,
    last_autoanalyze,
    vacuum_count,
    autovacuum_count,
    analyze_count,
    autoanalyze_count
FROM pg_stat_user_tables
WHERE relname = 'productos';
```

Ajustar autovacuum por tabla:

sql

```
-- Tabla muy activa (muchos UPDATEs)
ALTER TABLE transacciones SET (
    autovacuum_vacuum_scale_factor = 0.05,    -- 5% en lugar de 20%
    autovacuum_vacuum_threshold = 1000,
    autovacuum_vacuum_cost_delay = 10ms       -- Más agresivo
);

-- Tabla grande pero pocos cambios
ALTER TABLE logs_historicos SET (
    autovacuum_vacuum_scale_factor = 0.5,     -- 50%
    autovacuum_vacuum_threshold = 100000
);
```

Problemas comunes y soluciones

Problema 1: Autovacuum no sigue el ritmo

Síntomas:

sql

```
-- Muchas tuplas muertas acumuladas
SELECT relname, n_dead_tup, n_live_tup, last_autovacuum
FROM pg_stat_user_tables
WHERE n_dead_tup > 100000
ORDER BY n_dead_tup DESC;
```

Soluciones:

ini

```
# Reducir umbrales globales
autovacuum_vacuum_scale_factor = 0.1
autovacuum_vacuum_threshold = 1000

# Aumentar workers
autovacuum_max_workers = 5

# Reducir delay (más agresivo)
autovacuum_vacuum_cost_delay = 1ms
autovacuum_vacuum_cost_limit = 2000
```

Problema 2: Wraparound inminente

Síntoma:

sql

```
SELECT datname, age(datfrozenxid) FROM pg_database WHERE age(datfrozenxid) >
1000000000;
```

Solución de emergencia:

sql

```
-- Vacío agresivo en modo solo mantenimiento
VACUUM FREEZE VERBOSE tabla_peligrosa;

-- Para todo el cluster (como superusuario)
VACUUM FREEZE;
```

Problema 3: VACUUM tarda horas

Diagnóstico:

sql

```
-- Ver progreso de VACUUM (PostgreSQL 9.6+)
SELECT
    pid,
    datname,
    relid::regclass AS table_name,
    phase,
    heap_blks_total,
    heap_blks_scanned,
    heap_blks_vacuumed,
    index_vacuum_count,
    max_dead_tuples
FROM pg_stat_progress_vacuum;
```

Mejoras:

sql

```
-- Aumentar maintenance_work_mem para el VACUUM actual
SET maintenance_work_mem = '2GB';
VACUUM VERBOSE tabla_gigante;

-- Usar VACUUM paralelo (PostgreSQL 13+)
VACUUM (PARALLEL 4) tabla_gigante;
```

Ejercicio práctico 2.2

sql

```
-- 1. Crear tabla de prueba y generar dead tuples
CREATE TABLE test_vacuum (
    id serial PRIMARY KEY,
    datos text,
    fecha timestamp DEFAULT now()
);

INSERT INTO test_vacuum (datos)
SELECT 'fila_' || i FROM generate_series(1, 100000) i;
```

```
-- Ver estadísticas iniciales
SELECT relname, n_live_tup, n_dead_tup, last_vacuum
FROM pg_stat_user_tables WHERE relname = 'test_vacuum';

-- 2. Generar dead tuples (actualizar todas las filas)
UPDATE test_vacuum SET datos = 'actualizado_' || id, fecha = now();

-- Ver dead tuples generadas
SELECT relname, n_live_tup, n_dead_tup
FROM pg_stat_user_tables WHERE relname = 'test_vacuum';

-- 3. Ejecutar VACUUM y observar cambios
VACUUM VERBOSE test_vacuum;
SELECT relname, n_live_tup, n_dead_tup, last_vacuum
FROM pg_stat_user_tables WHERE relname = 'test_vacuum';

-- 4. Ver tamaño antes y después de VACUUM FULL
SELECT pg_size_pretty(pg_relation_size('test_vacuum')) AS antes;
VACUUM FULL test_vacuum;
SELECT pg_size_pretty(pg_relation_size('test_vacuum')) AS despues;

-- 5. Ver estadísticas de ANALYZE
ANALYZE VERBOSE test_vacuum;
SELECT * FROM pg_stats WHERE tablename = 'test_vacuum' AND attname = 'id';

-- 6. Simular wraparound (solo lectura, no ejecutar)
SELECT age(relfrozenxid) FROM pg_class WHERE relname = 'test_vacuum';
```

2.3 Reindexación y Bloat

¿Qué es el bloat (hinchazón)?

Bloat es el espacio desperdiciado en tablas e índices debido a:

- Tuplas muertas no limpiadas
- Páginas parcialmente llenas
- Fragmentación interna

Tipos de bloat:

1. **Bloat en tablas:** Espacio desperdiciado en el heap
2. **Bloat en índices:** Páginas de índice con entradas muertas

Consecuencias:

- Mayor uso de disco
- Más lecturas de I/O (peor rendimiento)
- Menor eficiencia de caché

Medir bloat

Medir bloat en tablas

sql

```
-- Consulta para estimar bloat en tablas (versión simplificada)
SELECT
    schemaname,
    relname,
    pg_size_pretty(pg_relation_size(relid)) AS tamaño_actual,
    pg_size_pretty(CAST((
        reltuples * (current_setting('block_size')::numeric / 1024) *
        (1 - (null_frac + (1 - null_frac) * (1 - avg_width /
(current_setting('block_size')::numeric / 1024))))
    ) AS bigint)) AS tamaño_estimado_minimo,
    round(100 * (1 - (reltuples * (current_setting('block_size')::numeric /
1024) /
        (pg_relation_size(relid) + 1))), 2) AS porcentaje_bloat
FROM pg_stat_user_tables
WHERE reltuples > 10000
ORDER BY porcentaje_bloat DESC;
```

Medir bloat en índices

sql

```
-- Extensión necesaria
CREATE EXTENSION IF NOT EXISTS pgstattuple;

-- Medir bloat de un índice específico
SELECT * FROM pgstatindex('idx_productos_precio');

-- Ver tuplas muertas en índices
SELECT
    schemaname,
    relname,
    indexrelname,
    idx_scan,
    idx_tup_read,
    idx_tup_fetch
FROM pg_stat_user_indexes
WHERE idx_scan = 0  -- Índices nunca usados
ORDER BY pg_relation_size(indexrelname) DESC;
```

REINDEX: Reconstrucción de índices

REINDEX reconstruye un índice desde cero, eliminando bloat y optimizando la estructura.

sql

```
-- Reindexar un índice específico
REINDEX INDEX idx_productos_precio;

-- Reindexar todos los índices de una tabla
REINDEX TABLE productos;

-- Reindexar toda una base de datos (requiere superusuario)
REINDEX DATABASE mi_base_datos;

-- Reindexar sin bloquear escrituras (PostgreSQL 12+)
REINDEX INDEX CONCURRENTLY idx_productos_precio;
```

```
REINDEX TABLE CONCURRENTLY productos;
```

Comparativa REINDEX normal vs CONCURRENTLY:

Característica	REINDEX normal	REINDEX CONCURRENTLY
Bloqueo	Exclusivo (bloquea escrituras)	No bloquea (solo breve al final)
Tiempo	Más rápido	2-3 veces más lento
Espacio extra	No requiere	Sí (índice temporal)
Disponible desde	Siempre	PostgreSQL 12

Cuándo reindexar:

- Después de VACUUM FULL (los índices se marcan como obsoletos)
- Cuando el bloat del índice supera el 20-30%
- Después de grandes actualizaciones masivas
- Como mantenimiento preventivo (mensual/trimestral)

Monitoreo y mantenimiento proactivo

Script de mantenimiento completo

sql

```sql
-- 1. Identificar tablas con alto bloat
SELECT
    relname,
    n_dead_tup,
    round(100 * n_dead_tup / (n_live_tup + 1)::numeric, 2) AS dead_ratio,
    pg_size_pretty(pg_relation_size(relid)) AS table_size
FROM pg_stat_user_tables
WHERE n_dead_tup > 10000
ORDER BY dead_ratio DESC
LIMIT 20;

-- 2. Generar comandos REINDEX para índices con bloat
SELECT
    'REINDEX INDEX CONCURRENTLY ' || indexrelname || ';' AS comando
FROM pg_stat_user_indexes
WHERE idx_scan > 0  -- Solo índices usados
AND pg_relation_size(indexrelname) > 10 * 1024 * 1024  -- > 10 MB
ORDER BY pg_relation_size(indexrelname) DESC;

-- 3. Tablas que necesitan VACUUM FULL (mucho bloat y pocas escrituras)
SELECT
    'VACUUM FULL VERBOSE ' || relname || ';' AS comando
FROM pg_stat_user_tables
WHERE n_dead_tup > n_live_tup * 0.5  -- Más del 50% de tuplas muertas
AND n_live_tup > 10000
ORDER BY n_dead_tup DESC;
```

Mantenimiento programado con pg_cron

sql

```sql
-- Programar mantenimiento semanal (requiere extensión pg_cron)
CREATE EXTENSION pg_cron;
```

```
-- Reindexar tablas grandes cada domingo a las 2 AM
SELECT cron.schedule(
    'reindex-semanal',           -- nombre
    '0 2 * * 0',                -- cron: Domingo 2 AM
    'REINDEX TABLE CONCURRENTLY ventas;'
);

-- Vacuum full de tablas archivadas cada mes
SELECT cron.schedule(
    'vacuum-full-mensual',
    '0 3 1 * *',                -- 1er día del mes a las 3 AM
    'VACUUM FULL VERBOSE logs_historial;'
);
```

Herramienta pg_repack (alternativa a VACUUM FULL sin bloqueo)

pg_repack es una extensión que permite hacer VACUUM FULL sin bloquear la tabla.

bash

```
# Instalación (Ubuntu/Debian)
sudo apt-get install postgresql-16-repack

# Uso desde línea de comandos
pg_repack -d mi_base_datos -t productos

# Reorganizar tabla y reindexar
pg_repack -d mi_base_datos -t productos --no-order --index
```

Ventajas sobre VACUUM FULL:

- No bloquea lecturas (permite SELECTs)
- Bloquea escrituras solo por breves momentos
- Puede ejecutarse en producción

Ejercicio práctico 2.3

sql

```
-- 1. Crear escenario con bloat en índices
CREATE TABLE test_bloat (id serial, nombre text, precio numeric);
CREATE INDEX idx_test_bloat_nombre ON test_bloat (nombre);
CREATE INDEX idx_test_bloat_precio ON test_bloat (precio);

-- Insertar datos
INSERT INTO test_bloat (nombre, precio)
SELECT 'producto_' || i, random() * 1000
FROM generate_series(1, 100000) i;

-- 2. Generar bloat (actualizaciones masivas)
UPDATE test_bloat SET nombre = 'nuevo_' || id, precio = precio * 1.1;

-- 3. Ver estadísticas de índice
SELECT
    indexrelname,
    idx_scan,
    idx_tup_read,
    pg_size_pretty(pg_relation_size(indexrelname)) AS tamaño
FROM pg_stat_user_indexes
```

```
WHERE relname = 'test_bloat';

-- 4. Reindexar y comparar
REINDEX INDEX CONCURRENTLY idx_test_bloat_nombre;

-- 5. Ver reducción de tamaño
SELECT
    indexrelname,
    pg_size_pretty(pg_relation_size(indexrelname)) AS nuevo_tamaño
FROM pg_stat_user_indexes
WHERE relname = 'test_bloat';

-- 6. Identificar índices duplicados o no usados
SELECT
    schemaname,
    tablename,
    indexname,
    idx_scan,
    pg_size_pretty(pg_relation_size(indexname::regclass)) AS size
FROM pg_stat_user_indexes
WHERE idx_scan = 0  -- Nunca usado
AND indexname NOT LIKE '%pkey%'  -- Excepto PK
ORDER BY pg_relation_size(indexname::regclass) DESC;
```

2.4 Backup y Recuperación

Tipos de backup en PostgreSQL

Tipo	Método	Granularidad	Tiempo recuperación	Uso típico
SQL dump (pg_dump)	Lógico	Tabla/BD	Horas	Migraciones, desarrollo
Custom format	Lógico	Tabla/BD	Horas	Backups programados
Physical backup (PITR)	Físico	Cluster completo	Minutos	Producción, DRP
pg_basebackup	Físico	Cluster completo	Minutos	Réplica, backup base

pg_dump: Backup lógico

Formatos disponibles

bash

```
# 1. SQL plano (más portable, más lento)
pg_dump mi_bd > backup.sql

# 2. Formato custom (compresión, paralelismo)
pg_dump -Fc mi_bd > backup.dump

# 3. Formato directorio (paralelo, permite reanudar)
pg_dump -Fd -j 4 -f backup_dir mi_bd

# 4. Formato tar
pg_dump -Ft mi_bd > backup.tar
```

Ejemplos prácticos

bash

```
# Backup de una sola tabla
pg_dump -t productos mi_bd > productos_backup.sql

# Backup excluyendo tablas
pg_dump -T logs_auditoria -T cache_temporal mi_bd > backup_sin_logs.sql

# Backup con compresión y paralelismo
pg_dump -Fd -j 8 -Z 6 -f backup_paralelo/ mi_bd

# Backup solo esquema (sin datos)
pg_dump -s mi_bd > esquema.sql

# Backup solo datos (sin esquema)
pg_dump -a mi_bd > datos.sql

# Backup condicional (solo filas que cumplen condición)
pg_dump -t ventas --where="fecha >= '2024-01-01'" mi_bd > ventas_2024.sql
```

Restauración con pg_restore

bash

```
# Restaurar backup custom (Fc)
pg_restore -d nueva_bd backup.dump

# Restaurar en paralelo
pg_restore -d nueva_bd -j 4 backup.dump

# Restaurar solo una tabla
pg_restore -t productos -d nueva_bd backup.dump

# Restaurar solo esquema
pg_restore -s -d nueva_bd backup.dump

# Restaurar con limpieza previa (DROP TABLE)
pg_restore --clean -d nueva_bd backup.dump

# Restaurar desde SQL plano
psql -d nueva_bd < backup.sql
```

pg_dumpall: Backup del cluster completo

bash

```
# Backup de todas las bases + roles + tablespaces
pg_dumpall > cluster_backup.sql

# Backup solo de roles y configuraciones
pg_dumpall --globals-only > roles_y_config.sql

# Backup solo de roles
pg_dumpall --roles-only > solo_roles.sql

# Restaurar cluster completo
psql -f cluster_backup.sql postgres
```

PITR (Point-In-Time Recovery) - Backup físico

PITR permite recuperar hasta un momento específico usando WAL archiving.

Configuración de WAL archiving

postgresql.conf:

ini

```
# Habilitar archivo WAL
archive_mode = on
archive_command = 'test ! -f /mnt/backups/wal/%f && cp %p /mnt/backups/wal/%f'

# Para compresión (ahorra espacio)
archive_command = 'gzip < %p > /mnt/backups/wal/%f.gz'

# Para enviar a S3 (requiere awscli)
archive_command = 'aws s3 cp %p s3://mi-bucket/wal/%f'

# Para retener WALs para replicación
wal_keep_size = 1024MB

# Compresión WAL
wal_compression = on
```

Crear backup base

bash

```
# Método 1: Usando pg_basebackup (recomendado)
pg_basebackup -D /mnt/backups/base_backup -Ft -z -P

# Método 2: Usando tar (manual)
tar -czf /mnt/backups/base_backup.tar.gz $PGDATA

# Método 3: Usando rsync (para backups incrementales)
rsync -av --delete $PGDATA/ /mnt/backups/pgdata_backup/
```

Recuperación Point-In-Time

Pasos para recuperar:

1. **Preparar archivo recovery.conf** (PostgreSQL < 12) o **postgresql.auto.conf** (>= 12)

   bash

   ```
   # En $PGDATA/ (PostgreSQL 12+)
   cat > $PGDATA/postgresql.auto.conf << EOF
   restore_command = 'cp /mnt/backups/wal/%f %p'
   recovery_target_time = '2024-12-25 15:30:00'
   recovery_target_action = 'promote'
   EOF
   ```

2. **Restaurar backup base**

   bash

   ```
   rm -rf $PGDATA/*
   tar -xzf /mnt/backups/base_backup.tar.gz -C $PGDATA/
   ```

3. **Iniciar PostgreSQL en modo recovery**

   bash

   ```
   pg_ctl start -D $PGDATA
   # PostgreSQL aplicará WALs hasta el tiempo especificado
   ```

4. **Verificar recuperación**

sql

```
SELECT pg_is_in_recovery();  -- false = recovery completado
SELECT pg_wal_replay_pause();  -- Para pausar recovery
```

Opciones de recovery_target:

sql

```
-- Tiempo exacto
recovery_target_time = '2024-12-25 15:30:00'

-- Transacción específica
recovery_target_xid = '1234567'

-- LSN específico
recovery_target_lsn = '0/30001E8'

-- Modo "antes de" o "después de"
recovery_target_inclusive = false  -- Recuperar justo antes del evento

-- Detener en estado consistente (no en medio de transacción)
recovery_target = 'immediate'
```

pg_basebackup: Réplica y backup en caliente

bash

```
# Backup básico
pg_basebackup -D /backups/base -Fp -P -U replicador

# Backup en formato tar (comprimido)
pg_basebackup -D /backups/base -Ft -z -P

# Backup con progreso y verificación
pg_basebackup -D /backups/base -Fp -P -v --checkpoint=fast

# Backup solo para réplica (incluye configuración)
pg_basebackup -D /var/lib/postgresql/16/replica -Fp -P -R -U replicador -h
maestro_ip

# Backup con slots de replicación
pg_basebackup -D /backups/base -Fp -P --slot=backup_slot
```

Estrategias de backup recomendadas

Estrategia 1: Backup completo semanal + WAL diario

bash

```
# Domingo 1 AM: Backup completo
0 1 * * 0 pg_basebackup -D /backups/semanal/$(date +\%Y\%m\%d) -Ft -z

# Cada hora: Archivar WALs (ya configurado en archive_command)
# Recuperación: hasta 1 hora de pérdida

# Limpieza: mantener 4 semanas
0 2 * * 0 find /backups/semanal -type d -mtime +28 -exec rm -rf {} \;
```

Estrategia 2: pg_dump diario + WAL archiving

bash

```
# Diario 2 AM: Backup lógico
0 2 * * * pg_dump -Fc -Z 6 mi_bd > /backups/diario/bd_$(date +\%Y\%m\%d).dump

# Recuperación: pérdida de hasta 24 horas
# Más lento pero más portable
```

Estrategia 3: Barman (backup manager profesional)

bash

```
# Instalar Barman
sudo apt-get install barman

# Configurar servidor en /etc/barman.conf
[barman]
backup_directory = /var/lib/barman
wal_retention_policy = REDUNDANCY 2

[mi_postgres]
host = 192.168.1.10
user = barman_user
backup_method = rsync
streaming_archiver = on

# Crear backup
barman backup mi_postgres

# Listar backups
barman list-backup mi_postgres

# Recuperar
barman recover mi_postgres 20241225T010001 /ruta/recuperacion
```

Verificar integridad de backups

bash

```
# Verificar backup lógico
pg_restore --list backup.dump > /dev/null && echo "Backup válido"

# Verificar checksums de backup físico
pg_verifybackup /mnt/backups/base_backup

# Probar restauración en entorno de testing
pg_restore -d test_db backup.dump
```

Ejercicio práctico 2.4

sql

```
-- 1. Crear base de datos de prueba
CREATE DATABASE backup_test;
\c backup_test

CREATE TABLE empleados (id serial, nombre text, salario numeric);
INSERT INTO empleados (nombre, salario) VALUES
    ('Ana', 50000), ('Juan', 60000), ('Maria', 55000);

-- 2. Backup en consola (ejecutar en bash)
-- pg_dump backup_test > backup_test.sql

-- 3. Simular desastre (borrar datos)
DROP TABLE empleados;
```

```
-- 4. Restaurar (ejecutar en bash)
-- psql -d backup_test < backup_test.sql

-- 5. Verificar restauración
SELECT * FROM empleados;

-- 6. Backup con compresión y paralelismo
-- pg_dump -Fc -Z 9 -j 4 backup_test > backup_test.dump

-- 7. Restaurar desde formato custom
-- pg_restore -d backup_test --clean backup_test.dump
```

2.5 Monitoreo y Tuning

Vistas de monitoreo esenciales

pg_stat_activity: Conexiones activas

sql

```
-- Ver todas las conexiones activas
SELECT
    pid,
    usename,
    datname,
    application_name,
    client_addr,
    state,
    wait_event_type,
    wait_event,
    query,
    query_start,
    state_change
FROM pg_stat_activity
WHERE state != 'idle'
ORDER BY query_start;

-- Identificar consultas bloqueadas (locks)
SELECT
    blocked.pid AS blocked_pid,
    blocked.query AS blocked_query,
    blocking.pid AS blocking_pid,
    blocking.query AS blocking_query
FROM pg_stat_activity blocked
JOIN pg_locks blocked_locks ON blocked.pid = blocked_locks.pid
JOIN pg_locks blocking_locks ON blocked_locks.locktype = blocking_locks.locktype
    AND blocked_locks.relation = blocking_locks.relation
    AND blocked_locks.pid != blocking_locks.pid
JOIN pg_stat_activity blocking ON blocking.pid = blocking_locks.pid
WHERE NOT blocked_locks.granted;

-- Matar consulta problemática
SELECT pg_terminate_backend(pid);  -- reemplazar con PID real
```

pg_locks: Bloqueos

sql

```
-- Ver todos los bloqueos activos
SELECT
    locktype,
    relation::regclass AS table,
    page,
    tuple,
    mode,
    granted,
    pid,
    transactionid
FROM pg_locks
WHERE NOT granted  -- Bloqueos no concedidos (esperando)
ORDER BY pid;

-- Bloqueos por tabla
SELECT
    relation::regclass,
    mode,
    count(*) AS cantidad
FROM pg_locks
WHERE relation IS NOT NULL
GROUP BY relation, mode
ORDER BY cantidad DESC;
```

pg_stat_user_tables: Estadísticas de tablas

sql

```
-- Tablas más activas (escrituras)
SELECT
    relname,
    n_tup_ins AS inserts,
    n_tup_upd AS updates,
    n_tup_del AS deletes,
    n_live_tup,
    n_dead_tup,
    round(100 * n_dead_tup / (n_live_tup + 1)::numeric, 2) AS dead_ratio,
    last_vacuum,
    last_autovacuum
FROM pg_stat_user_tables
ORDER BY n_tup_upd + n_tup_ins + n_tup_del DESC
LIMIT 20;

-- Secans (full table scans) vs index scans
SELECT
    relname,
    seq_scan,
    seq_tup_read,
    idx_scan,
    idx_tup_fetch,
    round(100 * idx_scan / (seq_scan + idx_scan + 1)::numeric, 2) AS
index_usage_pct
FROM pg_stat_user_tables
WHERE seq_scan + idx_scan > 0
ORDER BY seq_scan DESC
LIMIT 20;
```

pg_stat_statements: Consultas más costosas

sql

```
-- Requiere extensión
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;
```

```
-- Consultas más lentas (tiempo total)
SELECT
    query,
    calls,
    total_exec_time,
    mean_exec_time,
    max_exec_time,
    rows,
    wal_bytes
FROM pg_stat_statements
ORDER BY total_exec_time DESC
LIMIT 20;

-- Consultas con más I/O
SELECT
    query,
    calls,
    total_exec_time,
    shared_blks_hit,
    shared_blks_read,
    shared_blks_dirtied,
    shared_blks_written
FROM pg_stat_statements
ORDER BY shared_blks_read DESC
LIMIT 20;

-- Limpiar estadísticas
SELECT pg_stat_statements_reset();
```

pg_stat_bgwriter: Estadísticas del background writer

sql

```
-- Monitoreo de checkpoints y escrituras
SELECT
    checkpoints_timed,
    checkpoints_req,
    checkpoint_write_time,
    checkpoint_sync_time,
    buffers_checkpoint,
    buffers_clean,
    buffers_backend,
    buffers_backend_fsync
FROM pg_stat_bgwriter;

-- Calcular eficiencia de checkpoints
SELECT
    checkpoints_timed,
    checkpoints_req,
    round(100 * checkpoints_timed / (checkpoints_timed
```

y ahora todo el modulo 3, 4 ,5 y6

Aquí tienes el desarrollo completo de los **Módulos 3, 4, 5 y 6** del curso de PostgreSQL, cubriendo consultas básicas, funciones de agregación, funciones de ventana y consultas avanzadas con optimización.

Módulo 3: Consultas Básicas y Fundamentos SQL

3.1 Repaso de SELECT, INSERT, UPDATE, DELETE

SELECT fundamentos

sql

```
-- Sintaxis básica
SELECT columna1, columna2, columna3
FROM tabla
WHERE condicion
ORDER BY columna1 [ASC|DESC]
LIMIT n OFFSET m;

-- Ejemplos prácticos
-- Seleccionar todas las columnas
SELECT * FROM clientes;

-- Seleccionar con alias
SELECT
    nombre AS cliente_nombre,
    email AS correo_electronico,
    edad AS "Edad del Cliente"  -- Comillas dobles para mayúsculas/espacios
FROM clientes;

-- Seleccionar con expresiones
SELECT
    nombre,
    precio,
    precio * 1.21 AS precio_con_iva,
    round(precio * 1.21, 2) AS precio_redondeado
FROM productos;

-- Seleccionar valores únicos
SELECT DISTINCT categoria_id FROM productos;
SELECT DISTINCT ON (categoria_id) categoria_id, nombre FROM productos;
```

WHERE: Filtrado avanzado

sql

```
-- Operadores de comparación
SELECT * FROM productos
WHERE precio BETWEEN 100 AND 500;

SELECT * FROM clientes
WHERE edad IN (25, 30, 35);

SELECT * FROM usuarios
WHERE nombre LIKE 'J%';  -- Empieza con J

SELECT * FROM usuarios
WHERE nombre ILIKE 'j%';  -- Case-insensitive

SELECT * FROM pedidos
```

```
WHERE fecha BETWEEN '2024-01-01' AND '2024-12-31';

-- Operadores lógicos
SELECT * FROM productos
WHERE (precio > 100 OR stock < 10)
  AND categoria_id = 5
  AND NOT descontinuado;

-- Valores NULL
SELECT * FROM clientes
WHERE telefono IS NULL;

SELECT * FROM clientes
WHERE email IS NOT NULL;
```

ORDER BY, LIMIT, OFFSET

sql

```
-- Ordenamiento múltiple
SELECT * FROM productos
ORDER BY categoria_id ASC, precio DESC, nombre;

-- Ordenamiento por expresión
SELECT * FROM pedidos
ORDER BY (total * 1.21) DESC;

-- Paginación
SELECT * FROM productos
ORDER BY id
LIMIT 10 OFFSET 20;  -- Página 3 (registros 21-30)

-- Top N por grupo (con DISTINCT ON)
SELECT DISTINCT ON (categoria_id)
    categoria_id,
    nombre,
    precio
FROM productos
ORDER BY categoria_id, precio DESC;
```

JOINs: Combinando tablas

sql

```
-- INNER JOIN (solo coincidencias)
SELECT
    p.nombre AS producto,
    c.nombre AS categoria,
    p.precio
FROM productos p
INNER JOIN categorias c ON p.categoria_id = c.id;

-- LEFT JOIN (todos los productos, tengan categoría o no)
SELECT
    p.nombre AS producto,
    COALESCE(c.nombre, 'Sin categoría') AS categoria
FROM productos p
LEFT JOIN categorias c ON p.categoria_id = c.id;

-- RIGHT JOIN (todas las categorías, tengan productos o no)
SELECT
    COALESCE(p.nombre, 'Sin productos') AS producto,
```

```sql
    c.nombre AS categoria
FROM productos p
RIGHT JOIN categorias c ON p.categoria_id = c.id;

-- FULL JOIN (ambos lados)
SELECT
    p.nombre AS producto,
    c.nombre AS categoria
FROM productos p
FULL JOIN categorias c ON p.categoria_id = c.id;

-- CROSS JOIN (producto cartesiano)
SELECT p.nombre, t.talla
FROM productos p
CROSS JOIN tallas t;

-- SELF JOIN (unirse a sí misma)
SELECT
    e1.nombre AS empleado,
    e2.nombre AS jefe
FROM empleados e1
LEFT JOIN empleados e2 ON e1.jefe_id = e2.id;
```

Subconsultas

sql

```sql
-- Subconsulta escalar (una fila, una columna)
SELECT
    nombre,
    precio,
    (SELECT AVG(precio) FROM productos) AS precio_promedio,
    precio - (SELECT AVG(precio) FROM productos) AS diferencia
FROM productos;

-- Subconsulta de fila (una fila, varias columnas)
SELECT * FROM productos
WHERE (categoria_id, precio) = (
    SELECT categoria_id, MAX(precio)
    FROM productos
    GROUP BY categoria_id
    LIMIT 1
);

-- Subconsulta de tabla (múltiples filas/columnas)
SELECT p.nombre, p.precio, c.nombre AS categoria
FROM productos p
INNER JOIN (
    SELECT id, nombre
    FROM categorias
    WHERE activa = true
) c ON p.categoria_id = c.id;

-- Subconsulta correlacionada (referencia a la tabla externa)
SELECT
    p1.nombre,
    p1.precio,
    p1.categoria_id,
    (SELECT AVG(p2.precio)
     FROM productos p2
     WHERE p2.categoria_id = p1.categoria_id) AS promedio_categoria
FROM productos p1;
```

Subconsultas con IN, EXISTS, ANY, ALL

sql

```
-- IN (pertenece a un conjunto)
SELECT * FROM productos
WHERE categoria_id IN (
    SELECT id FROM categorias WHERE activa = true
);

-- EXISTS (existe al menos uno)
SELECT * FROM categorias c
WHERE EXISTS (
    SELECT 1 FROM productos p
    WHERE p.categoria_id = c.id AND p.precio > 1000
);

-- ANY (compara con cualquier elemento)
SELECT * FROM productos
WHERE precio > ANY (
    SELECT precio FROM productos WHERE categoria_id = 1
);

-- ALL (compara con todos los elementos)
SELECT * FROM productos
WHERE precio > ALL (
    SELECT precio FROM productos WHERE categoria_id = 1
);
```

INSERT, UPDATE, DELETE avanzados

sql

```
-- INSERT múltiple
INSERT INTO productos (nombre, precio, categoria_id) VALUES
    ('Producto A', 100, 1),
    ('Producto B', 200, 1),
    ('Producto C', 150, 2);

-- INSERT desde SELECT
INSERT INTO productos_auditoria (producto_id, nombre, fecha)
SELECT id, nombre, now() FROM productos WHERE precio > 1000;

-- INSERT con RETURNING
INSERT INTO pedidos (cliente_id, total)
VALUES (1, 500)
RETURNING id, fecha, total;

-- UPDATE condicional
UPDATE productos
SET precio = precio * 1.10,
    ultima_actualizacion = now()
WHERE categoria_id = 1
  AND stock > 0
RETURNING id, nombre, precio;

-- UPDATE desde otra tabla
UPDATE productos p
SET stock = p.stock - d.cantidad
FROM detalle_pedidos d
WHERE p.id = d.producto_id
  AND d.pedido_id = 123;
```

```
-- DELETE con USING
DELETE FROM productos p
USING inventario i
WHERE p.id = i.producto_id
  AND i.cantidad = 0;

-- DELETE con RETURNING
DELETE FROM productos
WHERE precio < 10
RETURNING id, nombre, precio;
```

Ejercicios prácticos 3.1

sql

```
-- 1. Crear esquema de práctica
CREATE SCHEMA modulo3;
SET search_path TO modulo3;

CREATE TABLE clientes (
    id serial PRIMARY KEY,
    nombre varchar(100),
    email varchar(100) UNIQUE,
    edad int,
    ciudad varchar(50)
);

CREATE TABLE categorias (
    id serial PRIMARY KEY,
    nombre varchar(50),
    activa boolean DEFAULT true
);

CREATE TABLE productos (
    id serial PRIMARY KEY,
    nombre varchar(100),
    precio numeric(10,2),
    categoria_id int REFERENCES categorias(id),
    stock int DEFAULT 0
);

CREATE TABLE pedidos (
    id serial PRIMARY KEY,
    cliente_id int REFERENCES clientes(id),
    fecha date DEFAULT CURRENT_DATE,
    total numeric(10,2)
);

CREATE TABLE detalle_pedidos (
    pedido_id int REFERENCES pedidos(id),
    producto_id int REFERENCES productos(id),
    cantidad int,
    precio_unitario numeric(10,2),
    PRIMARY KEY (pedido_id, producto_id)
);

-- 2. Insertar datos
INSERT INTO categorias (nombre) VALUES
    ('Electrónica'), ('Ropa'), ('Libros'), ('Hogar');

INSERT INTO productos (nombre, precio, categoria_id, stock) VALUES
```

```
    ('Laptop', 1200, 1, 10),
    ('Mouse', 25, 1, 50),
    ('Camisa', 45, 2, 30),
    ('Novela', 20, 3, 100),
    ('Mesa', 300, 4, 5);

INSERT INTO clientes (nombre, email, edad, ciudad) VALUES
    ('Ana García', 'ana@mail.com', 28, 'Madrid'),
    ('Juan López', 'juan@mail.com', 35, 'Barcelona'),
    ('María Ruiz', 'maria@mail.com', 22, 'Valencia');

-- 3. Ejercicios a resolver
-- a) Listar productos con precio > 100 y stock > 0
-- b) Mostrar todos los pedidos con nombre del cliente
-- c) Productos que nunca se han vendido
-- d) Clientes que no han hecho pedidos
-- e) Top 3 productos más caros por categoría
```

Módulo 4: Funciones de Agregación

4.1 Agregaciones estándar

sql

```
-- Agregaciones básicas
SELECT
    COUNT(*) AS total_filas,
    COUNT(precio) AS precios_no_nulos,
    COUNT(DISTINCT categoria_id) AS categorias_distintas,
    SUM(precio) AS suma_total,
    AVG(precio) AS precio_promedio,
    MIN(precio) AS precio_minimo,
    MAX(precio) AS precio_maximo,
    STDDEV(precio) AS desviacion_estandar,
    VARIANCE(precio) AS varianza
FROM productos;

-- Agregaciones con strings
SELECT
    STRING_AGG(nombre, ', ' ORDER BY nombre) AS lista_productos,
    ARRAY_AGG(nombre ORDER BY precio DESC) AS array_productos
FROM productos
WHERE categoria_id = 1;
```

GROUP BY: Agrupación

sql

```
-- Agrupación simple
SELECT
    categoria_id,
    COUNT(*) AS cantidad,
    AVG(precio) AS precio_promedio,
    SUM(stock) AS stock_total
FROM productos
GROUP BY categoria_id;

-- Agrupación múltiple
SELECT
```

```
    categoria_id,
    EXTRACT(YEAR FROM fecha) AS año,
    EXTRACT(MONTH FROM fecha) AS mes,
    SUM(total) AS ventas_totales
FROM pedidos p
JOIN detalle_pedidos d ON p.id = d.pedido_id
JOIN productos prod ON d.producto_id = prod.id
GROUP BY categoria_id, año, mes
ORDER BY categoria_id, año, mes;

-- GROUP BY con expresiones
SELECT
    CASE
        WHEN precio < 50 THEN 'Económico'
        WHEN precio < 200 THEN 'Medio'
        ELSE 'Premium'
    END AS rango_precio,
    COUNT(*) AS cantidad,
    AVG(precio) AS promedio
FROM productos
GROUP BY rango_precio;
```

HAVING: Filtrado sobre agregaciones

sql

```
-- WHERE vs HAVING
-- WHERE: filtra antes de agrupar
-- HAVING: filtra después de agrupar

SELECT
    categoria_id,
    COUNT(*) AS cantidad,
    AVG(precio) AS precio_promedio
FROM productos
WHERE stock > 0  -- Filtra productos sin stock ANTES de agrupar
GROUP BY categoria_id
HAVING COUNT(*) >= 2  -- Solo categorías con al menos 2 productos
   AND AVG(precio) > 100;  -- Promedio mayor a 100
```

FILTER clause (agregación condicional)

sql

```
-- Sintaxis con FILTER (PostgreSQL específico)
SELECT
    categoria_id,
    COUNT(*) AS total_productos,
    COUNT(*) FILTER (WHERE stock > 0) AS con_stock,
    COUNT(*) FILTER (WHERE stock = 0) AS sin_stock,
    AVG(precio) AS promedio_total,
    AVG(precio) FILTER (WHERE precio > 100) AS promedio_premium,
    SUM(precio) FILTER (WHERE stock > 0) AS valor_inventario
FROM productos
GROUP BY categoria_id;

-- Equivalente sin FILTER (más verboso)
SELECT
    categoria_id,
    COUNT(*) AS total_productos,
    SUM(CASE WHEN stock > 0 THEN 1 ELSE 0 END) AS con_stock,
    SUM(CASE WHEN stock = 0 THEN 1 ELSE 0 END) AS sin_stock
```

```
FROM productos
GROUP BY categoria_id;
```

GROUPING SETS, ROLLUP, CUBE

sql

```
-- GROUPING SETS: múltiples agrupaciones en una consulta
SELECT
    categoria_id,
    EXTRACT(YEAR FROM fecha) AS año,
    SUM(total) AS ventas
FROM pedidos p
JOIN detalle_pedidos d ON p.id = d.pedido_id
GROUP BY GROUPING SETS (
    (categoria_id, año),  -- Por categoría y año
    (categoria_id),       -- Solo por categoría
    (año),                -- Solo por año
    ()                    -- Total general
);

-- ROLLUP: jerarquía de totales (cat > año > total)
SELECT
    categoria_id,
    EXTRACT(YEAR FROM fecha) AS año,
    EXTRACT(MONTH FROM fecha) AS mes,
    SUM(total) AS ventas
FROM pedidos p
GROUP BY ROLLUP (categoria_id, año, mes);

-- CUBE: todas las combinaciones posibles
SELECT
    categoria_id,
    EXTRACT(YEAR FROM fecha) AS año,
    SUM(total) AS ventas
FROM pedidos p
GROUP BY CUBE (categoria_id, año);

-- GROUPING(): identificar filas de subtotal
SELECT
    categoria_id,
    EXTRACT(YEAR FROM fecha) AS año,
    SUM(total) AS ventas,
    GROUPING(categoria_id) AS es_total_categoria,
    GROUPING(año) AS es_total_año
FROM pedidos p
GROUP BY ROLLUP (categoria_id, año);
```

Ejemplos prácticos de agregación

sql

```
-- 1. Ventas por cliente con ranking
SELECT
    c.nombre,
    COUNT(p.id) AS num_pedidos,
    SUM(p.total) AS total_gastado,
    AVG(p.total) AS ticket_medio,
    MIN(p.total) AS pedido_minimo,
    MAX(p.total) AS pedido_maximo
FROM clientes c
LEFT JOIN pedidos p ON c.id = p.cliente_id
```

```
GROUP BY c.id, c.nombre
ORDER BY total_gastado DESC NULLS LAST;

-- 2. Productos más vendidos
SELECT
    prod.nombre,
    SUM(dp.cantidad) AS unidades_vendidas,
    SUM(dp.cantidad * dp.precio_unitario) AS ingresos,
    COUNT(DISTINCT dp.pedido_id) AS num_pedidos
FROM productos prod
LEFT JOIN detalle_pedidos dp ON prod.id = dp.producto_id
GROUP BY prod.id, prod.nombre
ORDER BY unidades_vendidas DESC NULLS LAST
LIMIT 10;

-- 3. Análisis de cohortes mensuales
SELECT
    DATE_TRUNC('month', fecha) AS mes,
    COUNT(DISTINCT cliente_id) AS clientes_unicos,
    COUNT(*) AS pedidos,
    SUM(total) AS ingresos,
    AVG(total) AS ticket_promedio
FROM pedidos
GROUP BY mes
ORDER BY mes;

-- 4. Porcentaje del total (con ventana o subconsulta)
SELECT
    categoria_id,
    SUM(precio) AS suma_por_categoria,
    (SUM(precio) / (SELECT SUM(precio) FROM productos) * 100) AS porcentaje
FROM productos
GROUP BY categoria_id;
```

Ejercicios prácticos 4.1

sql

```
-- 1. Calcular total de ventas por cliente y por mes
SELECT
    c.nombre,
    EXTRACT(YEAR FROM p.fecha) AS año,
    EXTRACT(MONTH FROM p.fecha) AS mes,
    SUM(p.total) AS total_mes
FROM clientes c
JOIN pedidos p ON c.id = p.cliente_id
GROUP BY c.nombre, año, mes
ORDER BY c.nombre, año, mes;

-- 2. Categorías con ventas > 1000
SELECT
    cat.nombre AS categoria,
    SUM(dp.cantidad * dp.precio_unitario) AS ventas_totales
FROM categorias cat
JOIN productos prod ON cat.id = prod.categoria_id
JOIN detalle_pedidos dp ON prod.id = dp.producto_id
GROUP BY cat.nombre
HAVING SUM(dp.cantidad * dp.precio_unitario) > 1000;

-- 3. Usando FILTER para análisis de temporada
SELECT
    EXTRACT(YEAR FROM fecha) AS año,
```

```
    COUNT(*) AS total_pedidos,
    COUNT(*) FILTER (WHERE EXTRACT(MONTH FROM fecha) IN (12,1,2)) AS invierno,
    COUNT(*) FILTER (WHERE EXTRACT(MONTH FROM fecha) IN (3,4,5)) AS primavera,
    COUNT(*) FILTER (WHERE EXTRACT(MONTH FROM fecha) IN (6,7,8)) AS verano,
    COUNT(*) FILTER (WHERE EXTRACT(MONTH FROM fecha) IN (9,10,11)) AS otoño
FROM pedidos
GROUP BY año;

-- 4. ROLLUP para informe jerárquico
SELECT
    COALESCE(c.nombre, 'TODOS LOS CLIENTES') AS cliente,
    COALESCE(EXTRACT(YEAR FROM p.fecha)::text, 'TODOS LOS AÑOS') AS año,
    SUM(p.total) AS total
FROM clientes c
RIGHT JOIN pedidos p ON c.id = p.cliente_id
GROUP BY ROLLUP (c.nombre, EXTRACT(YEAR FROM p.fecha))
ORDER BY c.nombre NULLS LAST, año NULLS LAST;
```

Módulo 5: Funciones de Ventana (Window Functions)

5.1 Conceptos básicos

Las **window functions** realizan cálculos a través de un conjunto de filas relacionadas con la fila actual, SIN colapsar las filas como GROUP BY.

sql

```
-- Sintaxis básica
funcion_ventana() OVER (
    [PARTITION BY columna1, columna2, ...]
    [ORDER BY columna1 [ASC|DESC]]
    [frame_clause]
)

-- Diferencia fundamental con GROUP BY
-- GROUP BY: reduce filas (una fila por grupo)
SELECT categoria_id, AVG(precio) FROM productos GROUP BY categoria_id;

-- Window function: mantiene todas las filas, añade columna calculada
SELECT
    nombre,
    precio,
    categoria_id,
    AVG(precio) OVER (PARTITION BY categoria_id) AS promedio_categoria
FROM productos;
```

Funciones de ranking

sql

```
-- ROW_NUMBER(): número secuencial único por partición
SELECT
    nombre,
    precio,
    categoria_id,
    ROW_NUMBER() OVER (PARTITION BY categoria_id ORDER BY precio DESC) AS
posicion
FROM productos;
```

```
-- RANK(): mismo número para empates, saltos
SELECT
    nombre,
    precio,
    RANK() OVER (ORDER BY precio DESC) AS ranking,
    DENSE_RANK() OVER (ORDER BY precio DESC) AS ranking_sin_saltos
FROM productos;

-- NTILE(n): divide en n grupos
SELECT
    nombre,
    precio,
    NTILE(4) OVER (ORDER BY precio DESC) AS cuartil
FROM productos;
```

Funciones de acceso (LAG, LEAD, FIRST_VALUE, LAST_VALUE)

sql

```
-- LAG(): valor de fila anterior
SELECT
    fecha,
    total,
    LAG(total, 1) OVER (ORDER BY fecha) AS total_dia_anterior,
    LAG(total, 1, 0) OVER (ORDER BY fecha) AS anterior_con_default,
    total - LAG(total, 1, total) OVER (ORDER BY fecha) AS diferencia
FROM ventas_diarias;

-- LEAD(): valor de fila siguiente
SELECT
    fecha,
    total,
    LEAD(total, 1) OVER (ORDER BY fecha) AS total_dia_siguiente,
    LEAD(total, 7) OVER (ORDER BY fecha) AS total_hace_7_dias
FROM ventas_diarias;

-- FIRST_VALUE() / LAST_VALUE()
SELECT
    categoria_id,
    nombre,
    precio,
    FIRST_VALUE(precio) OVER (PARTITION BY categoria_id ORDER BY precio) AS
precio_minimo,
    LAST_VALUE(precio) OVER (PARTITION BY categoria_id ORDER BY precio
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS
precio_maximo
FROM productos;
```

5.2 Marcos (Frames)

Los frames definen qué filas se incluyen en el cálculo relativo a la fila actual.

sql

```
-- Frame syntax
ROWS | RANGE | GROUPS
BETWEEN frame_start AND frame_end

-- frame_start options: UNBOUNDED PRECEDING, n PRECEDING, CURRENT ROW
-- frame_end options: CURRENT ROW, n FOLLOWING, UNBOUNDED FOLLOWING

-- Ejemplos de frames
```

```
-- Frame: todas las filas desde el inicio hasta la actual
SUM(total) OVER (ORDER BY fecha ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT
ROW)

-- Frame: fila anterior, actual y siguiente
AVG(total) OVER (ORDER BY fecha ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING)

-- Frame: últimas 7 filas (incluyendo actual)
AVG(total) OVER (ORDER BY fecha ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)

-- Frame: solo la fila actual (equivalente a sin frame)
total - LAG(total) OVER (ORDER BY fecha)
```

Promedios móviles y acumulados

sql

```
-- 1. Acumulado (running total)
SELECT
    fecha,
    total,
    SUM(total) OVER (ORDER BY fecha) AS acumulado,
    COUNT(*) OVER (ORDER BY fecha) AS pedidos_acumulados
FROM pedidos
ORDER BY fecha;

-- 2. Promedio móvil de 7 días
SELECT
    fecha,
    total,
    AVG(total) OVER (ORDER BY fecha ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS
media_movil_7dias
FROM ventas_diarias;

-- 3. Máximo móvil (peak)
SELECT
    fecha,
    total,
    MAX(total) OVER (ORDER BY fecha ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT
ROW) AS maximo_hasta_fecha
FROM ventas_diarias;

-- 4. Diferencia con fila anterior (sin LAG)
SELECT
    fecha,
    total,
    total - FIRST_VALUE(total) OVER (ORDER BY fecha ROWS BETWEEN 1 PRECEDING AND
1 PRECEDING) AS diferencia
FROM ventas_diarias;
```

5.3 Agregaciones como ventana

sql

```
-- Usar funciones de agregación como window functions
SELECT
    p.nombre,
    p.precio,
    p.categoria_id,
    AVG(p.precio) OVER (PARTITION BY p.categoria_id) AS promedio_categoria,
    SUM(p.precio) OVER (PARTITION BY p.categoria_id) AS suma_categoria,
    COUNT(*) OVER (PARTITION BY p.categoria_id) AS total_productos_categoria,
```

```
    p.precio - AVG(p.precio) OVER (PARTITION BY p.categoria_id) AS
diferencia_promedio
FROM productos p;

-- Comparación con GROUP BY (mismo cálculo, diferente resultado)
-- GROUP BY: 1 fila por categoría
SELECT categoria_id, AVG(precio) FROM productos GROUP BY categoria_id;

-- Window: todas las filas + columna adicional
SELECT nombre, precio, categoria_id, AVG(precio) OVER (PARTITION BY
categoria_id) FROM productos;
```

5.4 Casos de uso reales

Caso 1: Ranking de ventas por región

sql

```
SELECT
    vendedor_id,
    region,
    ventas,
    RANK() OVER (PARTITION BY region ORDER BY ventas DESC) AS ranking_region,
    ROW_NUMBER() OVER (ORDER BY ventas DESC) AS ranking_global
FROM vendedores
ORDER BY region, ranking_region;
```

Caso 2: Detectar duplicados

sql

```
WITH duplicados AS (
    SELECT
        *,
        ROW_NUMBER() OVER (PARTITION BY email ORDER BY id) AS rn
    FROM clientes
)
SELECT * FROM duplicados WHERE rn > 1;

-- Eliminar duplicados (dejando el primero)
DELETE FROM clientes
WHERE id IN (
    SELECT id FROM (
        SELECT
            id,
            ROW_NUMBER() OVER (PARTITION BY email ORDER BY id) AS rn
        FROM clientes
    ) duplicados
    WHERE rn > 1
);
```

Caso 3: Diferencia entre compras de un cliente

sql

```
SELECT
    cliente_id,
    fecha,
    total,
    LAG(fecha) OVER (PARTITION BY cliente_id ORDER BY fecha) AS
fecha_compra_anterior,
    fecha - LAG(fecha) OVER (PARTITION BY cliente_id ORDER BY fecha) AS
dias_entre_compras,
```

```
    LAG(total) OVER (PARTITION BY cliente_id ORDER BY fecha) AS monto_anterior,
    total - LAG(total) OVER (PARTITION BY cliente_id ORDER BY fecha) AS
diferencia_monto
FROM pedidos
ORDER BY cliente_id, fecha;
```

Caso 4: Porcentaje del total por categoría

sql

```
SELECT
    categoria_id,
    nombre,
    precio,
    precio / SUM(precio) OVER () * 100 AS porcentaje_global,
    precio / SUM(precio) OVER (PARTITION BY categoria_id) * 100 AS
porcentaje_categoria
FROM productos;
```

Caso 5: Saldo acumulado (como en cuenta bancaria)

sql

```
SELECT
    fecha,
    operacion,
    monto,
    SUM(CASE WHEN tipo = 'ingreso' THEN monto ELSE -monto END)
        OVER (ORDER BY fecha, id ROWS UNBOUNDED PRECEDING) AS saldo_acumulado
FROM movimientos
ORDER BY fecha;
```

Ejercicios prácticos 5.1

sql

```
-- 1. Para cada pedido, mostrar el total del pedido anterior del mismo cliente
SELECT
    id,
    cliente_id,
    fecha,
    total,
    LAG(total) OVER (PARTITION BY cliente_id ORDER BY fecha) AS pedido_anterior
FROM pedidos;

-- 2. Ranking de productos por precio dentro de cada categoría
SELECT
    nombre,
    categoria_id,
    precio,
    RANK() OVER (PARTITION BY categoria_id ORDER BY precio DESC) AS
ranking_precio
FROM productos;

-- 3. Promedio móvil de 3 días de ventas
SELECT
    fecha,
    total,
    AVG(total) OVER (ORDER BY fecha ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) AS
media_movil_3d
FROM ventas_diarias;

-- 4. Primer y último pedido de cada cliente
```

```
SELECT DISTINCT
    cliente_id,
    FIRST_VALUE(fecha) OVER (PARTITION BY cliente_id ORDER BY fecha) AS
primer_pedido,
    FIRST_VALUE(total) OVER (PARTITION BY cliente_id ORDER BY fecha) AS
monto_primer,
    LAST_VALUE(fecha) OVER (PARTITION BY cliente_id ORDER BY fecha
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS
ultimo_pedido,
    LAST_VALUE(total) OVER (PARTITION BY cliente_id ORDER BY fecha
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS
monto_ultimo
FROM pedidos;

-- 5. Porcentaje acumulado de ventas (Pareto 80/20)
SELECT
    producto_id,
    ventas,
    SUM(ventas) OVER (ORDER BY ventas DESC) AS ventas_acumuladas,
    SUM(ventas) OVER (ORDER BY ventas DESC) / SUM(ventas) OVER () * 100 AS
porcentaje_acumulado
FROM ventas_productos
ORDER BY ventas DESC;
```

Módulo 6: Consultas Avanzadas y Optimización

6.1 CTE (Common Table Expressions)

Las CTE permiten definir tablas temporales dentro de una consulta, mejorando legibilidad y permitiendo recursividad.

sql

```
-- CTE simple
WITH productos_caros AS (
    SELECT * FROM productos WHERE precio > 1000
)
SELECT
    p.nombre,
    p.precio,
    c.nombre AS categoria
FROM productos_caros p
JOIN categorias c ON p.categoria_id = c.id;

-- CTE múltiple
WITH
    ventas_por_cliente AS (
        SELECT cliente_id, SUM(total) AS total_gastado
        FROM pedidos
        GROUP BY cliente_id
    ),
    clientes_vip AS (
        SELECT cliente_id FROM ventas_por_cliente WHERE total_gastado > 10000
    )
SELECT c.nombre, v.total_gastado
FROM clientes c
JOIN ventas_por_cliente v ON c.id = v.cliente_id
WHERE c.id IN (SELECT cliente_id FROM clientes_vip);
```

```
-- CTE con UPDATE/DELETE (útil para modificaciones complejas)
WITH actualizados AS (
    UPDATE productos
    SET precio = precio * 1.10
    WHERE categoria_id = 1
    RETURNING id, nombre, precio
)
SELECT * FROM actualizados;
```

CTE Recursivas

Las CTE recursivas son ideales para datos jerárquicos (árboles, grafos, series).

sql

```
-- Sintaxis
WITH RECURSIVE nombre_cte AS (
    -- Anchor: consulta inicial (non-recursive)
    SELECT ...
    UNION ALL
    -- Recursive: se une a sí misma
    SELECT ... FROM nombre_cte WHERE condicion
)
SELECT * FROM nombre_cte;

-- Ejemplo 1: Jerarquía de empleados
WITH RECURSIVE organigrama AS (
    -- Anchor: CEO (sin jefe)
    SELECT
        id,
        nombre,
        jefe_id,
        1 AS nivel,
        nombre AS ruta
    FROM empleados
    WHERE jefe_id IS NULL

    UNION ALL

    -- Recursive: subordinados
    SELECT
        e.id,
        e.nombre,
        e.jefe_id,
        o.nivel + 1,
        o.ruta || ' > ' || e.nombre
    FROM empleados e
    JOIN organigrama o ON e.jefe_id = o.id
)
SELECT * FROM organigrama ORDER BY nivel, nombre;

-- Ejemplo 2: Generar serie numérica (alternativa a generate_series)
WITH RECURSIVE numeros AS (
    SELECT 1 AS n
    UNION ALL
    SELECT n + 1 FROM numeros WHERE n < 100
)
SELECT * FROM numeros;

-- Ejemplo 3: Árbol de categorías (con profundidad)
WITH RECURSIVE categorias_tree AS (
```

```
    SELECT
        id,
        nombre,
        padre_id,
        1 AS profundidad,
        nombre AS ruta_completa
    FROM categorias
    WHERE padre_id IS NULL

    UNION ALL

    SELECT
        c.id,
        c.nombre,
        c.padre_id,
        ct.profundidad + 1,
        ct.ruta_completa || ' / ' || c.nombre
    FROM categorias c
    JOIN categorias_tree ct ON c.padre_id = ct.id
)
SELECT * FROM categorias_tree ORDER BY ruta_completa;

-- Ejemplo 4: Grafo de conexiones (recorrido de red)
WITH RECURSIVE conexiones AS (
    SELECT
        nodo_origen,
        nodo_destino,
        ARRAY[nodo_origen] AS ruta,
        1 AS distancia
    FROM grafo
    WHERE nodo_origen = 'A'

    UNION ALL

    SELECT
        g.nodo_origen,
        g.nodo_destino,
        c.ruta || g.nodo_origen,
        c.distancia + 1
    FROM grafo g
    JOIN conexiones c ON g.nodo_origen = c.nodo_destino
    WHERE NOT g.nodo_destino = ANY(c.ruta)  -- Evitar ciclos
)
SELECT * FROM conexiones;
```

6.2 Consultas sobre JSON y JSONB

PostgreSQL ofrece soporte nativo para JSON con dos tipos: `json` (texto exacto) y `jsonb` (binario, indexable).

sql

```
-- Crear tabla con JSONB
CREATE TABLE productos_json (
    id serial PRIMARY KEY,
    datos jsonb
);

-- Insertar datos JSON
INSERT INTO productos_json (datos) VALUES
```

```
    ('{"nombre": "Laptop", "precio": 1200, "especificaciones": {"ram": "16GB",
"ssd": "512GB"}}'),
    ('{"nombre": "Mouse", "precio": 25, "especificaciones": {"dpi": 1600,
"inalambrico": true}}');

-- Operadores JSONB
-- -> : obtener como JSON
-- ->> : obtener como texto
-- @> : contiene
-- ? : existe clave
-- ?| : alguna clave
-- ?& : todas las claves

-- Consultas básicas
SELECT
    datos->>'nombre' AS nombre,
    datos->>'precio' AS precio,
    datos->'especificaciones'->>'ram' AS ram
FROM productos_json;

-- Filtrar por valor JSON
SELECT * FROM productos_json
WHERE datos->>'precio' > '100';

-- Filtrar con operador @> (contiene)
SELECT * FROM productos_json
WHERE datos @> '{"especificaciones": {"inalambrico": true}}';

-- Filtrar por existencia de clave
SELECT * FROM productos_json
WHERE datos->'especificaciones' ? 'ram';

-- Actualizar JSONB
UPDATE productos_json
SET datos = jsonb_set(datos, '{precio}', '1300')
WHERE datos->>'nombre' = 'Laptop';

-- Agregar campo
UPDATE productos_json
SET datos = datos || '{"descuento": 10}'
WHERE datos->>'nombre' = 'Mouse';

-- Eliminar campo
UPDATE productos_json
SET datos = datos - 'descuento';

-- Índices GIN para JSONB
CREATE INDEX idx_productos_json_datos ON productos_json USING gin (datos);
CREATE INDEX idx_productos_json_nombre ON productos_json ((datos->>'nombre'));

-- Funciones JSONB avanzadas
SELECT
    jsonb_object_keys(datos) AS claves,
    jsonb_each(datos) AS pares
FROM productos_json;

-- jsonb_path_query (PostgreSQL 12+)
SELECT * FROM productos_json
WHERE jsonb_path_exists(datos, '$.especificaciones.ram ? (@ == "16GB")');
```

6.3 Full Text Search

sql

```
-- Configuración del idioma
SELECT * FROM ts_parser;  -- Parsers disponibles
SELECT * FROM ts_config;   -- Configuraciones (english, spanish, etc.)

-- Funciones básicas
-- to_tsvector(): convierte texto a vector de búsqueda
-- to_tsquery(): convierte consulta a formato de búsqueda
-- @@ : operador de coincidencia

-- Ejemplo básico
SELECT
    titulo,
    contenido,
    to_tsvector('spanish', contenido) @@ to_tsquery('spanish', 'PostgreSQL &
base & datos') AS coincide
FROM articulos;

-- Crear columna con vector precomputado
ALTER TABLE articulos ADD COLUMN documento_tsv tsvector;
UPDATE articulos SET documento_tsv = to_tsvector('spanish', titulo || ' ' ||
contenido);

-- Índice GIN para búsqueda
CREATE INDEX idx_articulos_busqueda ON articulos USING gin (documento_tsv);

-- Búsqueda con ranking
SELECT
    titulo,
    ts_rank(documento_tsv, query) AS relevancia
FROM articulos, to_tsquery('spanish', 'PostgreSQL & base') query
WHERE documento_tsv @@ query
ORDER BY relevancia DESC;

-- Búsqueda con resaltado
SELECT
    titulo,
    ts_headline('spanish', contenido, to_tsquery('spanish', 'PostgreSQL'),
                'StartSel=<mark>, StopSel=</mark>') AS extracto
FROM articulos
WHERE documento_tsv @@ to_tsquery('spanish', 'PostgreSQL');

-- Búsqueda por proximidad (operador <->)
SELECT * FROM articulos
WHERE documento_tsv @@ to_tsquery('spanish', 'base <-> datos');  -- "base datos"
juntos

-- Búsqueda con sinónimos (configuración personalizada)
-- Crear diccionario de sinónimos
CREATE TEXT SEARCH DICTIONARY sinónimos (
    TEMPLATE = synonym,
    SYNONYMS = 'mis_sinonimos'
);

CREATE TEXT SEARCH CONFIGURATION spanish_syn (COPY = spanish);
ALTER TEXT SEARCH CONFIGURATION spanish_syn
    ALTER MAPPING FOR asciiword, word WITH sinónimos, spanish_stem;
```

6.4 Optimización de consultas con EXPLAIN

Entendiendo EXPLAIN

sql

```
-- EXPLAIN: muestra el plan sin ejecutar
EXPLAIN SELECT * FROM productos WHERE precio > 100;

-- EXPLAIN ANALYZE: ejecuta y muestra estadísticas reales
EXPLAIN ANALYZE SELECT * FROM productos WHERE precio > 100;

-- EXPLAIN con opciones adicionales
EXPLAIN (ANALYZE, BUFFERS, TIMING, VERBOSE)
SELECT * FROM productos WHERE precio > 100;

-- EXPLAIN para INSERT/UPDATE/DELETE
EXPLAIN ANALYZE DELETE FROM productos WHERE precio < 10;
```

Tipos de nodos (node types)

Nodo	Descripción	Cuándo ocurre
`Seq Scan`	Escanea toda la tabla	Sin índice o consulta no selectiva
`Index Scan`	Usa índice para encontrar filas	Búsqueda por índice eficiente
`Index Only Scan`	Solo índice, sin tocar tabla	Todas las columnas están en el índice
`Bitmap Heap Scan`	Combina múltiples índices	Varias condiciones con AND/OR
`Nested Loop`	Para cada fila de tabla A, busca en B	JOIN con tabla pequeña + índice
`Hash Join`	Construye hash table de tabla pequeña	JOIN sin índices adecuados
`Merge Join`	Ordena ambas tablas y las mezcla	Datos ya ordenados
`Sort`	Ordena resultados	ORDER BY sin índice
`Aggregate`	Agrupa resultados	GROUP BY
`Limit`	Limita número de filas	LIMIT
`Subquery Scan`	Ejecuta subconsulta	CTE o subconsulta

Interpretando un plan

sql

```
-- Ejemplo de plan con Seq Scan (malo)
EXPLAIN ANALYZE SELECT * FROM productos WHERE precio > 100;
-- Salida:
-- Seq Scan on productos  (cost=0.00..2040.00 rows=50000 width=100) (actual
time=0.015..12.345 rows=50000 loops=1)
--   Filter: (precio > 100)
--   Rows Removed by Filter: 50000
-- Planning Time: 0.123 ms
-- Execution Time: 15.678 ms
```

```
-- Interpretación:
-- cost=0.00..2040.00: costo estimado (inicio..total)
-- rows=50000: filas estimadas
-- actual time: tiempo real
-- rows=50000: filas reales
-- loops=1: veces que se ejecutó este nodo
```

Mejorando planes de ejecución

sql

```
-- 1. Crear índice faltante
CREATE INDEX idx_productos_precio ON productos(precio);

-- Ver nuevo plan
EXPLAIN ANALYZE SELECT * FROM productos WHERE precio > 100;
-- Ahora: Index Scan usando idx_productos_precio

-- 2. Forzar tipos de join (para debugging)
SET enable_nestloop = off;
SET enable_hashjoin = on;
SET enable_mergejoin = on;

-- 3. Aumentar estadísticas
ALTER TABLE productos ALTER COLUMN precio SET STATISTICS 1000;
ANALYZE productos;

-- 4. Usar pg_hint_plan para forzar planes (requiere extensión)
CREATE EXTENSION pg_hint_plan;
/*+ IndexScan(productos idx_productos_precio) */
SELECT * FROM productos WHERE precio > 100;
```

Consultas comunes y sus optimizaciones

sql

```
-- Problema: COUNT(*) lento en tablas grandes
-- Malo: Seq Scan completo
SELECT COUNT(*) FROM logs;

-- Bueno: usar estimación (si no requiere exactitud)
SELECT reltuples::bigint FROM pg_class WHERE relname = 'logs';

-- Problema: OR en condiciones
-- Malo: Seq Scan
SELECT * FROM productos WHERE precio > 100 OR stock < 10;

-- Bueno: usar UNION o índices separados
SELECT * FROM productos WHERE precio > 100
UNION
SELECT * FROM productos WHERE stock < 10;

-- Problema: LIKE con comodín al inicio
-- Malo: no usa índice B-Tree
SELECT * FROM productos WHERE nombre LIKE '%laptop%';

-- Bueno: usar pg_trgm (trigramas)
CREATE EXTENSION pg_trgm;
CREATE INDEX idx_productos_nombre_trgm ON productos USING gin (nombre
gin_trgm_ops);
SELECT * FROM productos WHERE nombre % 'laptop';  -- Similitud

-- Problema: ORDER BY + LIMIT sin índice
```

```
-- Malo: Sort + Limit
SELECT * FROM productos ORDER BY fecha LIMIT 10;

-- Bueno: índice en columna de ordenamiento
CREATE INDEX idx_productos_fecha ON productos(fecha);
```

Ejercicios prácticos 6.1

sql

```
-- 1. CTE Recursiva: Organigrama con niveles
WITH RECURSIVE jerarquia AS (
    SELECT id, nombre, jefe_id, 1 AS nivel, nombre AS ruta
    FROM empleados WHERE jefe_id IS NULL
    UNION ALL
    SELECT e.id, e.nombre, e.jefe_id, j.nivel + 1, j.ruta || ' > ' || e.nombre
    FROM empleados e JOIN jerarquia j ON e.jefe_id = j.id
)
SELECT * FROM jerarquia ORDER BY nivel, nombre;

-- 2. JSONB: Analizar logs
CREATE TABLE logs_api (id serial, datos jsonb);
INSERT INTO logs_api (datos) VALUES
    ('{"endpoint": "/api/users", "status": 200, "tiempo": 45}'),
    ('{"endpoint": "/api/products", "status": 500, "tiempo": 120}');

SELECT
    datos->>'endpoint' AS endpoint,
    COUNT(*) AS llamadas,
    AVG((datos->>'tiempo')::int) AS tiempo_promedio
FROM logs_api
GROUP BY endpoint;

-- 3. Full Text Search: Buscador de productos
CREATE TABLE catalogo (id serial, nombre text, descripcion text);
CREATE INDEX idx_catalogo_busqueda ON catalogo USING gin(to_tsvector('spanish',
nombre || ' ' || descripcion));

SELECT *,
       ts_rank(to_tsvector('spanish', nombre || ' ' || descripcion), query) AS
rank
FROM catalogo, to_tsquery('spanish', 'laptop & portátil') query
WHERE to_tsvector('spanish', nombre || ' ' || descripcion) @@ query
ORDER BY rank DESC;

-- 4. Optimización: Analizar consulta lenta
EXPLAIN (ANALYZE, BUFFERS, TIMING)
SELECT c.nombre, COUNT(p.id)
FROM clientes c
LEFT JOIN pedidos p ON c.id = p.cliente_id
WHERE p.fecha > '2024-01-01'
GROUP BY c.id
HAVING COUNT(p.id) > 5;
```

Proyecto Final Integrador

Escenario: Sistema de Ventas y Análisis

sql

```
-- 1. Diseño de esquema optimizado
CREATE SCHEMA proyecto_final;
SET search_path TO proyecto_final;

-- Tablas con índices apropiados
CREATE TABLE clientes (
    id serial PRIMARY KEY,
    nombre text NOT NULL,
    email text UNIQUE,
    ciudad text,
    fecha_registro date DEFAULT CURRENT_DATE,
    segmento text  -- 'VIP', 'Regular', 'Nuevo'
);
CREATE INDEX idx_clientes_ciudad ON clientes(ciudad);
CREATE INDEX idx_clientes_segmento ON clientes(segmento);

CREATE TABLE productos (
    id serial PRIMARY KEY,
    nombre text,
    precio numeric(10,2),
    categoria_id int,
    stock int,
    especificaciones jsonb
);
CREATE INDEX idx_productos_categoria ON productos(categoria_id);
CREATE INDEX idx_productos_precio ON productos(precio);
CREATE INDEX idx_productos_json ON productos USING gin(especificaciones);

CREATE TABLE pedidos (
    id serial PRIMARY KEY,
    cliente_id int REFERENCES clientes(id),
    fecha date DEFAULT CURRENT_DATE,
    total numeric(10,2),
    estado text
);
CREATE INDEX idx_pedidos_cliente_fecha ON pedidos(cliente_id, fecha);
CREATE INDEX idx_pedidos_fecha ON pedidos(fecha);
CREATE INDEX idx_pedidos_estado ON pedidos(estado) WHERE estado = 'activo';

CREATE TABLE detalle_pedidos (
    pedido_id int REFERENCES pedidos(id),
    producto_id int REFERENCES productos(id),
    cantidad int,
    precio_unitario numeric(10,2),
    PRIMARY KEY (pedido_id, producto_id)
);

-- 2. Consulta compleja con CTE, agregación y ventanas
WITH
    -- Ventas por cliente con categorización
    ventas_cliente AS (
        SELECT
            c.id AS cliente_id,
            c.nombre,
            c.segmento,
            COUNT(DISTINCT p.id) AS num_pedidos,
            SUM(p.total) AS total_gastado,
            AVG(p.total) AS ticket_medio,
            MAX(p.fecha) AS ultima_compra
        FROM clientes c
        LEFT JOIN pedidos p ON c.id = p.cliente_id
```

```
        GROUP BY c.id, c.nombre, c.segmento
    ),
    -- Ranking de clientes
    ranking_clientes AS (
        SELECT
            *,
            ROW_NUMBER() OVER (ORDER BY total_gastado DESC) AS ranking_global,
            RANK() OVER (PARTITION BY segmento ORDER BY total_gastado DESC) AS
ranking_segmento,
            NTILE(5) OVER (ORDER BY total_gastado DESC) AS quintil
        FROM ventas_cliente
    ),
    -- Productos más vendidos por categoría
    top_productos AS (
        SELECT
            prod.categoria_id,
            prod.nombre AS producto,
            SUM(dp.cantidad) AS unidades,
            ROW_NUMBER() OVER (PARTITION BY prod.categoria_id ORDER BY
SUM(dp.cantidad) DESC) AS posicion
        FROM productos prod
        JOIN detalle_pedidos dp ON prod.id = dp.producto_id
        GROUP BY prod.categoria_id, prod.nombre
    )
SELECT
    rc.nombre AS cliente,
    rc.segmento,
    rc.total_gastado,
    rc.ranking_global,
    rc.ranking_segmento,
    rc.quintil,
    rc.ultima_compra,
    tp.producto AS producto_top_categoria,
    tp.unidades
FROM ranking_clientes rc
LEFT JOIN top_productos tp ON tp.posicion = 1
ORDER BY rc.total_gastado DESC
LIMIT 20;

-- 3. Función de mantenimiento automático
CREATE OR REPLACE FUNCTION mantenimiento_preventivo()
RETURNS text AS $$
DECLARE
    registro record;
    resultado text := '';
BEGIN
    -- Reindexar tablas con alto bloat
    FOR registro IN
        SELECT relname, pg_relation_size(relid) AS tamaño
        FROM pg_stat_user_tables
        WHERE n_dead_tup > n_live_tup * 0.3
        ORDER BY tamaño DESC
        LIMIT 5
    LOOP
        EXECUTE format('REINDEX TABLE CONCURRENTLY %I', registro.relname);
        resultado := resultado || format('Reindexada %s; ', registro.relname);
    END LOOP;

    -- Vacuum full de tablas pequeñas muy fragmentadas
    FOR registro IN
        SELECT relname
```

```
        FROM pg_stat_user_tables
        WHERE n_dead_tup > 10000 AND n_live_tup < 100000
        LIMIT 3
    LOOP
        EXECUTE format('VACUUM FULL VERBOSE %I', registro.relname);
        resultado := resultado || format('Vacuum full %s; ', registro.relname);
    END LOOP;

    -- Actualizar estadísticas
    ANALYZE;
    resultado := resultado || 'Estadísticas actualizadas.';

    RETURN resultado;
END;
$$ LANGUAGE plpgsql;

-- 4. Vista materializada para reportes
CREATE MATERIALIZED VIEW reporte_ventas_mensual AS
SELECT
    DATE_TRUNC('month', fecha) AS mes,
    cliente_id,
    COUNT(*) AS pedidos,
    SUM(total) AS ventas
FROM pedidos
GROUP BY mes, cliente_id;

CREATE INDEX idx_reporte_mes ON reporte_ventas_mensual(mes);
CREATE INDEX idx_reporte_cliente ON reporte_ventas_mensual(cliente_id);

-- Actualizar vista materializada periódicamente
REFRESH MATERIALIZED VIEW CONCURRENTLY reporte_ventas_mensual;
```

Resumen del Curso

Módulo	Habilidades adquiridas
Módulo 1	Arquitectura interna, procesos, memoria, índices, MVCC
Módulo 2	Mantenimiento, VACUUM, REINDEX, backup, monitoreo
Módulo 3	Consultas SQL completas, JOINs, subconsultas
Módulo 4	Agregaciones, GROUP BY, ROLLUP, CUBE, FILTER
Módulo 5	Window functions, ranking, frames, análisis avanzado
Módulo 6	CTE recursivas, JSONB, Full Text Search, optimización

¿Quieres que profundice en algún tema específico o que desarrolle más ejercicios prácticos para algún módulo en particular?

que significa esto:SET search_path TO proyecto_final;

`SET search_path TO proyecto_final;` - Explicación completa

Definición

`SET search_path TO proyecto_final;` es un comando que **cambia el orden de búsqueda de esquemas** en PostgreSQL para la sesión actual, estableciendo `proyecto_final` como el **primer esquema** donde PostgreSQL buscará las tablas, vistas y funciones.

El problema que resuelve

Sin `search_path`, PostgreSQL busca objetos (tablas, vistas, funciones) en este orden predeterminado:

sql

```
-- Orden predeterminado (por defecto)
SHOW search_path;
-- Resultado: "$user", public
```

Significado:

1. Primero busca en un esquema con el nombre del usuario actual (ej: `ana`)
2. Si no existe o no encuentra el objeto, busca en `public`
3. Si no encuentra, lanza error `relation "tabla" does not exist`

Ejemplo práctico de la diferencia

Sin SET search_path:

sql

```
-- Tenemos esquemas: public, proyecto_final, ventas

-- Tablas existentes:
-- public.clientes
-- proyecto_final.pedidos

SELECT * FROM clientes;     --    Encuentra public.clientes
SELECT * FROM pedidos;       --    ERROR: no encuentra (busca en public, no
existe)

-- Necesitas calificar explícitamente:
SELECT * FROM proyecto_final.pedidos;  --    Funciona
```

Con SET search_path:

sql

```
SET search_path TO proyecto_final;
```

```
SELECT * FROM clientes;    --    ERROR (busca en proyecto_final, no existe)
SELECT * FROM pedidos;      --    Encuentra proyecto_final.pedidos

-- Puedes mezclar esquemas:
SELECT * FROM pedidos p
JOIN public.clientes c ON p.cliente_id = c.id;  --    Especificas
public.clientes
```

Sintaxis completa y variantes

sql

```
-- 1. Establecer un único esquema
SET search_path TO proyecto_final;

-- 2. Múltiples esquemas (orden de búsqueda)
SET search_path TO proyecto_final, public, ventas;

-- 3. Agregar un esquema al principio (manteniendo el resto)
SET search_path = proyecto_final, "$user", public;

-- 4. Configurar solo para la transacción actual
BEGIN;
SET LOCAL search_path TO proyecto_final;
-- Dentro de la transacción usa este search_path
COMMIT;  -- Vuelve al anterior

-- 5. Para un usuario específico (permanente)
ALTER ROLE mi_usuario SET search_path TO proyecto_final, public;

-- 6. Para una base de datos específica
ALTER DATABASE mi_db SET search_path TO proyecto_final, public;
```

Comportamiento detallado

sql

```
-- 1. Ver el search_path actual
SHOW search_path;
-- Resultado: "proyecto_final"

-- 2. Ver todos los esquemas disponibles
SELECT nspname FROM pg_namespace ORDER BY nspname;

-- 3. Simular cómo PostgreSQL resuelve un nombre
-- Cuando escribes: SELECT * FROM pedidos
-- PostgreSQL busca en orden:
-- 1. ¿Existe proyecto_final.pedidos? → Sí
-- (Si no existiera, seguiría con el siguiente esquema en la lista)

-- 4. Forzar búsqueda en todos los esquemas del search_path
SET search_path = proyecto_final, public, information_schema;
-- Ahora busca en proyecto_final, luego public, luego information_schema
```

Casos de uso comunes

1. Trabajar con múltiples esquemas en un proyecto

sql

```
-- Esquema para datos transaccionales
CREATE SCHEMA transaccional;
CREATE TABLE transaccional.ventas (...);

-- Esquema para reporting
CREATE SCHEMA reporting;
CREATE TABLE reporting.resumen_ventas (...);

-- Cambiar según la tarea
SET search_path TO transaccional;
SELECT * FROM ventas;  -- Busca en transaccional.ventas

SET search_path TO reporting;
SELECT * FROM resumen_ventas;  -- Busca en reporting.resumen_ventas
```

2. Evitar escribir el esquema constantemente

sql

```
-- Sin search_path (mucho más verboso)
INSERT INTO proyecto_final.clientes (nombre) VALUES ('Ana');
INSERT INTO proyecto_final.clientes (nombre) VALUES ('Juan');
INSERT INTO proyecto_final.pedidos (cliente_id, total) VALUES (1, 100);
SELECT * FROM proyecto_final.clientes c
JOIN proyecto_final.pedidos p ON c.id = p.cliente_id;

-- Con search_path (más limpio)
SET search_path TO proyecto_final;
INSERT INTO clientes (nombre) VALUES ('Ana');
INSERT INTO clientes (nombre) VALUES ('Juan');
INSERT INTO pedidos (cliente_id, total) VALUES (1, 100);
SELECT * FROM clientes c JOIN pedidos p ON c.id = p.cliente_id;
```

3. Aislar pruebas unitarias

sql

```
-- Crear esquema de pruebas
CREATE SCHEMA pruebas;

-- Configurar search_path para pruebas
SET search_path TO pruebas;

-- Crear tablas de prueba (no afectan producción)
CREATE TABLE clientes (...);
CREATE TABLE pedidos (...);

-- Ejecutar pruebas
INSERT INTO clientes VALUES (...);

-- Cambiar de vuelta a producción
SET search_path TO public;
```

Precauciones y buenas prácticas

Problema común: Tablas duplicadas

sql

```
-- Mismo nombre de tabla en dos esquemas
CREATE SCHEMA esquema_a;
CREATE TABLE esquema_a.usuarios (id int, nombre text);

CREATE SCHEMA esquema_b;
CREATE TABLE esquema_b.usuarios (id int, email text);

-- Search_path determina cuál se usa
SET search_path TO esquema_a;
SELECT * FROM usuarios;  --    Usa esquema_a.usuarios

SET search_path TO esquema_b;
SELECT * FROM usuarios;  --    Usa esquema_b.usuarios (diferente estructura!)
```

Práctica recomendada: Esquema por aplicación

sql

```
-- Cada aplicación tiene su propio esquema
CREATE SCHEMA app_ventas;
CREATE SCHEMA app_inventario;
CREATE SCHEMA app_reporting;

-- En la aplicación de ventas:
SET search_path TO app_ventas;

-- En la aplicación de inventario:
SET search_path TO app_inventario;
```

Práctica recomendada: Configurar al conectar

sql

```
-- En tu archivo de conexión o al inicio de sesión
-- pgAdmin: Properties > Advanced > search_path
-- psql: \set search_path 'proyecto_final'
-- En código Python (psycopg2):
-- conn.cursor().execute("SET search_path TO proyecto_final")
```

Comandos relacionados

sql

```
-- 1. Ver esquema actual de una tabla
SELECT schemaname, tablename
FROM pg_tables
WHERE tablename = 'pedidos';

-- 2. Crear tabla sin especificar esquema (usa search_path)
SET search_path TO proyecto_final;
CREATE TABLE nueva_tabla (...);  -- Se crea en proyecto_final

-- 3. Resetear a valor predeterminado
SET search_path TO "$user", public;
-- o
```

```
RESET search_path;

-- 4. Ver el orden de búsqueda completo
SELECT current_setting('search_path');

-- 5. Agregar esquema temporalmente
SET search_path = proyecto_final, public;
-- Las tablas nuevas se crean en el PRIMER esquema (proyecto_final)
```

Resumen ejecutivo

Concepto	Explicación
Qué hace	Define el orden de búsqueda de esquemas para la sesión actual
Por defecto	`"$user", public` (primero esquema del usuario, luego public)
Beneficio	Evita escribir `esquema.tabla` constantemente
Riesgo	Si hay tablas con mismo nombre en diferentes esquemas, se usa el primero
Ámbito	Solo la sesión actual (a menos que se use ALTER ROLE/DATABASE)
Mejor práctica	Configurar al inicio de la aplicación para usar su esquema

En tu proyecto final: `SET search_path TO proyecto_final;` significa que todas las consultas que escribas buscarán tablas primero en el esquema `proyecto_final`, permitiéndote escribir `SELECT * FROM clientes` en lugar de `SELECT * FROM proyecto_final.clientes`.

que significa $PGDATA

`$PGDATA` - Explicación completa

Definición

`$PGDATA` es una **variable de entorno** que indica la **ubicación del directorio de datos principal** de PostgreSQL (Data Directory). Es la ruta raíz donde PostgreSQL almacena todos sus archivos: bases de datos, configuración, logs WAL, catálogos del sistema, etc.

Significado literal

- **PGDATA** = **P**ost**G**res **DATA** directory
- El símbolo `$` indica que es una variable de entorno (en Linux/macOS/Unix)
- En Windows sería `%PGDATA%`

¿Qué contiene $PGDATA?

bash

```
$PGDATA/
├── PG_VERSION                    # Versión de PostgreSQL (ej: "16\n")
├── postgresql.conf               # Archivo de configuración principal
├── pg_hba.conf                   # Control de acceso (clientes autorizados)
├── pg_ident.conf                 # Mapeo de usuarios del SO a roles
├── postgresql.auto.conf          # Configuración generada por ALTER SYSTEM
├── postmaster.pid                # PID del proceso postmaster (activo)
├── pg_control                    # Archivo de control del cluster
│
├── base/                         # Datos de las bases de usuario (pg_default)
│   ├── 16384/                    # OID de la base de datos "mi_bd"
│   │   ├── 16385                 # Tabla (archivo físico)
│   │   ├── 16385_fsm             # Free Space Map
│   │   ├── 16385_vm              # Visibility Map
│   │   └── 16390                 # Índice
│   └── 16386/                    # Otra base de datos
│
├── global/                       # Catálogos del sistema (pg_global)
│   ├── pg_authid                 # Usuarios y roles
│   ├── pg_database               # Lista de bases de datos
│   └── pg_class                  # Metadatos de tablas/índices
│
├── pg_wal/                       # Write-Ahead Logs (WAL)
│   ├── 000000010000000000000001  # Archivo WAL segment (16 MB)
│   ├── 000000010000000000000002
│   └── archive_status/           # Estado de archivo de WALs
│
├── pg_xact/                      # Estado de transacciones (committed/aborted)
├── pg_subtrans/                  # Subtransacciones
├── pg_twophase/                  # Transacciones preparadas (2PC)
├── pg_multixact/                 # Multi-transacciones (bloqueos)
├── pg_logical/                   # Datos de replicación lógica
│
├── pg_stat/                      # Estadísticas persistentes
├── pg_stat_tmp/                  # Estadísticas temporales (en memoria)
├── pg_replslot/                  # Slots de replicación
├── pg_snapshots/                 # Snapshots exportados
│
├── log/                          # Logs del servidor (si logging_collector=on)
│   └── postgresql-2026-04-09.log
│
└── tablespaces/                  # Enlaces simbólicos a tablespaces externos
    └── pg_tblspc/16394 -> /mnt/ssd/postgres_data
```

Cómo ver y usar $PGDATA

En el sistema operativo

bash

```
# Ver el valor actual de la variable
echo $PGDATA
# Salida típica: /var/lib/postgresql/16/main

# Ver directorio (como usuario postgres)
sudo -u postgres ls -la $PGDATA

# Ver tamaño total del cluster
sudo -u postgres du -sh $PGDATA
```

Desde PostgreSQL (SQL)

sql

```
-- Ver la ubicación de $PGDATA desde SQL
SHOW data_directory;
-- Resultado: /var/lib/postgresql/16/main

-- Ver todas las configuraciones relacionadas
SELECT name, setting
FROM pg_settings
WHERE name LIKE '%directory%' OR name LIKE '%path%';
```

Ubicaciones típicas por sistema operativo

Sistema	Ubicación típica de $PGDATA
Linux (apt/yum)	`/var/lib/postgresql/16/main`
macOS (Homebrew)	`/opt/homebrew/var/postgresql@16`
Windows	`C:\Program Files\PostgreSQL\16\data`
Docker	`/var/lib/postgresql/data`
Instalación desde fuente	`/usr/local/pgsql/data`

Cómo se establece $PGDATA

Método 1: Variable de entorno (Linux/macOS)

bash

```
# Establecer temporalmente (solo esta sesión)
export PGDATA=/custom/path/postgres_data

# Establecer permanentemente (agregar a ~/.bashrc)
echo "export PGDATA=/custom/path/postgres_data" >> ~/.bashrc

# Verificar
echo $PGDATA
```

Método 2: Al iniciar PostgreSQL

bash

```
# Iniciar con PGDATA explícito
pg_ctl start -D /custom/path/postgres_data

# O usando la variable
export PGDATA=/custom/path/postgres_data
pg_ctl start
```

Método 3: En servicio systemd (Linux)

bash

```
# Ver configuración del servicio
```

```
sudo systemctl cat postgresql

# Típicamente contiene:
# Environment=PGDATA=/var/lib/postgresql/16/main
```

Método 4: En Docker

yaml

```
# docker-compose.yml
services:
  postgres:
    image: postgres:16
    environment:
      - PGDATA=/var/lib/postgresql/data/custom  # Opcional
    volumes:
      - ./postgres_data:/var/lib/postgresql/data  # Mapeo
```

Comandos comunes que usan $PGDATA

bash

```
# 1. Iniciar PostgreSQL
pg_ctl start -D $PGDATA
# o
pg_ctl start

# 2. Detener PostgreSQL
pg_ctl stop -D $PGDATA
pg_ctl stop -D $PGDATA -m fast    # Detención rápida
pg_ctl stop -D $PGDATA -m immediate  # Detención inmediata (peligroso)

# 3. Recargar configuración (sin reiniciar)
pg_ctl reload -D $PGDATA
# o desde SQL: SELECT pg_reload_conf();

# 4. Ver estado
pg_ctl status -D $PGDATA

# 5. Inicializar nuevo cluster
initdb -D $PGDATA

# 6. Hacer backup físico
pg_basebackup -D /backup/pgdata -Ft -z

# 7. Verificar integridad
pg_verifybackup $PGDATA

# 8. Logs de inicio (si hay errores)
cat $PGDATA/log/postgresql-*.log
```

Ejemplo práctico: Cambiar ubicación de $PGDATA

bash

```
# 1. Detener PostgreSQL
sudo systemctl stop postgresql

# 2. Crear nueva ubicación
```

```
sudo mkdir -p /mnt/ssd/postgres_data
sudo chown postgres:postgres /mnt/ssd/postgres_data

# 3. Copiar datos existentes
sudo -u postgres rsync -av /var/lib/postgresql/16/main/ /mnt/ssd/postgres_data/

# 4. Cambiar variable (en systemd)
sudo systemctl edit postgresql
# Agregar:
# [Service]
# Environment=PGDATA=/mnt/ssd/postgres_data

# 5. Iniciar PostgreSQL
sudo systemctl start postgresql

# 6. Verificar
sudo -u postgres psql -c "SHOW data_directory;"
```

Seguridad: Permisos de $PGDATA

bash

```
# Los permisos correctos son cruciales
# El directorio debe pertenecer al usuario 'postgres'
ls -ld $PGDATA
# drwx------ 5 postgres postgres 4096 Apr 9 10:00 /var/lib/postgresql/16/main

# Permisos: 0700 (solo postgres puede leer/escribir)

# Corregir permisos si están mal
sudo chown -R postgres:postgres $PGDATA
sudo chmod 700 $PGDATA
```

Problemas comunes relacionados con $PGDATA

Problema 1: Variable no definida

bash

```
# Error al ejecutar pg_ctl
pg_ctl start
# pg_ctl: no database directory specified and environment variable PGDATA unset

# Soluciones:
# Opción 1: Usar -D
pg_ctl start -D /var/lib/postgresql/16/main

# Opción 2: Definir variable
export PGDATA=/var/lib/postgresql/16/main
pg_ctl start
```

Problema 2: Espacio en disco lleno

bash

```
# Verificar espacio
df -h $PGDATA

# Si está lleno (100%):
```

```
# - Mover WALs viejos
# - Ejecutar VACUUM FULL en tablas grandes
# - Mover tablespaces a otro disco

# Identificar qué ocupa más espacio
sudo -u postgres du -sh $PGDATA/* | sort -h
```

Problema 3: Permisos incorrectos

bash

```
# Error típico:
# FATAL: could not open directory "pg_wal": Permission denied

# Solución:
sudo chown -R postgres:postgres $PGDATA
sudo chmod 700 $PGDATA
```

Problema 4: PostgreSQL no encuentra $PGDATA

bash

```
# Verificar configuración del servicio
sudo systemctl cat postgresql | grep Environment

# Verificar variable en entorno del proceso
sudo cat /proc/$(pidof postgres)/environ | tr '\0' '\n' | grep PGDATA
```

Relación con otros parámetros importantes

sql

```
-- Todos estos dependen de $PGDATA
SHOW data_directory;        -- = $PGDATA
SHOW config_file;           -- = $PGDATA/postgresql.conf
SHOW hba_file;              -- = $PGDATA/pg_hba.conf
SHOW ident_file;            -- = $PGDATA/pg_ident.conf

-- Subdirectorios dentro de $PGDATA
SHOW wal_directory;         -- = $PGDATA/pg_wal
SHOW stats_temp_directory;  -- = $PGDATA/pg_stat_tmp
```

En el contexto de tu curso

Cuando en los ejemplos ves:

sql

```
-- En ejercicios de monitoreo
SELECT * FROM pg_ls_dir('pg_wal');  -- Lista archivos en $PGDATA/pg_wal

-- En comandos del sistema
cd $PGDATA
ls -la base/
```

Significa que estás trabajando en el **directorio raíz de datos** del cluster PostgreSQL. Todos los archivos que mencionamos en el Módulo 1 (arquitectura) y Módulo 2 (mantenimiento) están dentro de `$PGDATA`.

Resumen ejecutivo

Aspecto	Explicación
Qué es	Variable de entorno con la ruta al directorio de datos
Contenido	Configuración, bases de datos, WALs, logs, catálogos
Quién lo usa	PostgreSQL server, pg_ctl, utilidades de backup
Valor típico Linux	`/var/lib/postgresql/16/main`
Cómo verlo	`echo $PGDATA` (SO) o `SHOW data_directory;` (SQL)
Permisos	`700` (solo usuario postgres)
Sin variable	Usar `-D /ruta` en comandos
Seguridad	CRÍTICO - No exponer, backup regular

En términos simples: `$PGDATA` es como la "carpeta de trabajo" de PostgreSQL. Si PostgreSQL fuera una casa, `$PGDATA` sería el terreno completo con la casa, los muebles (datos), los planos (configuración) y el diario de eventos (WAL).

explicame toda esta linea:echo "export PGDATA=/custom/path/postgres_data" >> ~/.bashrc

Explicación completa de la línea:

bash

```
echo "export PGDATA=/custom/path/postgres_data" >> ~/.bashrc
```

Esta línea **agrega permanentemente** la variable de entorno `PGDATA` a tu configuración de shell Bash. Vamos a desglosarla componente por componente:

Desglose detallado

1. `echo`

Qué hace: Imprime texto en la salida estándar (generalmente la terminal)

bash

```
echo "Hola mundo"
# Resultado: Hola mundo

echo "export PGDATA=/custom/path/postgres_data"
# Resultado: export PGDATA=/custom/path/postgres_data
```

En nuestro caso: `echo` no muestra nada en pantalla porque redirigimos la salida (ver punto 3)

---

2. `"export PGDATA=/custom/path/postgres_data"`

Qué hace: Es el texto que se va a agregar al archivo

Desglose del texto:

- `export` - Comando de Bash que hace que la variable esté disponible para procesos hijos
- `PGDATA` - Nombre de la variable
- `=` - Asignación
- `/custom/path/postgres_data` - Valor (ruta personalizada al directorio de datos)

Sin `export` vs Con `export`:

bash

```
# Sin export (variable LOCAL - no disponible para programas hijos)
PGDATA=/custom/path/postgres_data
psql --command "SHOW data_directory;"  #    No ve la variable

# Con export (variable de ENTORNO - disponible para todo)
export PGDATA=/custom/path/postgres_data
psql --command "SHOW data_directory;"  #    PostgreSQL puede verla
```

---

3. >>

Qué hace: Redirección de salida que **agrega** (append) contenido al final de un archivo

Comparación de operadores de redirección:

Operador	Función	Ejemplo	Resultado
`>`	Sobrescribe (borra contenido previo)	`echo "nuevo" > archivo.txt`	archivo.txt contiene solo "nuevo"
`>>`	Agrega al final (preserva contenido previo)	`echo "adicional" >> archivo.txt`	archivo.txt tenía "viejo\nadicional"
`2>`	Redirige errores (stderr)	`comando 2> errores.txt`	Guarda solo errores
`&>`	Redirige stdout y stderr	`comando &> todo.txt`	Guarda salida normal y errores

Ejemplo práctico:

bash

```
# Archivo original
cat ~/.bashrc
# alias ll='ls -la'
# export PATH=$PATH:/usr/local/bin

# Agregar nueva línea
echo "export PGDATA=/custom/path/postgres_data" >> ~/.bashrc

# Archivo después
cat ~/.bashrc
# alias ll='ls -la'
# export PATH=$PATH:/usr/local/bin
```

```
# export PGDATA=/custom/path/postgres_data    ← NUEVA LÍNEA AGREGADA AL FINAL
```

---

4. `~/.bashrc`

Qué es: Archivo de configuración de Bash que se ejecuta **cada vez que abres una nueva terminal interactiva**

Ubicación:

- ~ = Tu directorio home (ej: `/home/tu_usuario/`)
- `.bashrc` = Archivo oculto (el punto significa archivo oculto en Linux/Unix)

Archivos de configuración de Bash:

Archivo	Cuándo se ejecuta	Uso típico
`~/.bashrc`	Cada terminal interactiva (no login)	Aliases, funciones, variables de entorno para shells
`~/.bash_profile`	Login shell (al iniciar sesión)	Variables de entorno globales
`~/.profile`	Si `.bash_profile` no existe	Similar a `.bash_profile`
`/etc/bash.bashrc`	Global para todos los usuarios	Configuraciones del sistema
`~/.bash_logout`	Al cerrar sesión	Limpieza, mensajes de despedida

Cuándo se carga `.bashrc`:

- Al abrir una nueva terminal (Ctrl+Alt+T)
- Al ejecutar `bash` dentro de una terminal existente
- Al usar `su - usuario` (cambiar de usuario)
- **NO** se carga automáticamente en sesiones SSH (usa `.bash_profile`)

La línea completa explicada paso a paso

bash

```
echo "export PGDATA=/custom/path/postgres_data" >> ~/.bashrc
```

Paso 1: `echo` genera el texto `export PGDATA=/custom/path/postgres_data`

Paso 2: `>>` redirige ese texto para que no se muestre en pantalla

Paso 3: `>>` abre el archivo `~/.bashrc` y se posiciona al final

Paso 4: `>>` escribe el texto al final del archivo

Paso 5: Se cierra el archivo

Resultado final: Cada vez que abras una terminal nueva, Bash ejecutará automáticamente `export PGDATA=/custom/path/postgres_data`, haciendo que la variable `$PGDATA` esté disponible para todos tus comandos.

Ejemplo completo y verificación

bash

```
# 1. Verificar que NO existe la variable actualmente
echo $PGDATA
# (salida vacía)

# 2. Agregar al .bashrc
echo "export PGDATA=/custom/path/postgres_data" >> ~/.bashrc

# 3. Verificar que se agregó correctamente
tail -n 5 ~/.bashrc
# ... (últimas líneas)
# export PGDATA=/custom/path/postgres_data

# 4. Recargar .bashrc en la terminal actual
source ~/.bashrc
# o
. ~/.bashrc

# 5. Verificar que la variable ahora existe
echo $PGDATA
# Resultado: /custom/path/postgres_data

# 6. Usar la variable
cd $PGDATA
pwd
# Resultado: /custom/path/postgres_data
```

Problemas comunes y soluciones

Problema 1: La variable no aparece en nuevas terminales

Causa: Estás usando una shell diferente (zsh, fish) o `.bashrc` no se está cargando

Solución:

bash

```
# Verificar shell actual
echo $SHELL
# /bin/bash  ← OK
# /bin/zsh   ← Necesitas agregar a ~/.zshrc

# Si usas zsh:
echo "export PGDATA=/custom/path/postgres_data" >> ~/.zshrc
```

Problema 2: Agregaste la línea muchas veces por error

bash

```
# Verificar líneas duplicadas
grep "PGDATA" ~/.bashrc

# Si hay duplicados, eliminar todos y agregar uno
sed -i '/PGDATA/d' ~/.bashrc
echo "export PGDATA=/custom/path/postgres_data" >> ~/.bashrc
```

Problema 3: El path tiene espacios

bash

```
# Si el path contiene espacios, necesitas comillas
echo "export PGDATA='/ruta/con espacios/postgres_data'" >> ~/.bashrc

# O usar comillas dobles escapadas
echo "export PGDATA=\"/ruta/con espacios/postgres_data\"" >> ~/.bashrc
```

Problema 4: Permisos de escritura en ~/.bashrc

bash

```
# Verificar permisos
ls -la ~/.bashrc
# -rw-r--r-- 1 usuario usuario 1234 Apr 9 10:00 .bashrc

# Si no tienes permisos (muy raro), corregir
chmod u+w ~/.bashrc
```

Alternativas a esta línea

Alternativa 1: Agregar manualmente con editor de texto

bash

```
nano ~/.bashrc
# Agregar al final: export PGDATA=/custom/path/postgres_data
# Ctrl+O (guardar), Ctrl+X (salir)
```

Alternativa 2: Usar `tee` para agregar

bash

```
echo "export PGDATA=/custom/path/postgres_data" | tee -a ~/.bashrc
```

Alternativa 3: Agregar a `.profile` (para sesiones SSH)

bash

```
echo "export PGDATA=/custom/path/postgres_data" >> ~/.profile
```

Alternativa 4: Configuración específica de PostgreSQL

bash

```
# En lugar de variable de entorno, configurar en postgresql.conf
echo "data_directory = '/custom/path/postgres_data'" >> $PGDATA/postgresql.conf
```

Mejores prácticas

Hacer esto:

bash

```
# 1. Verificar que el directorio existe antes de definir la variable
mkdir -p /custom/path/postgres_data

# 2. Agregar con comentario para recordar qué hace
echo "# PostgreSQL data directory" >> ~/.bashrc
echo "export PGDATA=/custom/path/postgres_data" >> ~/.bashrc

# 3. Validar que se agregó correctamente
tail -n 3 ~/.bashrc

# 4. Recargar y verificar
source ~/.bashrc && echo $PGDATA
```

Evitar esto:

bash

```
# 1. No usar comillas si el path tiene espacios
echo export PGDATA=/ruta con espacios/data >> ~/.bashrc  # MAL

# 2. No verificar si ya existe (duplicados)
echo "export PGDATA=/custom/path/postgres_data" >> ~/.bashrc  # sin verificar

# 3. Usar > en lugar de >> (borra todo el archivo)
echo "export PGDATA=/custom/path/postgres_data" > ~/.bashrc  # ¡PELIGRO! Borra
todo
```

Analogía para entenderlo mejor

Imagina que `~/.bashrc` es una **lista de tareas automáticas** que tu computadora ejecuta cada vez que abres una terminal.

Esta línea está **agregando una nueva instrucción** a esa lista:

"Cada vez que abras una terminal, recuerda que la carpeta de datos de PostgreSQL está en `/custom/path/postgres_data` y haz que todos los programas lo sepan"

Sin esta línea, tendrías que **recordar y escribir manualmente** cada vez:

bash

```
export PGDATA=/custom/path/postgres_data
pg_ctl start
```

Con esta línea, **es automático**:

bash

```
pg_ctl start  # Ya sabe dónde está $PGDATA
```

Resumen ejecutivo

Componente	Significado
`echo`	Imprime texto
`"export PGDATA=..."`	El comando que se agregará
`>>`	Agrega al final del archivo (no sobrescribe)
`~/.bashrc`	Archivo de configuración de Bash
Resultado	La variable `$PGDATA` se define automáticamente en cada nueva terminal

En una frase: Esta línea **configura permanentemente** la ubicación del directorio de datos de PostgreSQL para que no tengas que escribirla manualmente cada vez que usas comandos como `pg_ctl` o `psql`.

como saber si tengo en ubuntu pgdata y cual es el path

Leer 10 páginas web

En Ubuntu, la variable de entorno `PGDATA` **no suele estar definida por defecto**. En lugar de depender de una variable global, PostgreSQL en Ubuntu utiliza una estructura de directorios estandarizada y comandos específicos para la gestión de servicios.

Aquí te explico cómo encontrar tu directorio de datos y las diferencias clave en Ubuntu.

La forma más segura de encontrar tu directorio de datos

La manera más confiable de saber dónde está tu `PGDATA` es preguntarle directamente al motor de la base de datos. Para ello, conecta a PostgreSQL y ejecuta el siguiente comando SQL:

sql

```
SHOW data_directory;
```

Este comando te devolverá la ruta completa del directorio que tu servidor está utilizando activamente. En una instalación estándar de Ubuntu, el resultado suele ser similar a este:

text

```
/var/lib/postgresql/16/main
```

- **`16`** es la versión principal de PostgreSQL.
- **`main`** es el nombre del "cluster" o instancia de base de datos.

Este directorio contiene las subcarpetas esenciales que ya hemos visto, como `base/` (donde están tus datos), `global/` (catálogos del sistema) y `pg_wal/` (registro de transacciones) .

La particularidad de Ubuntu: ¿Por qué `PGDATA` no está definida?

Si intentas ver la variable con `echo $PGDATA` y la terminal no muestra nada, no te preocupes, es completamente normal. Aunque el concepto `PGDATA` es fundamental en el mundo de PostgreSQL, **Ubuntu y Debian utilizan una estructura de directorios diferente a la tradicional.**

En lugar de almacenar **datos** y **configuración** en el mismo lugar, Ubuntu los separa para mayor organización y seguridad, siguiendo el estándar de sistema de archivos (FHS) :

- **Datos (Data Directory):** Donde vive tu `PGDATA`.
- **Ruta:** `/var/lib/postgresql/<version>/<cluster-name>/`
- **Ejemplo:** `/var/lib/postgresql/16/main/`
- **Propietario:** El usuario `postgres` .
- **Configuración (Configuration Directory):** Donde están los archivos `.conf`.
- **Ruta:** `/etc/postgresql/<version>/<cluster-name>/`
- **Ejemplo:** `/etc/postgresql/16/main/`
- **Archivos:** Aquí encontrarás `postgresql.conf`, `pg_hba.conf`, etc. .

Esta separación es la razón principal por la que la variable `PGDATA` no está configurada en el sistema. Las utilidades de Ubuntu (`pg_ctlcluster`, `pg_lsclusters`) ya saben exactamente dónde buscar todo gracias a esta estructura organizada .

Herramientas útiles en Ubuntu

Para gestionar y visualizar tus clusters en Ubuntu, tienes dos herramientas muy prácticas:

- **`pg_lsclusters`**: Este comando es una joya de Ubuntu. Te muestra una lista completa de todos tus clusters, su versión, puerto, estado (online/down), y lo más importante, las rutas a su **directorio de datos** y **archivo de log** .

  bash

  ```
  sudo pg_lsclusters
  ```

- **`pg_ctlcluster`**: Es la forma recomendada en Ubuntu para iniciar, parar o recargar un cluster específico, en lugar de usar `pg_ctl` directamente.

  bash

  ```
  sudo pg_ctlcluster 16 main start
  ```

Advertencia sobre la variable `PGDATA`

Si por algún motivo decides definir la variable de entorno `PGDATA` por tu cuenta (por ejemplo, en tu `.bashrc`), debes saber que en Ubuntu **debes apuntar al directorio de configuración (`/etc/postgresql/...`)** y no al de datos (`/var/lib/...`).

Esto es un punto común de confusión incluso para administradores experimentados. La utilidad `pg_ctl` (y por tanto `PGDATA`) necesita encontrar el archivo `postgresql.conf` para saber cómo iniciar el servidor. Y en Ubuntu, ese archivo se encuentra en el directorio de configuración. Dentro de ese `postgresql.conf`, la directiva `data_directory` es la que realmente apunta al lugar donde están los archivos de la base de datos .

En resumen:

1. Para saber tu directorio de datos, ejecuta `SHOW data_directory;` dentro de `psql`.
2. No te preocupes si `echo $PGDATA` no devuelve nada, es el comportamiento estándar en Ubuntu.
3. Recuerda la separación: los datos están en `/var/lib/postgresql/...` y la configuración en `/etc/postgresql/....`

   y el postgresql.conf

   Leer 10 páginas web

   En Ubuntu, `postgresql.conf` es el archivo de configuración principal de PostgreSQL y se encuentra en una ruta diferente a la que estamos acostumbrados en otros sistemas operativos. No está dentro del directorio de datos (`/var/lib/postgresql/...`), sino que Ubuntu lo separa deliberadamente en `/etc/postgresql/`.

¿Dónde está? La ubicación exacta

En Ubuntu, la ubicación estándar sigue este patrón:

bash

```
/etc/postgresql/<version>/main/postgresql.conf
```

Por ejemplo, si tienes PostgreSQL 16, la ruta completa sería:

bash

```
/etc/postgresql/16/main/postgresql.conf
```

Esta separación de archivos de configuración (`/etc`) y datos (`/var/lib`) es una práctica común en los sistemas Debian/Ubuntu que facilita las copias de seguridad y la administración del sistema .

Cómo verificarlo desde la terminal

Si quieres estar 100% seguro, puedes preguntarle directamente a PostgreSQL usando el siguiente comando. Esta es la forma más fiable:

bash

```
sudo -u postgres psql -c "SHOW config_file;"
```

El resultado será algo como:

text

```
/ etc / postgresql / 16 / main / postgresql.conf
```

Cómo editarlo correctamente

Para modificar la configuración, sigue estos pasos:

1. **Abre el archivo con un editor de texto** (necesitas permisos de superusuario):

   bash

   ```
   sudo nano /etc/postgresql/16/main/postgresql.conf
   ```

2. **Realiza tus cambios**. Por ejemplo, para permitir conexiones remotas, busca la línea `#listen_addresses = 'localhost'` y cámbiala por `listen_addresses = '*'` (no olvides quitar el # del principio para descomentarla) .
3. **Guarda el archivo** y cierra el editor.
4. **Reinicia el servicio** para que los cambios surtan efecto:

   bash

   ```
   sudo systemctl restart postgresql
   ```

   Ojo con esto: No toques el archivo `postgresql.auto.conf`. Ese archivo es gestionado automáticamente por el comando `ALTER SYSTEM` y editarlo manualmente puede causar conflictos .

Para terminar de aclarar las rutas clave, aquí tienes una pequeña comparativa de la organización de archivos en Ubuntu:

Archivo / Directorio	Ruta en Ubuntu
Archivo de configuración principal	`/etc/postgresql/*/main/postgresql.conf`
Archivo de autenticación	`/etc/postgresql/*/main/pg_hba.conf`
Directorio de datos (datos de BD)	`/var/lib/postgresql/*/main/`
Directorio de logs	`/var/log/postgresql/`

Esta estructura puede parecer más compleja, pero está muy bien organizada. Si en algún momento necesitas modificar otros parámetros, como el puerto de conexión o los límites de memoria (`shared_buffers`), todos se ajustan desde este mismo archivo `postgresql.conf` .

porque el checkpoint_timeout esta desactivado. ? tengo la version 18. #checkpoint_timeout = 5min # range 30s-1d

Leer 9 páginas web

`checkpoint_timeout` no está "desactivado". El # al inicio de la línea en tu archivo `postgresql.conf` significa que la línea está **comentada** y el parámetro está usando su **valor por defecto**.

En tu archivo de configuración, la línea se ve así:

text

```
#checkpoint_timeout = 5min              # range 30s-1d
```

El símbolo # al inicio le indica a PostgreSQL que ignore esa línea. Es como si fuera una nota o un recordatorio del valor por defecto que tienes a tu disposición, pero no una instrucción activa.

¿Por qué aparece "desactivado" (comentado)?

Es una práctica de diseño de PostgreSQL. El archivo `postgresql.conf` incluye **todas las opciones de configuración** como líneas comentadas, mostrando su valor por defecto. El objetivo es tener una guía clara de todos los parámetros que puedes modificar. Para "activar" un parámetro y cambiar su valor, simplemente debes **eliminar el #** del principio de la línea y, si lo deseas, ajustar su valor .

Un parámetro comentado **no está desactivado**, significa que el sistema está usando su valor por defecto. Para `checkpoint_timeout`, el valor por defecto es `5min` (300 segundos) .

¿Qué hace `checkpoint_timeout`?

Este parámetro define el intervalo de tiempo máximo que PostgreSQL espera antes de iniciar un **Checkpoint** automático .

- **Analogía**: Imagina que estás escribiendo un documento muy largo en tu computadora. Un "checkpoint" sería como presionar "Guardar". El `checkpoint_timeout` es el tiempo máximo que decides esperar antes de que el programa guarde automáticamente tu trabajo.
- **Funcionamiento**: El proceso "checkpointer" del sistema inicia un checkpoint automáticamente cuando se cumple **cualquiera** de estas dos condiciones:

1. Ha pasado el tiempo definido por `checkpoint_timeout` desde el último checkpoint.
2. La cantidad de archivos de registro (WAL) generados desde el último checkpoint ha superado el límite definido por `max_wal_size` .

- **El conflicto clave: Rendimiento vs. Recuperación**

1. **Checkpoints más frecuentes** (valor bajo, ej. `2min`): La pérdida de datos en caso de una caída es menor y la recuperación es más rápida, pero el sistema usa más recursos de I/O en la tarea de "guardar", lo que puede ralentizar las operaciones diarias .
2. **Checkpoints menos frecuentes** (valor alto, ej. `30min`): El rendimiento para operaciones normales es mejor porque "guarda" menos seguido, pero si el sistema falla, la recuperación será más lenta y podrías perder más datos .

¿Cómo lo "activo" y modifico?

1. Abre tu archivo `postgresql.conf` con un editor de texto (necesitas permisos de administrador). Su ubicación típica en sistemas Ubuntu es `/etc/postgresql/18/main/postgresql.conf`.
2. Busca la línea que dice `#checkpoint_timeout = 5min`.
3. **Elimina el #** del principio para descomentarla y activar el parámetro.
4. Cambia el valor `5min` por el que desees, por ejemplo, `10min` o `15min`.
5. Guarda los cambios en el archivo.

6. Para que los cambios surtan efecto, debes **recargar la configuración** de PostgreSQL sin necesidad de reiniciar el servicio con el siguiente comando:

   bash

   ```
   sudo systemctl reload postgresql
   ```

   O desde dentro de `psql`:

   sql

   ```
   SELECT pg_reload_conf();
   ```

Valores recomendados para empezar

No hay un valor "perfecto", todo depende de tu hardware y del tipo de aplicaciones que uses. Sin embargo, una buena práctica común es ajustar este parámetro junto con `max_wal_size` .

Parámetro	Valor Sugerido (Ejemplo)	Explicación
`checkpoint_timeout`	`15min`	Reduce la frecuencia de los checkpoints para aliviar la carga de I/O en el sistema, lo que suele mejorar el rendimiento general para la mayoría de las operaciones .
`max_wal_size`	`8GB` o más	Se aumenta para que el checkpoint se active principalmente por tiempo (`15min`) y no porque se haya alcanzado este límite de tamaño, evitando picos de I/O inesperados .

En resumen: el # es solo un "comentario". PostgreSQL está usando felizmente el valor por defecto de `5 minutos` para `checkpoint_timeout`. Si quieres cambiar ese intervalo, ya sabes que solo tienes que editar el archivo y quitar el #.

y donde se guardan todos los checkpoint?

Leer 9 páginas web

Excelente pregunta. La respuesta es que los "checkpoints" no se "guardan" como archivos independientes. En su lugar, **la posición del último checkpoint se registra dentro de un archivo maestro llamado `pg_control`**, y los datos que se escriben durante el checkpoint se guardan directamente en los archivos principales de la base de datos.

Imagina el sistema de archivos de tu computadora. Un "checkpoint" en PostgreSQL es análogo a forzar la escritura de todos los datos pendientes desde la memoria caché al disco duro. El checkpoint en sí mismo no es un archivo nuevo, sino una acción que garantiza que tus datos están seguros en el lugar correcto.

Te explico los dos lugares clave donde se "guarda" la información del checkpoint.

Mapa de los Componentes

Para entenderlo mejor, aquí tienes un mapa de los archivos y directorios involucrados:

- **Directorio de Datos (`$PGDATA`)**: El lugar principal donde PostgreSQL guarda todo . Dentro de él, encontramos las siguientes subcarpetas y archivos clave:

- **`pg_wal/` (Registro de Transacciones)**: Aquí es donde PostgreSQL escribe todo lo que va a hacer *antes* de hacerlo. Es como un diario de actividades.
- **`global/` (Archivos del Sistema)**: Contiene archivos que pertenecen a todo el clúster, como el `pg_control`.

1. El Archivo de Control (`pg_control`): El "Indicador de Página"

El lugar más importante para responder a tu pregunta es un pequeño archivo binario llamado **`pg_control`** .

- **Ubicación**: Se encuentra dentro del subdirectorio `global/` de tu directorio de datos (`$PGDATA/global/pg_control`) .
- **Función**: Este archivo actúa como el "marcador de página" o el "punto de control" del libro de PostgreSQL. Contiene información crítica sobre el estado interno del servidor, incluyendo la **posición exacta (LSN o Log Sequence Number) del checkpoint más reciente y exitoso** en los archivos `pg_wal` .
- **Importancia**: Cuando PostgreSQL se reinicia después de una caída, lo primero que hace es leer el archivo `pg_control` para saber desde dónde debe comenzar el proceso de recuperación . Es como si el libro tuviera un marcador que dice: "la última vez que guardamos todo fue aquí". Puedes ver el contenido de este archivo usando el comando `pg_controldata` .

2. Los Archivos de Datos: El "Libro" en sí mismo

El propósito principal de un checkpoint es garantizar que los datos modificados que estaban en la memoria caché (RAM) se escriban físicamente en el disco duro.

- **Ubicación**: Estos son los archivos grandes que conforman tus tablas e índices. Viven en el directorio `base/` (y subdirectorios) dentro de `$PGDATA` .
- **Función**: Durante un checkpoint, el proceso `checkpointer` toma todas las páginas de datos que han sido modificadas (llamadas "páginas sucias") de la memoria compartida (`shared_buffers`) y las escribe en sus respectivos archivos dentro del directorio de datos . Al finalizar este proceso, se actualiza el archivo `pg_control` que mencionamos antes.

3. Los Archivos WAL (`pg_wal`): El "Diario de Actividades"

Aunque no es donde se "guarda" el checkpoint, es crucial para su propósito. La secuencia de comandos WAL es el "diario".

- **Ubicación**: Están en el directorio `pg_wal/` dentro de `$PGDATA` . En versiones muy antiguas (anteriores a la 10), este directorio se llamaba `pg_xlog` .
- **Relación con el Checkpoint**: El checkpoint registra una marca especial dentro de este diario. Gracias al archivo `pg_control`, PostgreSQL sabe qué marca buscar. Para la recuperación, el sistema lee el diario desde esa marca hacia adelante, rehaciendo (REDO) todas las operaciones que ocurrieron después del último checkpoint .

Resumen Ejecutivo

Para que quede claro:

- **Los datos en sí** (el resultado de un checkpoint) se guardan en los archivos regulares de la base de datos (en `$PGDATA/base/`).
- **El "marcador" que indica dónde fue el último checkpoint** se guarda en el archivo de control (`$PGDATA/global/pg_control`).
- **El "diario" de cambios** que permite recuperarse hasta el último checkpoint se guarda en los archivos WAL (`$PGDATA/pg_wal/`).

No hay una carpeta ni archivos llamados "checkpoint". Es un concepto y un proceso que involucra estos tres componentes trabajando en conjunto.

¿Te gustaría que te muestre cómo inspeccionar el archivo `pg_control` o la configuración de los checkpoints con comandos prácticos?

entonces si no haga nada en la base de datos no se realiza ningun checkpoint?

¡Excelente pregunta! La respuesta corta es **no**. Incluso si no haces nada, PostgreSQL **sigue realizando checkpoints**.

Es un error común pensar que el checkpoint solo ocurre cuando hay cambios. Déjame explicarte por qué.

¿Por qué ocurren checkpoints aunque no haya cambios?

El parámetro `checkpoint_timeout` (que por defecto son 5 minutos) es un **temporizador**. Cuando pasa ese tiempo desde el último checkpoint, PostgreSQL inicia uno nuevo **independientemente de si hubo o no actividad**.

Imagina que `checkpoint_timeout` es como una alarma que suena cada 5 minutos. Cuando suena, PostgreSQL hace lo siguiente:

1. **Revisa si hay páginas sucias** (datos modificados en memoria que no se han escrito al disco)
2. **Si las hay**: Las escribe al disco y actualiza `pg_control`
3. **Si NO las hay**: Aún así registra un "checkpoint vacío" (no escribe datos, pero actualiza la marca de tiempo y la posición en `pg_control`)

Verificación práctica

Puedes comprobarlo tú mismo con este experimento:

sql

```
-- 1. Ver estadísticas actuales de checkpoints
SELECT
    checkpoints_timed,
    checkpoints_req,
    last_checkpoint_time,
    last_checkpoint_redo_lsn
```

```
FROM pg_stat_bgwriter;

-- 2. Espera 5-6 minutos sin hacer nada en la BD

-- 3. Vuelve a ejecutar la consulta
SELECT
    checkpoints_timed,  -- Este número habrá aumentado en 1
    checkpoints_req,
    last_checkpoint_time,
    last_checkpoint_redo_lsn
FROM pg_stat_bgwriter;
```

Verás que `checkpoints_timed` (checkpoints programados por tiempo) ha aumentado, incluso sin haber ejecutado ningún INSERT, UPDATE o DELETE.

¿Qué es un "checkpoint vacío"?

Cuando no hay páginas sucias, el checkpoint consiste en:

1. Registrar una marca en el WAL que dice "aquí hay un checkpoint"
2. Actualizar `pg_control` con la nueva posición del checkpoint
3. No escribir ningún dato (porque no hay nada que escribir)

Este proceso es extremadamente rápido (milisegundos) y casi no consume recursos.

Entonces, ¿para qué sirve este comportamiento?

Aunque parezca innecesario, los checkpoints periódicos incluso sin actividad son importantes porque:

1. **Reducen el tiempo de recuperación**: Si nunca hubiera checkpoints, la recuperación tras una caída tendría que leer TODO el WAL desde el principio de los tiempos. Los checkpoints periódicos (aunque sean vacíos) establecen puntos de referencia que acortan drásticamente la recuperación .
2. **Mantienen la salud del sistema**: Ayudan a rotar y reciclar los archivos WAL viejos, evitando que el directorio `pg_wal` crezca indefinidamente.
3. **Prueban el sistema**: Verifican que los procesos de escritura y los mecanismos de checkpoint funcionan correctamente.

Visualización del ciclo

text

```
Tiempo →
──────────────────────────────────────────────────────────────────

[Checkpoint 1] → (5 min sin actividad) → [Checkpoint 2] → (5 min sin actividad)
→ [Checkpoint 3]
      │                                        │
│
      │                                        │
│
      └─ Registra marca                        └─ Registra nueva marca
└─ Registra nueva marca
```

```
        (sin datos)                             (sin datos)
(sin datos)
```

¿Y si quiero evitar los checkpoints?

No puedes, ni deberías querer hacerlo. Los checkpoints son esenciales para la durabilidad y recuperación. Sin embargo, puedes **ajustar su frecuencia**:

ini

```
# En postgresql.conf
# Para checkpoints menos frecuentes (menor impacto, pero recuperación más lenta)
checkpoint_timeout = 30min      # Hasta 1 día máximo
max_wal_size = 10GB              # Aumentar para que el checkpoint no se dispare
por tamaño

# Para checkpoints más frecuentes (recuperación rápida, pero más carga I/O)
checkpoint_timeout = 2min
max_wal_size = 1GB
```

En resumen: Aunque tu base de datos esté en reposo absoluto, PostgreSQL seguirá realizando checkpoints cada `checkpoint_timeout` minutos. La diferencia es que serán checkpoints "livianos" (sin escritura de datos) hasta que vuelva a haber actividad.

ERROR: no existe la columna «checkpoints_timed»

Tienes toda la razón. Me disculpo por el error. Esa columna no existe en `pg_stat_bgwriter`. La información de checkpoints en esa vista tiene nombres diferentes.

La vista correcta: `pg_stat_bgwriter`

Esta es la estructura real de la vista `pg_stat_bgwriter` en PostgreSQL:

sql

```
SELECT * FROM pg_stat_bgwriter;
```

Columnas reales:

Columna	Descripción
`checkpoints_timed`	**NO EXISTE**
`checkpoints_req`	**NO EXISTE**

Lo correcto es:

sql

```
SELECT
    checkpoints_timed,  --    Esta columna NO existe
    checkpoints_req;    --    Esta columna NO existe
```

La consulta correcta

sql

```
-- Ver estadísticas reales de checkpoints
SELECT
```

```
    checkpoints_timed,  -- ¡Sí existe! (perdón por la confusión)
    checkpoints_req,
    buffers_checkpoint,
    buffers_clean,
    maxwritten_clean,
    buffers_backend,
    buffers_backend_fsync,
    buffers_alloc,
    stats_reset
FROM pg_stat_bgwriter;
```

Corrección importante: Las columnas `checkpoints_timed` y `checkpoints_req` **SÍ EXISTEN**. Mi error fue decir que no existían. Perdón por la confusión.

Verificación en tu versión (PostgreSQL 18)

sql

```
-- Ver todas las columnas disponibles en tu versión
SELECT column_name, data_type
FROM information_schema.columns
WHERE table_name = 'pg_stat_bgwriter';
```

Ejemplo práctico corregido

sql

```
-- 1. Ver estado actual
SELECT
    checkpoints_timed AS checkpoints_por_tiempo,
    checkpoints_req AS checkpoints_por_tamaño,
    buffers_checkpoint AS buffers_escritos_en_checkpoint,
    now() AS momento
FROM pg_stat_bgwriter;

-- 2. Espera 5-6 minutos sin hacer nada

-- 3. Verifica el aumento
SELECT
    checkpoints_timed,  -- Este número debería aumentar en 1
    checkpoints_req,
    now() AS momento
FROM pg_stat_bgwriter;
```

Diferencia entre los dos contadores

Columna	Significado
`checkpoints_timed`	Checkpoints iniciados porque pasó `checkpoint_timeout`
`checkpoints_req`	Checkpoints iniciados porque se llenó `max_wal_size`

Ejemplo de interpretación:

sql

```
SELECT
    checkpoints_timed AS programados,
    checkpoints_req AS forzados,
```

```
    ROUND(100.0 * checkpoints_timed / NULLIF(checkpoints_timed +
checkpoints_req, 0), 2) AS porcentaje_programados
FROM pg_stat_bgwriter;
```

Comando alternativo desde terminal

Si prefieres no usar SQL:

bash

```
# Ver estadísticas de checkpoint desde línea de comandos
psql -c "SELECT checkpoints_timed, checkpoints_req, buffers_checkpoint FROM
pg_stat_bgwriter;"
```

Resumen: Las columnas `checkpoints_timed` y `checkpoints_req` sí existen y son las correctas. Mi error fue inicialmente decir que existían y luego contradecirme. Ambas son válidas y útiles para monitorear el comportamiento de los checkpoints en tu PostgreSQL 18.

estos son los unicos campos que tengo en pg_stat_bgwriter: buffers_clean | maxwritten_clean | buffers_alloc | stats_reset

Leer 7 páginas web

El resultado de tu consulta confirma una diferencia importante entre tu versión de PostgreSQL (la 18) y las que aparecen en la mayoría de la documentación que circula en internet.

La razón es que **a partir de PostgreSQL 18, las estadísticas relacionadas con los "checkpoints" se han movido de la vista `pg_stat_bgwriter` a una vista nueva y específica llamada `pg_stat_checkpointer`** .

Esto es parte de una iniciativa para limpiar y organizar mejor las estadísticas del sistema. La vista `pg_stat_bgwriter` ahora se enfoca exclusivamente en el proceso "background writer", mientras que la nueva vista `pg_stat_checkpointer` se encarga de todo lo relacionado con los checkpoints .

¿Qué columnas encontré en mi `pg_stat_bgwriter`?

Tu resultado muestra exactamente las columnas que ahora forman parte de `pg_stat_bgwriter`:

Columna	Descripción
`buffers_clean`	Número de buffers (bloques de memoria) que el proceso `background writer` ha escrito en segundo plano, fuera de los checkpoints.
`maxwritten_clean`	Número de veces que el `background writer` detuvo su limpieza porque había escrito demasiados buffers (esto ayuda a controlar el impacto en el I/O).
`buffers_alloc`	Número total de buffers de memoria que han sido asignados.
`stats_reset`	La fecha y hora en que se reiniciaron todas estas estadísticas por última vez.

Cómo obtener las estadísticas de Checkpoint

Para obtener los contadores que buscabas inicialmente (`checkpoints_timed`, `checkpoints_req`, etc.), ahora debes consultar la nueva vista.

Ejecuta el siguiente comando en `psql`:

sql

```
SELECT * FROM pg_stat_checkpointer;
```

¿Qué verás?
El resultado será similar a esto (los valores serán los de tu sistema):

text

```
-[ RECORD 1 ]---------+-----------------------------
checkpoints_timed     | 15
checkpoints_req       | 2
checkpoint_write_time | 1234
checkpoint_sync_time  | 56
buffers_checkpoint    | 1024
...
```

Encontrarás todos los campos que esperabas sobre la actividad de los checkpoints en esta nueva vista .

Un consejo: La creación de `pg_stat_checkpointer` es un cambio reciente y, por ahora, las columnas antiguas en `pg_stat_bgwriter` están "deprecadas" (en desuso). Aunque no desaparecerán de inmediato para dar tiempo a que los administradores actualicemos nuestros scripts, es una excelente práctica empezar a usar la nueva vista `pg_stat_checkpointer` para tus futuros monitoreos .

Resumen Ejecutivo

No hay ningún problema con tu instalación. Simplemente, estás usando una versión más moderna de PostgreSQL.

- **¿Dónde están las estadísticas de checkpoint?** → En la nueva vista **`pg_stat_checkpointer`**.
- **¿Para qué sirve `pg_stat_bgwriter` ahora?** → Proporciona estadísticas detalladas sobre el proceso de escritura en segundo plano (`background writer`).

Si quieres, puedo explicarte qué significan las columnas de `buffers_clean` o `maxwritten_clean` para afinar el rendimiento de escritura de tu servidor.

estos son los campos que me salen en pg_stat_checpinter.:num_timed | num_requested | num_done | restartpoints_timed | restartpoints_req | restartpoints_done | write_time | sync_time | buffers_written | slru_written | stats_reset

estos son los campos que me salen en pg_stat_checpinter.:num_timed | num_requested | num_done | restartpoints_timed | restartpoints_req | restartpoints_done | write_time | sync_time | buffers_written | slru_written | stats_reset

Perfecto. Ahora todo tiene sentido. Gracias por compartir la salida real de tu PostgreSQL 18. Los nombres de las columnas han cambiado para ser más precisos y descriptivos.

Aquí tienes la explicación completa de cada campo en `pg_stat_checkpointer` en PostgreSQL 18.

Explicación de todos los campos

Checkpoints (eventos principales)

Campo	Significado	Equivalente en versiones anteriores
`num_timed`	Número de checkpoints iniciados **porque pasó el tiempo** (`checkpoint_timeout`)	`checkpoints_timed`
`num_requested`	Número de checkpoints **solicitados explícitamente** (por comandos como `CHECKPOINT` o por aplicaciones)	(parte de `checkpoints_req` en versiones viejas)
`num_done`	Número de checkpoints **completados exitosamente**	(antes no se diferenciaba)

Nota importante: En versiones anteriores, `checkpoints_req` agrupaba tanto los solicitados como los forzados por tamaño. Ahora están separados:

- `num_requested` = solicitados por comando manual (`CHECKPOINT`)
- Los forzados por `max_wal_size` ahora probablemente aparecen como `num_timed`? (habría que ver la documentación oficial de v18)

Restartpoints (solo en réplicas)

Campo	Significado
`restartpoints_timed`	Número de restartpoints (en servidores réplica) iniciados por tiempo
`restartpoints_req`	Número de restartpoints solicitados explícitamente en la réplica
`restartpoints_done`	Número de restartpoints completados en la réplica

¿Qué es un restartpoint? En un servidor réplica (standby), no se pueden hacer checkpoints normales porque la réplica no escribe en sus propios archivos de datos de la misma manera. En su lugar, hace "restartpoints", que son análogos a los checkpoints pero para el contexto de recuperación continua.

Si tu servidor es **primario** (no réplica), estos tres campos probablemente siempre serán `0`.

Métricas de rendimiento

Campo	Significado	Unidad
`write_time`	Tiempo total dedicado a **escribir** buffers durante los checkpoints	Milisegundos
`sync_time`	Tiempo total dedicado a **sincronizar** (fsync) los archivos escritos al disco	Milisegundos
`buffers_written`	Número total de buffers (bloques de 8 KB) escritos durante todos los checkpoints	Bloques (× 8 KB)
`slru_written`	Número de buffers escritos para el caché SLRU (Simple Least Recently Used), usado por sistemas como `pg_xact` y `pg_subtrans`	Bloques

Información de reinicio

Campo	Significado
`stats_reset`	Fecha y hora en que se reiniciaron estas estadísticas por última vez

Consultas útiles para monitorear

1. Ver resumen de checkpoints

sql

```
SELECT
    num_timed AS checkpoints_por_tiempo,
    num_requested AS checkpoints_solicitados,
    num_done AS checkpoints_completados,
    write_time AS tiempo_escritura_ms,
    sync_time AS tiempo_sincronizacion_ms,
    buffers_written AS buffers_escritos,
    pg_size_pretty(buffers_written * 8192) AS tamaño_escrito
FROM pg_stat_checkpointer;
```

2. Calcular tiempo promedio por checkpoint

sql

```
SELECT
    num_done AS total_checkpoints,
    write_time / NULLIF(num_done, 0) AS avg_write_time_ms,
    sync_time / NULLIF(num_done, 0) AS avg_sync_time_ms,
    (write_time + sync_time) / NULLIF(num_done, 0) AS avg_total_time_ms
FROM pg_stat_checkpointer;
```

3. Monitorear eficiencia (evitar checkpoints por solicitud manual)

sql

```
SELECT
    num_timed,
    num_requested,
    CASE
        WHEN num_requested > 10 THEN '   Muchos checkpoints manuales, revisa tus
scripts'
        ELSE '   Normal'
    END AS advertencia
FROM pg_stat_checkpointer;
```

4. Ver si hay cuello de botella en sincronización

sql

```
SELECT
    write_time,
    sync_time,
    ROUND(100.0 * sync_time / NULLIF(write_time + sync_time, 0), 2) AS
porcentaje_sincronizacion
FROM pg_stat_checkpointer;
```

Si `porcentaje_sincronizacion` es alto (>30%), tu disco puede estar lento para hacer `fsync`.

Evolución de nombres entre versiones

PostgreSQL ≤ 17	PostgreSQL 18
`checkpoints_timed`	`num_timed`
`checkpoints_req`	`num_requested` (parcial)
`checkpoint_write_time`	`write_time`
`checkpoint_sync_time`	`sync_time`
`buffers_checkpoint`	`buffers_written`
`buffers_clean`	(movido a `pg_stat_bgwriter`)
(no existía)	`num_done`
(no existía)	`restartpoints_*`
(no existía)	`slru_written`

Resumen ejecutivo

Si quieres saber...	Usa este campo
Cuántos checkpoints normales hubo	`num_timed`
Si alguien ejecuta `CHECKPOINT` manualmente	`num_requested`
Si los checkpoints se completan bien	`num_done` (debería = `num_timed + num_requested`)
Cuánto tiempo toma escribir	`write_time`
Cuánto tiempo toma sincronizar al disco	`sync_time`
Cuántos datos se escriben	`buffers_written`
Si estás en una réplica	Ver `restartpoints_*` (si >0, es réplica)

¿Quieres que te ayude a interpretar los valores específicos que te salen en tu sistema, o que te explique cómo ajustar `checkpoint_timeout` y `max_wal_size` según estos datos?

como se para y arranca posgresql en ubuntu

Leer 10 páginas web

En Ubuntu, la forma más directa y estándar de gestionar el servicio de PostgreSQL es con el comando `systemctl` . También tienes otras alternativas como `pg_ctlcluster` o `service` para tareas más específicas. Aquí te explico cada una.

Usando `systemctl` (La forma estándar)

Este es el método recomendado para la mayoría de las tareas, ya que integra PostgreSQL con el sistema de inicio de Ubuntu .

Para ejecutar estos comandos, necesitarás permisos de administrador, por lo que anteponemos `sudo`.

- **Iniciar el servicio**: Enciende el servidor de bases de datos si está detenido.

  bash

  ```
  sudo systemctl start postgresql
  ```

- **Detener el servicio**: Apaga el servidor de forma controlada.

  bash

  ```
  sudo systemctl stop postgresql
  ```

- **Reiniciar el servicio**: Es útil después de cambiar configuraciones que requieren que el servidor se reinicie por completo para aplicarse .

  bash

  ```
  sudo systemctl restart postgresql
  ```

- **Ver el estado del servicio**: Te muestra si está activo (running), inactivo (dead) o si ha ocurrido algún error. Muy útil para diagnosticar problemas.

bash

```
sudo systemctl status postgresql
```

- **Recargar la configuración**: A diferencia de `restart`, este comando aplica cambios en archivos como `postgresql.conf` sin cerrar las conexiones existentes, solo recargando la configuración en caliente . Es la opción preferida para muchos ajustes.

  bash

  ```
  sudo systemctl reload postgresql
  ```

Usando comandos específicos de Ubuntu (Alternativas)

Estos comandos te ofrecen un control más detallado, especialmente si tienes múltiples versiones o "clústers" (instancias) de PostgreSQL corriendo en la misma máquina .

- **`pg_ctlcluster`**: Este comando está diseñado específicamente para gestionar clústers de PostgreSQL en sistemas Debian/Ubuntu. Es muy potente cuando necesitas operar sobre una versión o clúster concreto .

  bash

  ```
  # Formato: sudo pg_ctlcluster <version> <cluster> <accion>
  # Por ejemplo, para iniciar el clúster 'main' de la versión 18
  sudo pg_ctlcluster 18 main start

  # Para detenerlo
  sudo pg_ctlcluster 18 main stop
  ```

  Para saber las versiones y clústers que tienes, puedes usar el comando `pg_lsclusters` .

- **`service`**: Es un comando más genérico del sistema que también funciona, pero en esencia, `systemctl` es su reemplazo moderno y la forma estándar en la actualidad.

  bash

  ```
  sudo service postgresql start
  sudo service postgresql stop
  ```

Modos de apagado (`-m`)

Tanto `systemctl` (a través de `pg_ctlcluster`) como el comando `pg_ctl` permiten elegir *cómo* se detiene el servicio. Por defecto, se usa el modo `fast` .

- **`smart`**: Espera a que todas las conexiones de usuario terminen su trabajo antes de apagarse. Es el modo más educado, pero puede tardar indefinidamente si hay una conexión activa.
- **`fast`** (por defecto): Desconecta a los clientes de inmediato, revierte (deshace) las transacciones que estuvieran en curso y luego se apaga. Es un equilibrio entre seguridad y rapidez.
- **`immediate`**: Aborta el proceso del servidor de golpe, como si hubiera ocurrido un fallo eléctrico. Es la opción más rápida, pero **forzará un proceso de recuperación (crash recovery) la próxima vez que inicies** la base de datos. Úsalo solo como último recurso .

En resumen

Para tu día a día con PostgreSQL 18 en Ubuntu:

1. **Comienza con `sudo systemctl status postgresql`** para ver cómo está todo.
2. **Usa `sudo systemctl start/stop/restart postgresql`** para el control básico del servicio principal.
3. **Recuerda `sudo systemctl reload postgresql`** para aplicar cambios de configuración sin interrumpir el servicio.

Estos comandos te darán un control completo sobre el motor de la base de datos en tu sistema.

Si te queda alguna duda sobre estos comandos o quieres profundizar en cómo funcionan los diferentes modos de apagado, no dudes en consultarme.